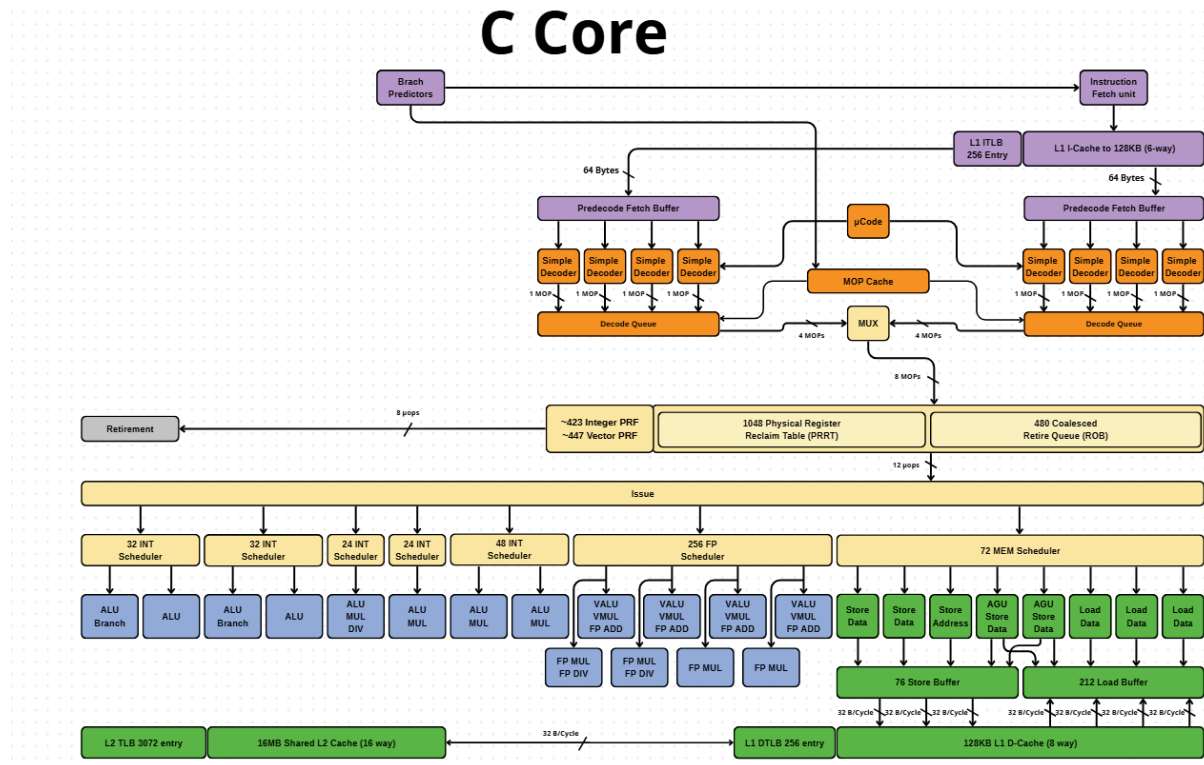


# CPU MicroArchitecture

## I. C core



## GIẢI THÍCH VI KIẾN TRÚC C CORE

### Dành Cho Người Không Chuyên

### MỤC LỤC

1. Giới Thiệu
2. Tổng Quan Quy Trình Xử Lý
3. Các Thành Phần Chi Tiết
4. Hệ Thống Bộ Nhớ
5. Retirement - Giai Đoạn Hoàn Thành
6. Kiến Trúc Out-of-Order Execution

7. Con Số Ẩn Tượng
8. So Sánh Với Thực Tế
9. Quy Trình Xử Lý Một Lệnh
10. Bandwidth - Bảng Thông Dữ Liệu

## 1. GIỚI THIỆU

Vi kiến trúc (microarchitecture) là cách mà một bộ xử lý (CPU) được thiết kế và tổ chức bên trong để xử lý các lệnh máy tính. Hãy tưởng tượng nó giống như bản thiết kế chi tiết của một nhà máy sản xuất, nơi mỗi bộ phận có vai trò riêng để hoàn thành công việc một cách hiệu quả nhất.

Tài liệu này sẽ giải thích sơ đồ vi kiến trúc **C Core** - một bộ xử lý hiện đại với khả năng xử lý mạnh mẽ.

## 2. TỔNG QUAN QUY TRÌNH XỬ LÝ

Khi bộ xử lý làm việc, nó thực hiện một quy trình gồm nhiều bước tuần tự. Hãy tưởng tượng như một dây chuyền lắp ráp:

1. **Lấy lệnh (Fetch):** Lấy chương trình từ bộ nhớ
2. **Giải mã (Decode):** Hiểu lệnh đó có nghĩa là gì
3. **Phát hành (Issue):** Chuẩn bị thực hiện
4. **Thực thi (Execute):** Thực sự làm việc
5. **Hoàn thành (Retire):** Ghi nhận kết quả

## 3. CÁC THÀNH PHẦN CHI TIẾT

### 3.1. Branch Predictors - Bộ Dự Đoán Nhánh

**Chức năng:** Dự đoán chương trình sẽ chạy theo hướng nào tiếp theo. **Ví dụ thực tế:**

Giống như khi bạn đi đến một ngã tư quen thuộc, bạn đã biết trước mình sẽ rẽ phải vì đó là đường bạn thường đi. Bộ dự đoán nhánh giúp CPU "đoán" trước để tiết kiệm thời gian. **Tại sao quan trọng:** Nếu đoán đúng, CPU tiết kiệm được nhiều thời gian. Nếu đoán sai, CPU phải quay lại và làm lại, tốn thời gian hơn.

## 3.2. Instruction Fetch Unit - Bộ Lấy Lệnh

Thành phần:

- **L1 ITLB (256 Entry):** Bảng tra cứu nhanh địa chỉ lệnh
- **L1 I-Cache (128KB, 6-way):** Bộ nhớ đệm lưu các lệnh gần đây

**Ví dụ thực tế:** Giống như bạn giữ những cuốn sách hay dùng nhất trên bàn làm việc thay vì phải đi tìm trong tủ sách mỗi lần cần. L1 I-Cache lưu các lệnh được dùng thường xuyên để truy cập nhanh hơn. **Kích thước 128KB:** Có thể chứa khoảng 30.000+ lệnh máy tính.

## 3.3. Predecode Fetch Buffer & Simple Decoders

**Cấu trúc:** Có 2 bộ đệm, mỗi bên có 4 bộ giải mã đơn giản. **Chức năng:** Nhận 64 bytes lệnh một lúc và phân tích sơ bộ thành các micro-operations ( $\mu$ OPs). **Ví dụ thực tế:** Giống như một đội nhân viên ở bưu điện, khi nhận một kiện hàng lớn, họ phân loại thành các gói nhỏ hơn để dễ xử lý. Mỗi "Simple Decoder" xử lý 1 lệnh đơn giản mỗi lần, tạo ra 1  $\mu$ OP. **Thông lượng:** 8 bộ giải mã có thể xử lý tối đa 8 lệnh đơn giản cùng lúc.

## 3.4. $\mu$ Code & MOP Cache

**$\mu$ Code (Microcode):** Dùng cho các lệnh phức tạp cần nhiều bước. **MOP Cache:** Lưu trữ các chuỗi  $\mu$ OPs đã giải mã trước đó. **Ví dụ thực tế:** Tưởng tượng bạn có một công thức nấu ăn phức tạp. Lần đầu bạn phải đọc kỹ từng bước ( $\mu$ Code), nhưng lần sau bạn nhớ rồi nên làm nhanh hơn (MOP Cache). **Lợi ích:** Tiết kiệm năng lượng và thời gian vì không phải giải mã lại các lệnh đã xử lý.

## 3.5. Decode Queue - Hàng Đợi Giải Mã

**Chức năng:** Nơi tập trung các  $\mu$ OPs sau khi giải mã, trước khi gửi đến bộ MUX. **Luồng dữ liệu:**

- Nhận 4  $\mu$ OPs từ mỗi Decode Queue
- MUX kết hợp thành 8  $\mu$ OPs
- Gửi xuống các Scheduler

**Ví dụ thực tế:** Giống như khu vực tập kết hàng hóa trước khi phân phối đến các bộ phận khác nhau trong nhà máy.

### 3.6. Physical Register Files - Bộ Thanh Ghi Vật Lý

Cấu trúc:

- **Integer PRF:** ~423 thanh ghi cho số nguyên
- **Vector PRF:** ~447 thanh ghi cho vector và số thực

**Chức năng:** Lưu trữ tạm thời dữ liệu trong khi xử lý. **Ví dụ thực tế:** Giống như bàn làm việc của bạn - nơi bạn đặt các giấy tờ đang xử lý. Có nhiều thanh ghi giúp CPU làm nhiều việc cùng lúc mà không bị xung đột.

### 3.7. Physical Register Reclaim Table (PRRT)

**Kích thước:** 1048 entries **Chức năng:** Theo dõi thanh ghi nào đang được dùng, thanh ghi nào có thể tái sử dụng. **Ví dụ thực tế:** Như một người quản lý bãi đỗ xe, biết chỗ nào đang trống, chỗ nào đã có xe, để sắp xếp hiệu quả.

### 3.8. Retire Queue (ROQ) - Hàng Đợi Hoàn Thành

**Kích thước:** 480 coalesced entries **Chức năng:** Đảm bảo các lệnh hoàn thành đúng thứ tự, dù có thể được thực thi không theo thứ tự. **Ví dụ thực tế:** Giống như một nhà hàng buffet - khách có thể lấy món theo thứ tự bất kỳ, nhưng khi thanh toán phải xếp hàng đúng thứ tự.

**Con số 480:** Cho phép CPU "nhìn xa" 480 lệnh trong tương lai, tìm cơ hội thực thi song song.

### 3.9. Schedulers - Bộ Lập Lịch

Đây là "bộ não" phân công công việc. C Core có nhiều loại scheduler:

#### 3.9.1. 32 INT Scheduler (x2)

**Chức năng:** Lập lịch cho các phép tính số nguyên đơn giản. **Execution Units:** Mỗi scheduler điều khiển:

- **ALU Branch:** Xử lý phép tính và nhánh
- **ALU:** Phép tính số học cơ bản (cộng, trừ, AND, OR, XOR)

#### 3.9.2. 24 INT Scheduler (x2)

**Chức năng:** Lập lịch cho phép tính số nguyên phức tạp hơn. **Execution Units:**

- **ALU MUL DIV:** Nhân và chia số nguyên

**Lưu ý:** Nhân và chia phức tạp hơn cộng/trừ nên cần bộ xử lý riêng.

### 3.9.3. 48 INT Scheduler

**Kích thước lớn hơn:** Có 48 entry, xử lý khối lượng lớn. **Execution Units:**

- **ALU MUL:** Tập trung vào phép nhân

### 3.9.4. 256 FP Scheduler

**Kích thước:** Lớn nhất với 256 entries **Chức năng:** Lập lịch cho các phép tính số thực (floating-point) và vector. **Execution Units:**

- **VALU VMUL FP ADD:** Phép cộng vector và số thực
- **FP MUL:** Phép nhân số thực
- **FP DIV:** Phép chia số thực (chậm nhất)

**Ví dụ thực tế:** Số thực dùng trong đồ họa 3D, AI, khoa học. Ví dụ: tính toán vị trí của nhân vật trong game. **Tại sao lớn:** Các phép tính số thực thường phức tạp, mất nhiều chu kỳ, nên cần hàng đợi lớn để chứa nhiều lệnh chờ.

### 3.9.5. 72 MEM Scheduler

**Chức năng:** Quản lý truy cập bộ nhớ (đọc/ghi dữ liệu). **Execution Units:**

- **AGU Store Data/Address:** Tính địa chỉ và chuẩn bị dữ liệu ghi
- **Load Data:** Đọc dữ liệu từ bộ nhớ

**Buffers:**

- **Store Buffer:** 76 entries (32 B/cycle x3)
- **Load Buffer:** 212 entries (32 B/cycle x3)

**Ví dụ thực tế:** Giống như kho hàng có 2 khu vực: một để nhận hàng vào (Load), một để xuất hàng ra (Store).

## 4. HỆ THỐNG BỘ NHỚ

## 4.1. Cache Hierarchy - Phân Cấp Bộ Nhớ Đệm

CPU không thể truy cập RAM trực tiếp vì quá chậm. Thay vào đó, nó dùng hệ thống cache nhiều tầng:

### 4.1.1. L1 Cache - Cấp 1 (Nhanh Nhất)

#### L1 I-Cache (Instruction):

- Kích thước: 128KB
- Tổ chức: 6-way set associative
- Chức năng: Lưu lệnh chương trình

#### L1 D-Cache (Data):

- Kích thước: 128KB
- Tổ chức: 8-way set associative
- Chức năng: Lưu dữ liệu
- Băng thông:  $32 \text{ B/cycle} \times 3 = 96 \text{ bytes/cycle}$

**Ví dụ thực tế:** Giống như ngăn kéo bàn làm việc - nhỏ nhưng lấy rất nhanh.

### 4.1.2. L2 Cache - Cấp 2 (Lớn Hơn, Chậm Hơn)

- Kích thước: 16MB (Shared - dùng chung)
- Tổ chức: 16-way set associative
- Băng thông: 32 B/cycle

**Ví dụ thực tế:** Giống như tủ tài liệu trong phòng - lớn hơn ngăn kéo nhưng cũng gần, lấy khá nhanh. **Shared:** Nhiều CPU core dùng chung L2, giúp chia sẻ dữ liệu hiệu quả.

## 4.2. TLB - Translation Lookaside Buffer

**Chức năng:** Chuyển đổi địa chỉ ảo (virtual) sang địa chỉ vật lý (physical) nhanh chóng. **Cấu trúc:**

- **L1 ITLB:** 256 entries (cho lệnh)
- **L1 DTLB:** 256 entries (cho dữ liệu)
- **L2 TLB:** 3072 entries (dùng chung)

**Ví dụ thực tế:** Giống như danh bạ điện thoại - thay vì nhớ số điện thoại thực, bạn chỉ cần nhớ tên người (địa chỉ ảo), và danh bạ sẽ cho bạn số thật (địa chỉ vật lý). **Tại sao cần:** Mỗi chương trình nghĩ mình dùng toàn bộ bộ nhớ, nhưng thực tế OS phải quản lý và phân chia. TLB giúp quá trình này nhanh hơn.

## 5. RETIREMENT - GIAI ĐOẠN HOÀN THÀNH

**Chức năng:** Giai đoạn cuối cùng, nơi kết quả được chính thức "cam kết" (commit). **Thông lượng:** 8  $\mu$ OPs mỗi chu kỳ (8 pops) **Vai trò quan trọng:**

1. Đảm bảo lệnh hoàn thành đúng thứ tự chương trình
2. Xử lý ngoại lệ (exception)
3. Giải phóng tài nguyên (thanh ghi, buffers)
4. Cập nhật trạng thái kiến trúc

**Ví dụ thực tế:** Giống như việc một giáo viên chấm bài - học sinh có thể làm bài không theo thứ tự câu hỏi, nhưng giáo viên phải chấm và ghi điểm theo thứ tự để đảm bảo công bằng.

## 6. KIẾN TRÚC OUT-OF-ORDER EXECUTION

### 6.1. Khái Niệm

C Core sử dụng kiến trúc **thực thi không theo thứ tự** (out-of-order execution).

**Ý nghĩa:** CPU có thể thực hiện lệnh không đúng thứ tự chương trình viết, miễn là kết quả cuối cùng vẫn đúng. **Ví dụ thực tế:** *Chương trình gốc:*

1.  $A = B + C$
2.  $D = E + F$
3.  $G = A + D$

CPU nhận ra lệnh 1 và 2 không phụ thuộc nhau, nên có thể làm song song. Chỉ lệnh 3 phải chờ lệnh 1 và 2 xong.

### 6.2. Lợi Ích

- **Tăng hiệu suất:** Tận dụng tối đa các execution unit

- **Ẩn độ trễ:** Khi một lệnh chờ dữ liệu từ bộ nhớ, CPU làm lệnh khác
- **Throughput cao:** Có thể hoàn thành nhiều lệnh trong cùng thời gian

### 6.3. Thách Thức

- **Phức tạp:** Cần nhiều logic theo dõi phụ thuộc (dependency)
- **Tiêu thụ năng lượng:** Các buffer và scheduler lớn tốn điện
- **Misprediction penalty:** Nếu dự đoán sai, phải hủy nhiều lệnh

## 7. CON SỐ ẢN TƯỢNG

Hãy cùng xem các con số quan trọng của C Core:

| Thành phần    | Con số   | Ý nghĩa                      |
|---------------|----------|------------------------------|
| Decoders      | 8        | Giải mã 8 lệnh/chu kỳ        |
| Integer PRF   | ~423     | Lưu trữ 423 số nguyên tạm    |
| Vector PRF    | ~447     | Lưu trữ 447 vector/float tạm |
| ROQ           | 480      | Theo dõi 480 lệnh đang bay   |
| PRRT          | 1048     | Quản lý 1048 thanh ghi       |
| FP Scheduler  | 256      | Lập lịch 256 lệnh float      |
| MEM Scheduler | 72       | Lập lịch 72 lệnh memory      |
| Store Buffer  | 76       | Chờ ghi 76 lần               |
| Load Buffer   | 212      | Chờ đọc 212 lần              |
| L1 Cache      | 128KB x2 | Bộ nhớ đệm nhanh nhất        |
| L2 Cache      | 16MB     | Bộ nhớ đệm lớn chia sẻ       |



|              |             |                          |
|--------------|-------------|--------------------------|
| L2 TLB       | 3072        | Tra cứu 3072 địa chỉ     |
| Retire Width | 8 $\mu$ OPs | Hoàn thành 8 lệnh/chu kỳ |

## 8. SO SÁNH VỚI THỰC TẾ

### 8.1. Giống Như Một Nhà Máy Hiện Đại

- **Branch Predictors:** Phòng dự báo thị trường
- **Fetch & Decode:** Bộ phận nhận đơn hàng và phân tích
- **Schedulers:** Phòng kế hoạch sản xuất
- **Execution Units:** Các dây chuyền sản xuất chuyên môn
- **Caches:** Kho hàng gần dây chuyền
- **Retirement:** Bộ phận kiểm tra chất lượng và đóng gói

### 8.2. Tại Sao Cần Phức Tạp Như Vậy?

**1. Tốc độ:** Chương trình hiện đại yêu cầu tính toán cực nhanh (game, AI, video 4K) **2. Đa nhiệm:** Máy tính chạy hàng trăm chương trình cùng lúc **3. Hiệu quả năng lượng:** Làm nhiều việc trong ít chu kỳ = tiết kiệm điện **4. Che giấu độ trễ:** RAM chậm (100+ chu kỳ), CPU cần làm gì đó trong lúc chờ

## 9. QUY TRÌNH XỬ LÝ MỘT LỆNH

Hãy theo dõi một lệnh đơn giản:  $C = A + B$

### Bước 1: Fetch

1. Branch Predictor dự đoán lệnh tiếp theo
2. Fetch Unit kiểm tra L1 I-Cache
3. Nếu có (cache hit): lấy ngay (1-2 chu kỳ)
4. Nếu không (cache miss): lấy từ L2 (~15 chu kỳ) hoặc RAM (~100 chu kỳ)

### Bước 2: Decode

1. Lệnh vào Predecode Fetch Buffer
2. Simple Decoder phân tích: "Cộng 2 số nguyên"
3. Tạo 1  $\mu$ OP: ADD\_INT R\_C, R\_A, R\_B
4. Gửi vào Decode Queue

### Bước 3: Rename & Dispatch

1. Tra PRRT: tìm thanh ghi vật lý cho A, B, C
2. Giả sử: A = P45, B = P67, C = P89
3.  $\mu$ OP trở thành: ADD P89, P45, P67
4. Gửi đến 32-INT Scheduler phù hợp

### Bước 4: Schedule

1. Scheduler kiểm tra: P45 và P67 có sẵn chưa?
2. Nếu có: chọn lệnh này để thực thi
3. Gửi đến ALU rảnh

### Bước 5: Execute

1. ALU đọc giá trị từ P45 và P67
2. Thực hiện phép cộng (1 chu kỳ)
3. Ghi kết quả vào P89
4. Đánh dấu P89 "sẵn sàng" cho lệnh khác dùng

### Bước 6: Retire

1. Lệnh vào đầu ROQ (đúng thứ tự)
2. Kiểm tra không có exception
3. Commit kết quả
4. Giải phóng thanh ghi cũ của C (nếu có)
5. Lệnh biến mất khỏi pipeline

**Tổng thời gian lý tưởng:** Khoảng 15-20 chu kỳ từ đầu đến cuối. **Nhưng:** Nhờ pipeline và out-of-order, CPU có thể hoàn thành nhiều lệnh khác trong khoảng thời gian này, đạt throughput cao (8  $\mu$ OPs/chu kỳ).

## 10. BANDWIDTH - BẢNG THÔNG DỮ LIỆU

Quan sát các con số **32 B/Cycle** trong diagram:

### 10.1. Ý Nghĩa

- **32 B/Cycle = 32 bytes mỗi chu kỳ clock**
- Nếu CPU chạy 3 GHz (3 tỷ chu kỳ/giây):
- Băng thông =  $32 \times 3 \times 10^9 = 96 \text{ GB/s}$  (cho mỗi đường)

### 10.2. Nhiều Đường Song Song

- Load Buffer: 3 đường x 32 B/cycle = 96 B/cycle đọc
- Store Buffer: 3 đường x 32 B/cycle = 96 B/cycle ghi
- **Tổng cộng:** ~288 GB/s đọc + 288 GB/s ghi (lý thuyết)

**Ví dụ thực tế:** Đủ để xem 100+ video 4K cùng lúc (nếu không có bottleneck khác).

## KẾT LUẬN

Vi kiến trúc C Core là một thiết kế phức tạp nhưng cực kỳ hiệu quả, tối ưu hóa cho hiệu suất cao. Với các thành phần như:

- **8 bộ giải mã** xử lý nhiều lệnh song song
- **ROQ 480 entries** cho phép nhìn xa trong chương trình
- **FP Scheduler 256 entries** mạnh mẽ cho tính toán khoa học
- **Cache lớn** (128KB L1 + 16MB L2) giảm độ trễ bộ nhớ
- **Out-of-order execution** tận dụng tối đa tài nguyên

C Core đại diện cho sự cân bằng giữa hiệu suất, hiệu quả năng lượng và tính linh hoạt trong thiết kế CPU hiện đại. *Mỗi lần bạn click chuột, gõ phím, hay xem video, hàng tỷ transistor trong CPU đang làm việc theo đúng sơ đồ này - một kỳ tích kỹ thuật vô hình nhưng cực kỳ mạnh mẽ.*

[illegible]

# MỤC LỤC

- ## 1. GIỚI THIỆU

**P Core** (Performance Core) là một lõi xử lý hiệu suất cao được thiết kế để xử lý các tác vụ đòi hỏi sức mạnh tính toán lớn. Đây là một bộ xử lý thuộc kiến trúc ARM với khả năng thực thi không theo thứ tự (out-of-order execution).

Vi kiến trúc này được tối ưu hóa để đạt được throughput cao - khả năng xử lý nhiều lệnh trong cùng một khoảng thời gian.

## 2. TỔNG QUAN QUY TRÌNH XỬ LÝ

P Core thực hiện quy trình xử lý gồm các giai đoạn chính:

1. **Fetch (Lấy lệnh):** Lấy các lệnh chương trình từ bộ nhớ
2. **Decode (Giải mã):** Phân tích và chuyển đổi lệnh thành micro-operations
3. **Issue (Phát hành):** Phân phối các  $\mu$ OPs đến các bộ lập lịch
4. **Execute (Thực thi):** Thực hiện các phép tính thực sự
5. **Retire (Hoàn thành):** Cam kết kết quả theo đúng thứ tự chương trình

**Đặc điểm:** P Core có thể thực thi các lệnh không theo thứ tự ban đầu, nhưng vẫn đảm bảo kết quả cuối cùng đúng như chương trình yêu cầu.

## 3. CÁC THÀNH PHẦN CHI TIẾT - PHẦN FRONTEND

### 3.1. Branch Predictors - Bộ Dự Đoán Nhánh

**Vị trí:** Ở đầu pipeline, trước Instruction Fetch Unit. **Chức năng:** Dự đoán hướng đi của chương trình khi gặp các câu lệnh rẽ nhánh (if/else, loops). **Tại sao quan trọng:**

- Nếu dự đoán đúng: CPU tiếp tục xử lý mà không bị gián đoạn
- Nếu dự đoán sai: CPU phải hủy các lệnh đã xử lý và bắt đầu lại từ nhánh đúng, tốn khoảng 15-20 chu kỳ

**Ví dụ thực tế:** Giống như khi bạn đến một ngã ba đường quen thuộc, bạn tự động rẽ theo hướng thường đi mà không cần suy nghĩ. Branch predictor giúp CPU "học" được pattern của chương trình.

### 3.2. Instruction Fetch Unit - Bộ Lấy Lệnh

### Thành phần: L1 ITLB (384 Entry):

- ITLB = Instruction Translation Lookaside Buffer
- 384 entries có thể tra cứu địa chỉ của 384 trang bộ nhớ
- Chuyển đổi địa chỉ ảo (virtual address) sang địa chỉ vật lý (physical address)

### L1 I-Cache (192KB, 8-way):

- I-Cache = Instruction Cache
- Kích thước: 192KB
- Tổ chức: 8-way set associative
- Chứa khoảng 45,000-50,000 lệnh máy
- Độ trễ truy cập: 4-5 chu kỳ

**Bảng thông:** 64 Bytes mỗi chu kỳ được lấy từ cache. **Ví dụ thực tế:** L1 I-Cache giống như một quyển sổ tay nhỏ nơi bạn ghi các công thức hay dùng. Thay vì phải lật giáo trình dày (RAM), bạn chỉ cần xem sổ tay (cache) để lấy thông tin nhanh hơn.

## 3.3. Predecode Fetch Buffer & Simple Decoders

**Cấu trúc:** Có 2 Predecode Fetch Buffer, mỗi bên có 6 Simple Decoders. **Luồng xử lý:**

- Mỗi buffer nhận 64 bytes lệnh từ I-Cache
- 6 Simple Decoders phân tích các lệnh đơn giản
- Mỗi decoder tạo ra 1  $\mu$ OP (micro-operation)

**Thông lượng tối đa:**  $6 + 6 = 12$  lệnh đơn giản mỗi chu kỳ **Micro-operations ( $\mu$ OPs):** Là các lệnh nhỏ, đơn giản mà CPU thực sự thực thi. Một lệnh phức tạp của chương trình có thể được chia thành nhiều  $\mu$ OPs. **Ví dụ thực tế:** Giống như một nhà hàng có 12 đầu bếp phụ, mỗi người xử lý một món đơn giản. Nếu có món phức tạp, cần chuyển cho bếp trưởng ( $\mu$ Code).

## 3.4. $\mu$ Code & MOP Cache

**$\mu$ Code (Microcode):**

- Xử lý các lệnh phức tạp không thể giải mã bởi Simple Decoder
- Phân tách lệnh phức tạp thành chuỗi nhiều  $\mu$ OPs
- Ví dụ: lệnh string operations, lệnh SIMD phức tạp

### MOP Cache:

- Lưu trữ các chuỗi  $\mu$ OPs đã được giải mã trước đó
- Khi gặp lại lệnh tương tự, lấy trực tiếp từ MOP Cache thay vì giải mã lại
- Tiết kiệm năng lượng và thời gian

### Ví dụ thực tế:

- $\mu$ Code như một cuốn sách hướng dẫn chi tiết cho các công việc phức tạp
- MOP Cache như bộ nhớ của bạn - sau khi làm một lần, lần sau nhớ nhanh hơn

## 3.5. Decode Queue - Hàng Đợi Giải Mã

**Vị trí:** Nằm giữa các Decoders và MUX. **Chức năng:** Thu thập các  $\mu$ OPs từ nhiều nguồn:

- Simple Decoders (6  $\mu$ OPs từ mỗi bên)
- $\mu$ Code
- MOP Cache

### Luồng dữ liệu:

- Mỗi Decode Queue gửi 6  $\mu$ OPs
- Tổng:  $6 + 6 = 12$   $\mu$ OPs đến MUX

**Ví dụ thực tế:** Giống như một bãi tập kết hàng hóa, nơi các gói hàng từ nhiều nguồn được tập trung trước khi phân phối tiếp.

## 3.6. MUX (Multiplexer)

### Chức năng:

- Nhận 12  $\mu$ OPs từ hai Decode Queues
- Kết hợp và điều phối chúng
- Gửi tiếp xuống giai đoạn Issue

**Bảng thông:** 12  $\mu$ OPs mỗi chu kỳ (đường "12 MOPs" trong sơ đồ)

# 4. CÁC THÀNH PHẦN CHI TIẾT - PHẦN BACKEND

## 4.1. Physical Register Files - Bộ Thanh Ghi Vật Lý

**Cấu trúc: Integer Physical Register File:**

- Số lượng: **~576 thanh ghi**
- Chức năng: Lưu trữ các giá trị số nguyên (integers)
- Độ rộng: 64-bit mỗi thanh ghi

**Vector Physical Register File:**

- Số lượng: **~512 thanh ghi**
- Chức năng: Lưu trữ các giá trị số thực (floating-point) và vector
- Độ rộng: 128-bit hoặc 256-bit mỗi thanh ghi (hỗ trợ SIMD)

**Ý nghĩa:** Nhiều thanh ghi cho phép CPU theo dõi nhiều phép tính đang diễn ra đồng thời, tăng khả năng thực thi song song. **Ví dụ thực tế:** Giống như một văn phòng có nhiều bàn làm việc. Càng nhiều bàn, càng nhiều người có thể làm việc cùng lúc mà không phải chờ đợi.

## 4.2. Physical Register Reclaim Table (PRRT)

**Kích thước:** 1280 entries **Chức năng:**

- Theo dõi thanh ghi vật lý nào đang được sử dụng
- Quản lý việc phân bổ thanh ghi mới cho các lệnh
- Thu hồi thanh ghi không còn dùng để tái sử dụng

**Register Renaming:** PRRT là trung tâm của kỹ thuật "đổi tên thanh ghi", cho phép CPU loại bỏ các phụ thuộc giả (false dependencies) và tăng khả năng thực thi song song. **Ví dụ thực tế:** Giống như một người quản lý bãi đỗ xe, biết chỗ nào đang trống, chỗ nào có xe, và hướng dẫn tài xế đỗ xe vào vị trí thích hợp.

## 4.3. Retire Queue (ROQ) - Hàng Đợi Hoàn Thành

**Kích thước:** 620 coalesced entries **Chức năng chính:**

1. **Theo dõi thứ tự:** Ghi nhớ thứ tự ban đầu của các lệnh trong chương trình
2. **Đảm bảo tính đúng đắn:** Dù các lệnh thực thi không theo thứ tự, ROQ đảm bảo chúng "hoàn thành" (commit) đúng thứ tự
3. **Xử lý ngoại lệ:** Khi có lỗi xảy ra, ROQ giúp CPU biết cần rollback đến đâu



4. **Speculative execution:** Cho phép CPU thực thi "đầu cơ" các lệnh, hủy nếu sai

**Kích thước 620:** P Core có thể "nhìn xa" và theo dõi 620 lệnh đang bay trong pipeline cùng lúc. **Ví dụ thực tế:** Giống như hệ thống lấy số ở ngân hàng. Nhiều người có thể làm việc với nhân viên khác nhau (out-of-order), nhưng khi xong phải được gọi theo đúng thứ tự số đã lấy (in-order retirement).

#### 4.4. Issue Stage - Giai Đoạn Phát Hành

**Chức năng:** Phân phối các  $\mu$ OPs từ giai đoạn Rename đến các Schedulers thích hợp.

**Băng thông:** 24  $\mu$ OPs mỗi chu kỳ (theo đường "24 pops" trong sơ đồ) **Quyết định phân phối:**

- Lệnh số nguyên  $\rightarrow$  INT Schedulers
- Lệnh số thực/vector  $\rightarrow$  FP Schedulers
- Lệnh truy cập bộ nhớ  $\rightarrow$  MEM Scheduler

#### 4.5. Schedulers - Bộ Lập Lịch

Đây là "bộ não" của backend, quyết định lệnh nào được thực thi khi nào.

##### 4.5.1. 32 INT Scheduler (có 4 bộ)

**Kích thước mỗi bộ:** 32 entries **Execution Units được điều khiển: Bộ 1 & 2:**

- ALU Branch: Xử lý phép tính số học và logic cơ bản + branch operations
- ALU Branch: Mỗi scheduler điều khiển 2 ALUs

**Bộ 3 & 4:**

- ALU: Xử lý phép tính số học và logic cơ bản

**Các phép tính:** Cộng, trừ, AND, OR, XOR, shift, compare **Tổng:** 4 schedulers  $\times$  32 entries = 128 entries cho integer operations đơn giản

##### 4.5.2. 24 INT Scheduler (có 2 bộ)

**Kích thước mỗi bộ:** 24 entries **Execution Units:**

- ALU MUL DIV: Nhân và chia số nguyên

**Lý do riêng biệt:** Phép nhân và chia phức tạp hơn, mất nhiều chu kỳ hơn (3-10 chu kỳ), nên cần scheduler và execution units riêng. **Tổng:** 2 schedulers × 24 entries = 48 entries cho integer multiply/divide

#### 4.5.3. 48 INT Scheduler (có 1 bộ)

**Kích thước:** 48 entries **Execution Units:**

- ALU MUL: Chuyên xử lý phép nhân số nguyên

**Lý do có thêm:** Phép nhân được dùng rất nhiều trong code hiện đại, nên P Core có thêm một scheduler và execution unit riêng để tăng throughput.

#### 4.5.4. 52 FP Scheduler (có 6 bộ)

**Đây là điểm nổi bật của P Core! Kích thước mỗi bộ:** 52 entries **Tổng dung lượng:** 6 × 52 = **312 entries** cho floating-point và vector operations **Execution Units: 4 bộ scheduler đầu:**

- VALU VMUL FP ADD:
  - V = Vector
  - ALU = Arithmetic Logic Unit
  - MUL = Multiply
  - FP ADD = Floating Point Addition
  - Xử lý phép cộng số thực và vector

**2 bộ scheduler tiếp:**

- VALU VMUL FP ADD: Tiếp tục phép cộng
- VALU VMUL FP ADD: Tiếp tục phép cộng

**Execution Units bổ sung:**

- FP MUL FP DIV (có 4 units): Nhân và chia số thực
- FP MUL (có thêm units): Nhân số thực

**Ý nghĩa:** P Core có khả năng xử lý số thực và vector rất mạnh mẽ, phù hợp cho:

- Đồ họa 3D
- Machine Learning
- Scientific computing

- Video/Audio processing
- Physics simulations

**Ví dụ thực tế:** Khi bạn chơi game, GPU tính toán hình ảnh, nhưng CPU (P Core) tính toán vật lý, AI của NPC, âm thanh 3D - tất cả đều dùng floating-point.

#### 4.5.5. 96 MEM Scheduler

**Kích thước:** 96 entries - lớn nhất trong các schedulers **Chức năng:** Lập lịch cho tất cả các truy cập bộ nhớ (memory operations). **Execution Units: AGU Store Data (có 2 units):**

- AGU = Address Generation Unit
- Tính toán địa chỉ bộ nhớ cần ghi
- Chuẩn bị dữ liệu để ghi vào bộ nhớ

**AGU Store Address (có 2 units):**

- Tính toán địa chỉ cho các lệnh Store

**Load Data (có 4 units):**

- Đọc dữ liệu từ bộ nhớ
- 4 units cho phép đọc 4 địa chỉ khác nhau đồng thời

**Tại sao lớn:** Memory operations thường chậm (cache miss có thể mất 100+ chu kỳ), nên cần scheduler lớn để chứa nhiều lệnh đang chờ.

## 4.6. Store Buffer & Load Buffer

**Store Buffer:**

- Kích thước: **96 entries**
- Băng thông: 16 B/Cycle × 6 ports
- Chức năng: Giữ các lệnh ghi (store) chờ được ghi vào cache/memory
- Cho phép CPU tiếp tục xử lý mà không phải đợi ghi xong

**Load Buffer:**

- Kích thước: **250 entries**
- Băng thông: 16 B/Cycle × 8 ports
- Chức năng: Giữ các lệnh đọc (load) chờ dữ liệu từ cache/memory

- Lớn hơn Store Buffer vì đọc xảy ra thường xuyên hơn

**Memory Disambiguation:** Hai buffers này làm việc cùng nhau để phát hiện xung đột địa chỉ (khi một Load cần đọc dữ liệu mà một Store chưa ghi xong). **Ví dụ thực tế:** Giống như một kho hàng có 2 khu vực:

- **Khu nhập hàng** (Load Buffer - 250 chỗ): Nhận hàng vào, xử lý, phân loại
- **Khu xuất hàng** (Store Buffer - 96 chỗ): Đóng gói, chờ xe tải chở đi

## 5. HỆ THỐNG BỘ NHỚ

### 5.1. L1 Data Cache (L1 D-Cache)

**Kích thước:** 128KB **Tổ chức:** 12-way set associative **Bảng thông:**

- $16 \text{ B/Cycle} \times 8 \text{ ports} = 128 \text{ bytes mỗi chu kỳ}$
- 8 ports nghĩa là 8 lệnh có thể truy cập cache đồng thời

**Độ trễ:** 4-5 chu kỳ **Chức năng:** Lưu trữ dữ liệu (variables, arrays, objects) mà chương trình đang sử dụng. **Ví dụ thực tế:** Giống như bàn làm việc của bạn. Các tài liệu đang dùng đặt trên bàn (L1 cache) để lấy nhanh, thay vì phải đi tủ (L2) hoặc kho (RAM).

### 5.2. L1 Data TLB (L1 DTLB)

**Kích thước:** 384 entries **Chức năng:** Chuyển đổi địa chỉ ảo (virtual address) sang địa chỉ vật lý (physical address) cho các truy cập dữ liệu. **Tại sao cần:**

- Mỗi chương trình nghĩ mình có toàn bộ bộ nhớ
- OS thực sự quản lý và phân chia bộ nhớ vật lý
- TLB giúp chuyển đổi nhanh mà không cần hỏi OS mỗi lần

**Ví dụ thực tế:** Giống như một danh bạ điện thoại nhanh. Thay vì tìm trong toàn bộ danh bạ (page table), bạn có 384 số thường dùng nhất lưu sẵn (TLB).

### 5.3. L2 Cache

**Kích thước:** 24MB (Shared) **Tổ chức:** 16-way set associative **Bảng thông:** 32 B/Cycle **Độ trễ:** ~15-20 chu kỳ **Shared:** Cache này được chia sẻ giữa nhiều cores (nếu CPU có nhiều P

Cores), giúp các cores trao đổi dữ liệu hiệu quả. **Chức năng:** Backup cho L1 Cache. Khi L1 miss, tìm trong L2. Nếu L2 cũng miss, phải xuống RAM (mất 100+ chu kỳ). **Ví dụ thực tế:**

- L1 = Bàn làm việc (128KB)
- L2 = Tủ tài liệu trong phòng (24MB)
- RAM = Thư viện lớn (16GB+)

## 5.4. L2 TLB

**Kích thước:** 4096 entries **Chức năng:** TLB cấp 2, lớn hơn và chậm hơn L1 TLB một chút.

**Hierarchy:**

1. L1 DTLB (384 entries) - kiểm tra trước
2. Nếu miss → L2 TLB (4096 entries)
3. Nếu vẫn miss → Page table walk (chậm, mất nhiều chu kỳ)

**Lợi ích:** 4096 entries đủ lớn để cover hầu hết các truy cập bộ nhớ của chương trình, giảm page table walk.

## 6. RETIREMENT - GIAI ĐOẠN HOÀN THÀNH

**Băng thông:** 12  $\mu$ OPs mỗi chu kỳ (12 pops trong sơ đồ) **Vị trí:** Giai đoạn cuối cùng của pipeline. **Chức năng quan trọng:**

1. **Commit kết quả:**
  - Chỉ khi lệnh ở đầu ROQ (đúng thứ tự) mới được retire
  - Cập nhật architectural state (trạng thái chính thức của CPU)
  - Ghi kết quả vào register file hoặc memory
1. **Xử lý ngoại lệ:**
  - Nếu có exception (chia cho 0, page fault), xử lý tại đây
  - Rollback các lệnh sau exception
  - Chuyển điều khiển đến exception handler
1. **Giải phóng tài nguyên:**
  - Giải phóng entry trong ROQ
  - Thu hồi thanh ghi vật lý cũ (thông qua PRRT)
  - Giải phóng entries trong schedulers
1. **Branch resolution:**
  - Xác nhận branch prediction đúng hay sai

- Nếu sai: flush pipeline, bắt đầu lại từ nhánh đúng

**Retire Width 12:** P Core có thể hoàn thành tối đa 12  $\mu$ OPs mỗi chu kỳ, nghĩa là trong điều kiện lý tưởng, 12 lệnh nhỏ có thể "chính thức hoàn thành" mỗi chu kỳ. **Ví dụ thực tế:** Giống như bộ phận QC cuối cùng ở nhà máy. Sản phẩm được làm không theo thứ tự (out-of-order), nhưng QC phải kiểm tra và đóng gói đúng thứ tự đơn hàng (in-order retirement).

## 7. KIẾN TRÚC OUT-OF-ORDER EXECUTION

### 7.1. Khái Niệm

P Core sử dụng kiến trúc **Out-of-Order Execution** (thực thi không theo thứ tự).

**Ý nghĩa:** CPU có thể thực thi các lệnh không đúng thứ tự chương trình viết, miễn là không vi phạm phụ thuộc dữ liệu (data dependency) và kết quả cuối cùng vẫn đúng.

### 7.2. Cách Hoạt Động

**Ví dụ chương trình:**

1.  $R1 = R2 + R3$
2.  $R4 = R5 + R6$
3.  $R7 = R8 + R9$
4.  $R10 = R1 + R4$
5.  $R11 = R7 * 2$

**In-Order Execution (tuần tự):**

- Làm lệnh  $1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5$
- Tổng: 5 chu kỳ (giả sử mỗi lệnh 1 chu kỳ)

**Out-of-Order Execution:**

- Nhận ra lệnh 1, 2, 3 không phụ thuộc nhau
- Thực thi đồng thời lệnh 1, 2, 3 (trên 3 ALUs khác nhau)
- Lệnh 4 phải chờ lệnh 1 và 2 xong

- Lệnh 5 độc lập, có thể chạy song song với lệnh 4
- Tổng: 2-3 chu kỳ

**Lợi ích:** Tăng throughput, che giấu độ trễ, tận dụng tối đa execution units.

### 7.3. Các Thành Phần Hỗ Trợ

#### ROQ (620 entries):

- Theo dõi thứ tự ban đầu
- Đảm bảo retirement đúng thứ tự

#### Register Renaming (PRRT):

- Loại bỏ false dependencies
- Cho phép nhiều lệnh dùng cùng tên thanh ghi logic nhưng thanh ghi vật lý khác nhau

#### Large Schedulers:

- Giữ nhiều lệnh, tìm lệnh nào ready để thực thi
- Càng nhiều lệnh trong scheduler, càng nhiều cơ hội tìm instruction-level parallelism (ILP)

#### Many Execution Units:

- 4 bộ 32-INT Scheduler → nhiều ALUs
- 6 bộ 52-FP Scheduler → nhiều FP units
- Cho phép thực thi nhiều lệnh đồng thời

## 8. CON SỐ QUAN TRỌNG

#### Bảng Tổng Hợp Thông Số P Core:

| Thành phần | Giá trị | Ý nghĩa                         |
|------------|---------|---------------------------------|
| Frontend   |         |                                 |
| Decoders   | 12      | Giải mã 12 lệnh đơn giản/chu kỳ |

|                        |                     |                               |
|------------------------|---------------------|-------------------------------|
| L1 I-Cache             | 192KB               | Chứa ~45,000 lệnh             |
| L1 ITLB                | 384 entries         | Tra cứu địa chỉ lệnh          |
| <b>Rename/Dispatch</b> |                     |                               |
| Integer PRF            | ~576                | Lưu 576 số nguyên tạm thời    |
| Vector PRF             | ~512                | Lưu 512 vector/float tạm thời |
| PRRT                   | 1280 entries        | Quản lý thanh ghi             |
| ROQ                    | 620 entries         | Theo dõi 620 lệnh đang bay    |
| Issue Width            | 24 $\mu$ OPs        | Phát hành 24 $\mu$ OPs/chu kỳ |
| <b>Schedulers</b>      |                     |                               |
| 32-INT Scheduler       | $4 \times 32 = 128$ | Lập lịch INT đơn giản         |
| 24-INT Scheduler       | $2 \times 24 = 48$  | Lập lịch MUL/DIV              |
| 48-INT Scheduler       | $1 \times 48 = 48$  | Lập lịch MUL                  |
| 52-FP Scheduler        | $6 \times 52 = 312$ | Lập lịch FP/Vector            |
| 96-MEM Scheduler       | $1 \times 96 = 96$  | Lập lịch Memory               |
| <b>Memory</b>          |                     |                               |
| Store Buffer           | 96 entries          | Chờ ghi 96 lần                |
| Load Buffer            | 250 entries         | Chờ đọc 250 lần               |
| L1 D-Cache             | 128KB, 12-way       | Dữ liệu gần nhất              |
| L1 DTLB                | 384 entries         | Tra cứu địa chỉ dữ liệu       |



|                   |              |                                |
|-------------------|--------------|--------------------------------|
| L2 Cache          | 24MB, 16-way | Cache lớn chia sẻ              |
| L2 TLB            | 4096 entries | Tra cứu địa chỉ cấp 2          |
| <b>Retirement</b> |              |                                |
| Retire Width      | 12 $\mu$ OPs | Hoàn thành 12 $\mu$ OPs/chu kỳ |

### Điểm Nổi Bật:

**1. Decode Width 12:** P Core có thể giải mã 12 lệnh đơn giản mỗi chu kỳ - rất cao trong các CPU thương mại. **2. ROQ 620:** Một trong những ROQ lớn nhất, cho phép CPU "nhìn xa" và tìm nhiều cơ hội tối ưu. **3. 312 FP Scheduler Entries:** Khả năng lập lịch floating-point rất mạnh, phù hợp workload khoa học và đồ họa. **4. 96 MEM Scheduler:** Lập lịch memory operations mạnh mẽ, quan trọng cho data-intensive applications. **5. Large Caches:** 192KB L1-I + 128KB L1-D + 24MB L2 = rất lớn, giảm cache miss rate.

## 9. QUY TRÌNH XỬ LÝ MỘT LỆNH

Hãy theo dõi một lệnh cụ thể đi qua toàn bộ pipeline của P Core.

**Lệnh mẫu:** `R3 = R1 + R2` (cộng hai số nguyên)

### Bước 1: Fetch (2-5 chu kỳ)

- Branch Predictor** dự đoán địa chỉ lệnh tiếp theo
- L1 ITLB** tra cứu: chuyển địa chỉ ảo  $\rightarrow$  vật lý (1 chu kỳ)
- L1 I-Cache** kiểm tra:
  - Cache hit:** Lấy 64 bytes chứa lệnh (1 chu kỳ)
  - Cache miss:** Phải lấy từ L2 (15-20 chu kỳ) hoặc RAM (100+ chu kỳ)

Lệnh được gửi đến **Predecode Fetch Buffer**

### Bước 2: Decode (1-2 chu kỳ)

- Simple Decoder** nhận lệnh `ADD R3, R1, R2`

2. Phân tích: "Đây là phép cộng 2 thanh ghi số nguyên"
3. Tạo 1  $\mu$ OP: ``ADD_INT R3, R1, R2``
4. Gửi  $\mu$ OP vào **Decode Queue**
5. Đi qua **MUX**

### Bước 3: Rename & Dispatch (1-2 chu kỳ)

1. **Register Rename:**
  - Tra **PRRT**: Tìm thanh ghi vật lý cho R1, R2, R3
  - Giả sử:  $R1 \rightarrow P45$ ,  $R2 \rightarrow P67$ ,  $R3 \rightarrow P120$  (mới)
  - $\mu$ OP trở thành: ``ADD P120, P45, P67``
1. **Allocate ROQ Entry:**
  - Gán một entry trong ROQ (620 entries) để theo dõi lệnh này
  - Ghi nhớ thứ tự của lệnh
1. **Dispatch:**
  - Xác định đây là lệnh INT đơn giản
  - Gửi đến một trong 4 bộ **32-INT Scheduler**

### Bước 4: Schedule (0-N chu kỳ)

1. Lệnh vào **32-INT Scheduler**
2. Scheduler kiểm tra:
  - **P45** (R1) đã sẵn sàng chưa? (đã được tính xong bởi lệnh trước)
  - **P67** (R2) đã sẵn sàng chưa?

Nếu cả hai đã sẵn sàng:

- Lệnh được đánh dấu "ready to execute"
- Scheduler chọn lệnh này (nếu có ALU rảnh)

Nếu chưa sẵn sàng:

- Lệnh đợi trong scheduler
- Khi P45 hoặc P67 được tính xong, lệnh sẽ được "woken up"

### Bước 5: Execute (1 chu kỳ)

1. Scheduler gửi lệnh đến một **ALU** rảnh
2. ALU đọc giá trị từ:

- **P45** ở Integer Register File
- **P67** ở Integer Register File

ALU thực hiện phép cộng (1 chu kỳ cho phép cộng số nguyên)

Kết quả ghi vào **P120** trong Integer Register File

Đánh dấu **P120 = ready**, thông báo cho các lệnh khác đang chờ P120

### **Bước 6: Writeback (0 chu kỳ)**

- Kết quả đã ghi vào P120 ở bước Execute
- Không cần bước riêng

### **Bước 7: Retire (1 chu kỳ, khi đến lượt)**

1. Lệnh đợi trong **ROQ** cho đến khi:
  - Tất cả các lệnh trước nó đã retire
  - Lệnh này ở đầu ROQ

**Retirement Logic** kiểm tra:

- Không có exception
- Thực thi thành công

**Commit:**

- Xác nhận kết quả P120 là chính thức
- Thu hồi thanh ghi vật lý cũ của R3 (nếu có)
- Giải phóng entry trong ROQ

Lệnh biến mất khỏi pipeline

### **Tổng Thời Gian:**

**Trường hợp tốt nhất (cache hit, không chờ):**

- Fetch: 2 chu kỳ
- Decode: 1 chu kỳ
- Rename: 1 chu kỳ
- Schedule: 0 chu kỳ (sẵn sàng ngay)

- Execute: 1 chu kỳ
- Retire: 1 chu kỳ (khi đến lượt)
- **Tổng: ~6 chu kỳ từ fetch đến retire**

**Nhưng nhờ pipeline:** Khi lệnh này ở Execute, lệnh tiếp theo đang ở Decode, lệnh sau nữa đang ở Fetch. P Core có thể "launch" 12  $\mu$ OPs mỗi chu kỳ và retire 12  $\mu$ OPs mỗi chu kỳ trong điều kiện lý tưởng.

## 10. BẢNG THÔNG DỮ LIỆU

### 10.1. L1 D-Cache Bandwidth

**Cấu trúc:** 8 ports, mỗi port 16 B/Cycle **Tổng băng thông:**

- Read:  $16 \text{ B/Cycle} \times 8 = \mathbf{128 \text{ bytes/chu kỳ}}$
- Nếu CPU chạy 3 GHz:
  - $128 \text{ bytes} \times 3 \text{ tỷ chu kỳ/giây} = \mathbf{384 \text{ GB/s}}$

**Ý nghĩa:** 8 execution units có thể đọc/ghi L1 Cache đồng thời mà không phải chờ đợi. Rất quan trọng với P Core vì có nhiều execution units hoạt động song song.

### 10.2. Store/Load Buffer Bandwidth

**Load Buffer:**

- $8 \text{ ports} \times 16 \text{ B/Cycle} = 128 \text{ B/Cycle load}$
- 250 entries để chờ

**Store Buffer:**

- $6 \text{ ports} \times 16 \text{ B/Cycle} = 96 \text{ B/Cycle store}$
- 96 entries để chờ

**Tổng:**  $\sim 224 \text{ B/Cycle} = \sim 672 \text{ GB/s}$  tại 3 GHz (lý thuyết)

### 10.3. L2 Cache Bandwidth

**Băng thông:** 32 B/Cycle **Tính toán:**

- $32 \text{ bytes} \times 3 \text{ GHz} = \mathbf{96 \text{ GB/s}}$

**Lưu ý:** L2 chậm hơn và nhỏ hơn băng thông L1, nhưng có dung lượng lớn hơn nhiều (24MB).

## 10.4. Fetch Bandwidth

**Instruction Fetch:** 64 bytes/chu kỳ từ L1 I-Cache **Decode:** 12 lệnh/chu kỳ (giả sử mỗi lệnh ~4 bytes trung bình) = 48 bytes/chu kỳ **Nghĩa là:** Fetch bandwidth (64 B) đủ rộng để feed cho 12 decoders.

# KẾT LUẬN

## Tổng Quan P Core

P Core là một vi kiến trúc ARM hiệu suất cao với những đặc điểm nổi bật:

### 1. Frontend Mạnh Mẽ:

- 12 decoders có thể xử lý 12 lệnh đơn giản mỗi chu kỳ
- Cache lớn (192KB L1-I) giảm instruction cache miss
- Branch predictor tiên tiến cho độ chính xác cao

### 2. Backend Linh Hoạt:

- ROQ 620 entries cho phép CPU nhìn xa, tìm nhiều ILP
- 576 INT + 512 Vector PRF cho out-of-order execution mạnh
- Nhiều schedulers chuyên biệt tối ưu cho từng loại lệnh

### 3. Execution Units Phong Phú:

- Nhiều ALUs cho integer operations
- 6 FP schedulers (312 entries) với nhiều FP units cho tính toán số thực
- 96 MEM scheduler với nhiều AGUs và Load/Store units

### 4. Memory Subsystem Hiệu Quả:

- L1 D-Cache 128KB với 8 ports (128 B/cycle)
- L2 Cache 24MB lớn, giảm cache miss
- TLB lớn (384 L1, 4096 L2) xử lý tốt virtual memory
- Store/Load buffers lớn (96/250) che giấu memory latency

## 5. Retirement Cao:

- 12  $\mu$ OPs/cycle retirement bandwidth
- Đảm bảo correctness trong out-of-order execution

## Ứng Dụng

P Core được thiết kế cho workload đòi hỏi hiệu suất cao:

- **Single-threaded performance cao:** Gaming, CAD, simulations
- **Floating-point intensive:** Machine Learning, scientific computing, rendering
- **Memory-intensive:** Databases, big data processing
- **Compute-intensive:** Compilation, compression, encoding

## Triết Lý Thiết Kế

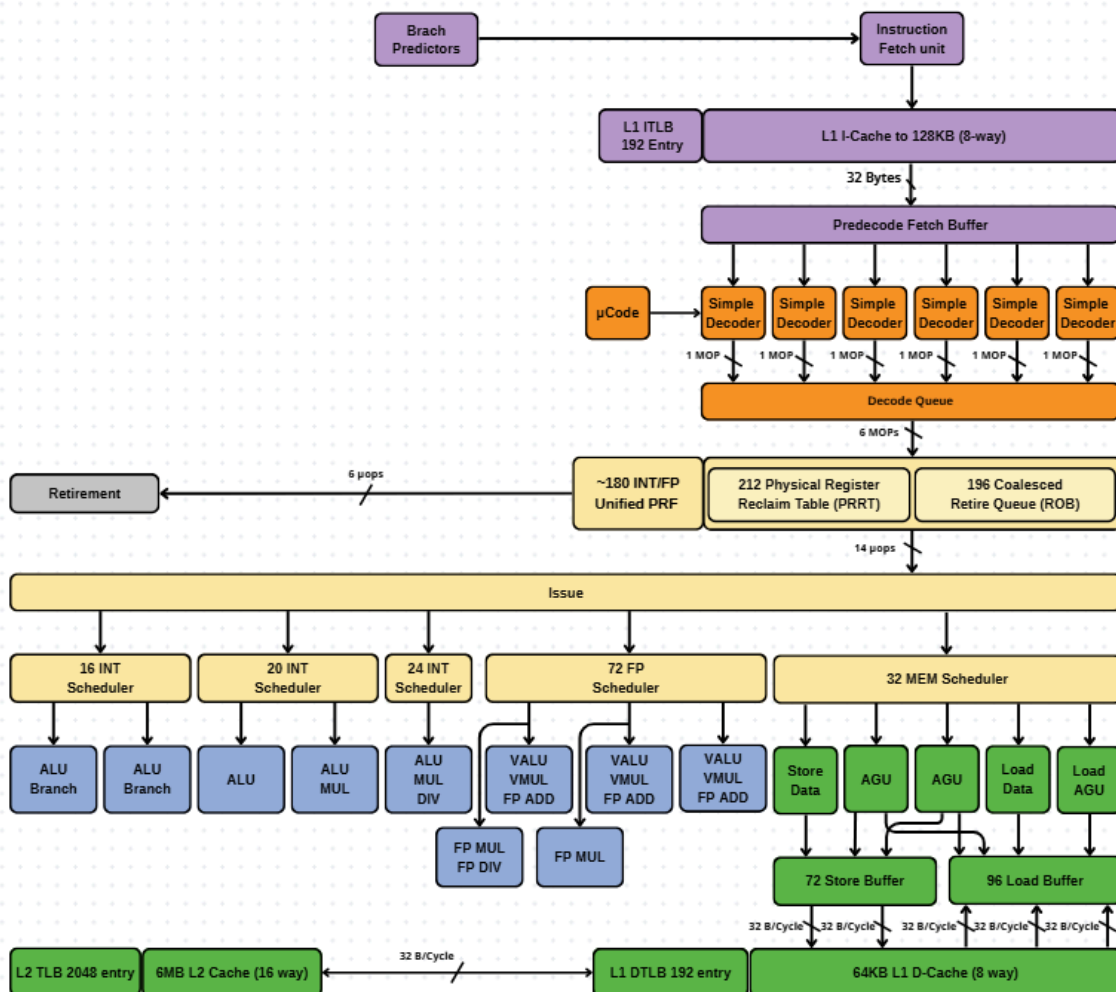
P Core ưu tiên:

- **Performance > Efficiency:** Làm việc nhanh nhất có thể
- **Throughput cao:** Nhiều execution units, wide pipeline
- **Latency hiding:** Large ROQ và buffers che giấu độ trễ bộ nhớ
- **ILP extraction:** Tìm và khai thác instruction-level parallelism tối đa

**P Core là minh chứng cho sự tinh vi của thiết kế CPU hiện đại - một hệ thống phức tạp với hàng trăm thành phần làm việc hài hòa để đạt được hiệu suất đỉnh cao.**

### III. E core

## E Core



## GIẢI THÍCH VI KIẾN TRÚC E CORE

### MỤC LỤC

1. Giới Thiệu
2. Tổng Quan Quy Trình Xử Lý
3. Các Thành Phần Chi Tiết - Phần Frontend
4. Các Thành Phần Chi Tiết - Phần Backend
5. Hệ Thống Bộ Nhớ
6. Retirement - Giai Đoạn Hoàn Thành
7. Kiến Trúc Out-of-Order Execution
8. Con Số Quan Trọng
9. Quy Trình Xử Lý Một Lệnh
10. Bảng Thông Dữ Liệu

## 1. GIỚI THIỆU

**E Core** (Efficiency Core) là một lõi xử lý được thiết kế theo triết lý cân bằng giữa hiệu suất và hiệu quả năng lượng. Đây là một bộ xử lý thuộc kiến trúc ARM với khả năng thực thi không theo thứ tự (out-of-order execution).

E Core tập trung vào việc xử lý nhiều tác vụ song song một cách hiệu quả, phù hợp cho các workload đa luồng (multi-threaded) và các tác vụ chạy nền (background tasks).

## 2. TỔNG QUAN QUY TRÌNH XỬ LÝ

E Core thực hiện quy trình xử lý gồm các giai đoạn chính:

1. **Fetch (Lấy lệnh):** Lấy các lệnh chương trình từ bộ nhớ
2. **Decode (Giải mã):** Phân tích và chuyển đổi lệnh thành micro-operations
3. **Issue (Phát hành):** Phân phối các  $\mu$ OPs đến các bộ lập lịch
4. **Execute (Thực thi):** Thực hiện các phép tính thực sự
5. **Retire (Hoàn thành):** Cam kết kết quả theo đúng thứ tự chương trình

**Đặc điểm:** E Core có pipeline hẹp hơn, ít execution units hơn, nhưng vẫn duy trì khả năng out-of-order execution để đảm bảo hiệu suất tốt.

## 3. CÁC THÀNH PHẦN CHI TIẾT - PHẦN FRONTEND



### 3.1. Branch Predictors - Bộ Dự Đoán Nhánh

**Vị trí:** Ở đầu pipeline, trước Instruction Fetch Unit. **Chức năng:** Dự đoán hướng đi của chương trình khi gặp các câu lệnh rẽ nhánh (if/else, loops, function calls). **Tầm quan trọng:**

- Dự đoán đúng: CPU tiếp tục xử lý mượt mà
- Dự đoán sai: Phải hủy các lệnh đã xử lý và bắt đầu lại, mất khoảng 10-15 chu kỳ

**Ví dụ thực tế:** Giống như việc bạn dự đoán đèn giao thông sẽ chuyển sang xanh dựa trên thời gian chờ, và chuẩn bị sẵn để tăng tốc.

### 3.2. Instruction Fetch Unit - Bộ Lấy Lệnh

**Thành phần: L1 ITLB (192 Entry):**

- ITLB = Instruction Translation Lookaside Buffer
- 192 entries có thể tra cứu địa chỉ của 192 trang bộ nhớ
- Chuyển đổi địa chỉ ảo (virtual address) sang địa chỉ vật lý (physical address)

**L1 I-Cache (128KB, 8-way):**

- I-Cache = Instruction Cache
- Kích thước: 128KB
- Tổ chức: 8-way set associative
- Chứa khoảng 30,000-35,000 lệnh máy
- Độ trễ truy cập: 3-4 chu kỳ

**Bảng thông:** 32 Bytes mỗi chu kỳ được lấy từ cache. **Ví dụ thực tế:** L1 I-Cache như một cuốn sổ tay ghi chép các công thức thường dùng. Khi cần, bạn mở sổ tay (cache) ra xem thay vì phải lật sách giáo khoa dày (RAM).

### 3.3. Predecode Fetch Buffer & Simple Decoders

**Cấu trúc:** Có 1 Predecode Fetch Buffer với 6 Simple Decoders. **Luồng xử lý:**

- Buffer nhận 32 bytes lệnh từ I-Cache
- 6 Simple Decoders phân tích các lệnh đơn giản
- Mỗi decoder tạo ra 1  $\mu$ OP (micro-operation)

**Thông lượng tối đa: 6 lệnh đơn giản mỗi chu kỳ Micro-operations ( $\mu$ OPs):** Là các lệnh nhỏ, đơn giản mà CPU thực sự thực thi bên trong. Một lệnh ARM phức tạp có thể được chia thành nhiều  $\mu$ OPs. **Ví dụ thực tế:** Giống như một nhà hàng nhỏ có 6 đầu bếp, mỗi người xử lý một món ăn đơn giản. Nếu có món phức tạp, cần chuyển cho bếp trưởng ( $\mu$ Code).

### 3.4. $\mu$ Code - Microcode

**$\mu$ Code (Microcode):**

- Xử lý các lệnh phức tạp không thể giải mã bởi Simple Decoder
- Phân tách lệnh phức tạp thành chuỗi nhiều  $\mu$ OPs
- Ví dụ: các lệnh chuỗi (string operations), lệnh SIMD phức tạp, lệnh hệ thống

**Hoạt động:**

- Khi Simple Decoder gặp lệnh phức tạp, chuyển đến  $\mu$ Code
- $\mu$ Code tra bảng ROM bên trong, lấy chuỗi  $\mu$ OPs tương ứng
- Gửi chuỗi  $\mu$ OPs này xuống pipeline

**Ví dụ thực tế:**  $\mu$ Code như một cuốn sách công thức chi tiết cho các món ăn phức tạp. Khi gặp món khó, đầu bếp phải mở sách ra đọc từng bước.

### 3.5. Decode Queue - Hàng Đợi Giải Mã

**Chức năng:** Thu thập các  $\mu$ OPs từ các nguồn:

- 6 Simple Decoders
- $\mu$ Code

**Luồng dữ liệu:**

- Decode Queue tập trung các  $\mu$ OPs
- Gửi **6  $\mu$ OPs** mỗi chu kỳ xuống giai đoạn tiếp theo

**Ví dụ thực tế:** Giống như một khu vực tập kết hàng hóa nhỏ, nơi các gói hàng được thu thập trước khi chuyển tiếp đi.

## 4. CÁC THÀNH PHẦN CHI TIẾT - PHẦN BACKEND

## 4.1. Unified Physical Register File

**Cấu trúc đặc biệt:** E Core sử dụng kiến trúc **Unified PRF** - một bộ thanh ghi dùng chung cho cả integer và floating-point. **Kích thước:** ~180 INT/FP **Unified PRF Chức năng:**

- Lưu trữ cả giá trị số nguyên (integers)
- Lưu trữ cả giá trị số thực (floating-point) và vector
- Độ rộng: có thể lưu 64-bit hoặc 128-bit tùy loại dữ liệu

**Lợi ích của Unified:**

- Linh hoạt: Phân bổ động theo nhu cầu workload
- Tiết kiệm diện tích: Một bộ thanh ghi thay vì hai
- Nếu workload dùng nhiều INT: có thể cấp phát nhiều cho INT
- Nếu workload dùng nhiều FP: có thể cấp phát nhiều cho FP

**Ví dụ thực tế:** Giống như một bãi đỗ xe chung, có thể đỗ cả xe hơi (INT) và xe tải (FP). Tùy vào nhu cầu mà phân chia chỗ đỗ linh hoạt.

## 4.2. Physical Register Reclaim Table (PRRT)

**Kích thước:** 212 entries **Chức năng:**

- Theo dõi thanh ghi vật lý nào đang được sử dụng
- Quản lý việc phân bổ thanh ghi mới
- Thu hồi thanh ghi không còn dùng để tái sử dụng

**Register Renaming:** PRRT thực hiện kỹ thuật đổi tên thanh ghi, loại bỏ WAR (Write-After-Read) và WAW (Write-After-Write) hazards, tăng khả năng out-of-order execution. **Ví dụ thực tế:** Giống như một người quản lý nhà nghỉ với 212 phòng, theo dõi phòng nào đang có khách, phòng nào trống, và sắp xếp khách vào phòng phù hợp.

## 4.3. Retire Queue (ROQ) - Hàng Đợi Hoàn Thành

**Kích thước:** 196 coalesced entries **Chức năng chính:**

1. **Theo dõi thứ tự:** Ghi nhớ thứ tự ban đầu của các lệnh trong chương trình
2. **Đảm bảo tính đúng đắn:** Các lệnh thực thi không theo thứ tự, nhưng ROQ đảm bảo chúng hoàn thành đúng thứ tự
3. **Xử lý ngoại lệ:** Khi có lỗi, ROQ giúp CPU biết cần rollback đến đâu

#### 4. **Speculative execution management:** Quản lý các lệnh thực thi đầu cơ

**Kích thước 196:** E Core có thể theo dõi và quản lý 196 lệnh đang "bay" trong pipeline cùng một lúc. **Ví dụ thực tế:** Giống như một hệ thống quản lý đơn hàng. Các đơn có thể được xử lý không theo thứ tự (out-of-order), nhưng khi giao hàng phải đúng thứ tự khách đặt (in-order commit).

### 4.4. Issue Stage - Giai Đoạn Phát Hành

**Chức năng:** Phân phối các  $\mu$ OPs từ giai đoạn Rename đến các Schedulers thích hợp.

**Băng thông: 14 pops** (14  $\mu$ OPs mỗi chu kỳ) **Quyết định phân phối:**

- Lệnh số nguyên  $\rightarrow$  INT Schedulers
- Lệnh số thực/vector  $\rightarrow$  FP Scheduler
- Lệnh truy cập bộ nhớ  $\rightarrow$  MEM Scheduler

**Lưu ý:** Issue width (14) rộng hơn decode width (6), cho phép dispatch các  $\mu$ OPs từ nhiều chu kỳ decode trước đó.

### 4.5. Schedulers - Bộ Lập Lịch

E Core có kiến trúc scheduler gọn nhẹ với các bộ lập lịch chuyên biệt:

#### 4.5.1. 16 INT Scheduler (có 2 bộ)

**Kích thước mỗi bộ:** 16 entries **Execution Units** được điều khiển: **Bộ 1:**

- **ALU Branch:** Xử lý phép tính số học/logic cơ bản + branch operations
- **ALU Branch:** Một ALU nữa

**Bộ 2:**

- **ALU:** Phép tính số học và logic cơ bản

**Các phép tính:** Cộng, trừ, AND, OR, XOR, shift, compare, conditional operations **Tổng:** 2 schedulers  $\times$  16 entries = **32 entries** cho integer operations đơn giản

#### 4.5.2. 20 INT Scheduler

**Kích thước:** 20 entries **Execution Units:**

- **ALU:** Phép tính số học và logic

**Chức năng:** Thêm một scheduler để tăng khả năng xử lý integer operations.

#### 4.5.3. 24 INT Scheduler

**Kích thước:** 24 entries - lớn nhất trong các INT schedulers **Execution Units:**

- **ALU MUL DIV:** Nhân và chia số nguyên

**Lý do riêng biệt:**

- Phép nhân: mất 3-4 chu kỳ
- Phép chia: mất 10-20 chu kỳ
- Cần scheduler riêng để không làm chậm các phép tính đơn giản

**Tổng INT Scheduling Capacity:**  $16 + 16 + 20 + 24 = 76$  entries

#### 4.5.4. 72 FP Scheduler

**Kích thước:** 72 entries **Chức năng:** Lập lịch cho tất cả các phép tính số thực (floating-point) và vector. **Execution Units: VALU VMUL FP ADD (có 2 units):**

- V = Vector
- ALU = Arithmetic Logic Unit
- MUL = Multiply
- FP ADD = Floating Point Addition
- Xử lý phép cộng số thực và vector

**FP MUL FP DIV:**

- Nhân và chia số thực
- Chia số thực rất chậm (10-20 chu kỳ)

**FP MUL:**

- Một unit nhân số thực bổ sung

**Ứng dụng số thực:**

- Tính toán khoa học cơ bản
- Xử lý âm thanh

- Tính toán đồ họa 2D
- Physics simulations đơn giản
- Machine learning inference (chạy model đã train)

**Ví dụ thực tế:** Khi bạn xem video, decode video cần nhiều phép tính số thực. FP Scheduler và execution units xử lý những phép tính này.

#### 4.5.5. 32 MEM Scheduler

**Kích thước:** 32 entries **Chức năng:** Lập lịch cho tất cả các truy cập bộ nhớ (memory operations). **Execution Units: Store Data (có 1 unit):**

- Xử lý việc ghi dữ liệu vào bộ nhớ
- Chuẩn bị dữ liệu để ghi

**AGU (Address Generation Unit) (có 2 units):**

- Tính toán địa chỉ bộ nhớ cho cả Load và Store
- Xử lý các chế độ địa chỉ phức tạp (base + offset, indexed, etc.)

**Load AGU (có 2 units):**

- Chuyên xử lý Load operations
- Tính địa chỉ và yêu cầu dữ liệu từ cache

**Tổng:** 5 execution units cho memory operations

## 4.6. Store Buffer & Load Buffer

**Store Buffer:**

- Kích thước: **72 entries**
- Băng thông: 32 B/Cycle × 2 ports
- Chức năng: Giữ các lệnh ghi (store) chờ được ghi vào cache/memory
- Cho phép CPU tiếp tục xử lý mà không phải đợi ghi xong

**Load Buffer:**

- Kích thước: **96 entries**
- Băng thông: 32 B/Cycle × 2 ports
- Chức năng: Giữ các lệnh đọc (load) chờ dữ liệu từ cache/memory

**Memory Ordering:** Hai buffers này làm việc cùng nhau để:

- Đảm bảo memory consistency
- Phát hiện xung đột địa chỉ (address conflicts)
- Cho phép store-to-load forwarding (load đọc trực tiếp từ store chưa commit)

**Ví dụ thực tế:** Giống như một siêu thị nhỏ:

- **Khu nhập hàng** (Load Buffer - 96 chỗ): Nhận và kiểm tra hàng vào
- **Khu xuất hàng** (Store Buffer - 72 chỗ): Chuẩn bị hàng để giao cho khách

## 5. HỆ THỐNG BỘ NHỚ

### 5.1. L1 Data Cache (L1 D-Cache)

**Kích thước:** 64KB **Tổ chức:** 8-way set associative **Băng thông:**

- $32 \text{ B/Cycle} \times 2 \text{ ports} = 64 \text{ bytes mỗi chu kỳ}$

**Độ trễ:** 3-4 chu kỳ **Chức năng:** Lưu trữ dữ liệu (variables, arrays, objects) mà chương trình đang sử dụng. **Ví dụ thực tế:** L1 D-Cache như một hộp tủ nhỏ trên bàn làm việc, chứa các tài liệu đang dùng để truy cập nhanh.

### 5.2. L1 Data TLB (L1 DTLB)

**Kích thước:** 192 entries **Chức năng:** Chuyển đổi địa chỉ ảo sang địa chỉ vật lý cho các truy cập dữ liệu. **Tại sao cần:**

- Hệ điều hành dùng virtual memory để bảo vệ và quản lý
- Mỗi chương trình có không gian địa chỉ ảo riêng
- TLB giúp chuyển đổi nhanh mà không cần truy vấn page table mỗi lần

**Hierarchy:**

1. Kiểm tra L1 DTLB (192 entries) trước
2. Nếu miss → L2 TLB
3. Nếu vẫn miss → Page table walk (chậm)

### 5.3. L2 Cache

**Kích thước:** 6MB (Shared) **Tổ chức:** 16-way set associative **Băng thông:** 32 B/Cycle **Độ trễ:** ~15-20 chu kỳ **Shared:** Cache này được chia sẻ giữa nhiều E Cores (nếu CPU có nhiều E Cores), cho phép các cores trao đổi dữ liệu hiệu quả và tiết kiệm diện tích chip. **Chức năng:**

- Là bộ đệm cấp 2 cho cả instruction và data
- Khi L1 miss, tìm trong L2
- Nếu L2 miss, phải xuống RAM (100+ chu kỳ)

**Cache Hierarchy:**

- **L1:** Nhỏ (64KB data, 128KB instruction), rất nhanh (3-4 chu kỳ)
- **L2:** Trung bình (6MB), chậm hơn (15-20 chu kỳ)
- **RAM:** Lớn (GBs), rất chậm (100+ chu kỳ)

## 5.4. L2 TLB

**Kích thước:** 2048 entries **Chức năng:** TLB cấp 2, lớn hơn L1 TLB. **Hierarchy:**

1. L1 ITLB (192 entries) cho instruction
2. L1 DTLB (192 entries) cho data
3. Nếu miss → **L2 TLB (2048 entries)** - dùng chung
4. Nếu vẫn miss → Page table walk

**Lợi ích:** 2048 entries đủ lớn để cover được memory footprint của hầu hết các ứng dụng thông thường, giảm thiểu page table walks tốn kém.

## 6. RETIREMENT - GIAI ĐOẠN HOÀN THÀNH

**Băng thông:** 6 pops (6  $\mu$ OPs mỗi chu kỳ) **Vị trí:** Giai đoạn cuối cùng của pipeline. **Chức năng quan trọng:**

1. **Commit kết quả:**
  - Chỉ khi lệnh ở đầu ROQ (đúng thứ tự) mới được retire
  - Cập nhật architectural state (trạng thái chính thức của CPU)
  - Làm cho kết quả "visible" cho hệ thống
1. **Xử lý ngoại lệ:**
  - Phát hiện exceptions (page faults, divide by zero, etc.)



- Rollback các lệnh sau exception
  - Chuyển control đến exception handler
1. **Giải phóng tài nguyên:**
    - Giải phóng entry trong ROQ (196 entries)
    - Thu hồi thanh ghi vật lý thông qua PRRT (212 entries)
    - Giải phóng entries trong schedulers
    - Giải phóng entries trong Load/Store buffers
  1. **Branch misprediction recovery:**
    - Xác định branch prediction có đúng không
    - Nếu sai: flush pipeline, restart từ địa chỉ đúng

**Retire Width 6:** E Core có thể hoàn thành tối đa 6  $\mu$ OPs mỗi chu kỳ. Con số này phù hợp với decode width (6), tạo ra một pipeline cân bằng. **Ví dụ thực tế:** Giống như một bộ phận đóng gói cuối cùng ở nhà máy. Dù sản phẩm được làm không theo thứ tự, bộ phận này phải kiểm tra và đóng gói đúng thứ tự đơn hàng, đảm bảo khách hàng nhận đúng.

## 7. KIẾN TRÚC OUT-OF-ORDER EXECUTION

### 7.1. Khái Niệm

E Core sử dụng kiến trúc **Out-of-Order Execution** - thực thi không theo thứ tự.

**Ý nghĩa:** CPU có thể thực thi các lệnh không đúng thứ tự chương trình viết, miễn là:

- Không vi phạm data dependencies (phụ thuộc dữ liệu)
- Kết quả cuối cùng vẫn đúng như chương trình yêu cầu
- Các lệnh retire (hoàn thành) theo đúng thứ tự

### 7.2. Ví Dụ Minh Họa

**Chương trình:**

1.  $R1 = R2 + R3$
2.  $R4 = R5 + R6$
3.  $R7 = R1 * 2$
4.  $R8 = R4 + R7$

### **Phân tích phụ thuộc:**

- Lệnh 1 và 2: Độc lập, có thể chạy song song
- Lệnh 3: Phụ thuộc vào lệnh 1 (cần R1)
- Lệnh 4: Phụ thuộc vào lệnh 2 và 3 (cần R4 và R7)

### **In-Order Execution (tuần tự):**

- Chu kỳ 1: Lệnh 1
- Chu kỳ 2: Lệnh 2
- Chu kỳ 3: Lệnh 3
- Chu kỳ 4: Lệnh 4
- **Tổng: 4 chu kỳ**

### **Out-of-Order Execution:**

- Chu kỳ 1: Lệnh 1 và 2 chạy song song (2 ALUs)
- Chu kỳ 2: Lệnh 3 chạy (sau khi lệnh 1 xong)
- Chu kỳ 3: Lệnh 4 chạy (sau khi lệnh 2 và 3 xong)
- **Tổng: 3 chu kỳ**

**Lợi ích:** Tiết kiệm 25% thời gian nhờ tận dụng instruction-level parallelism (ILP).

## **7.3. Các Thành Phần Hỗ Trợ Out-of-Order**

### **ROQ (196 entries):**

- Theo dõi thứ tự ban đầu của 196 lệnh
- Đảm bảo retirement đúng thứ tự dù thực thi không theo thứ tự

### **Register Renaming (PRRT 212 entries):**

- Loại bỏ WAR và WAW hazards
- Cho phép nhiều lệnh "viết" vào cùng thanh ghi logic nhưng thực tế dùng thanh ghi vật lý khác nhau

### **Schedulers:**

- Theo dõi operands của mỗi lệnh
- Khi tất cả operands sẵn sàng: lệnh "ready to execute"
- Chọn lệnh ready để gửi đến execution units

Unified PRF (180 entries):

- Đủ lớn để lưu kết quả trung gian của nhiều lệnh
- Hỗ trợ rename và out-of-order execution

8. CON SỐ QUAN TRỌNG

Bảng Tổng Hợp Thông Số E Core:

| Thành phần       | Giá trị            | Ý nghĩa                        |
|------------------|--------------------|--------------------------------|
| Frontend         |                    |                                |
| Decoders         | 6                  | Giải mã 6 lệnh đơn giản/chu kỳ |
| Decode Width     | 6 $\mu$ OPs        | Tạo ra 6 $\mu$ OPs/chu kỳ      |
| L1 I-Cache       | 128KB, 8-way       | Chứa ~30,000+ lệnh             |
| L1 ITLB          | 192 entries        | Tra cứu địa chỉ lệnh           |
| Fetch Bandwidth  | 32 B/cycle         | Lấy 32 bytes lệnh/chu kỳ       |
| Rename/Dispatch  |                    |                                |
| Unified PRF      | ~180               | Lưu cả INT và FP               |
| PRRT             | 212 entries        | Quản lý thanh ghi              |
| ROQ              | 196 entries        | Theo dõi 196 lệnh đang bay     |
| Issue Width      | 14 pops            | Phát hành 14 $\mu$ OPs/chu kỳ  |
| Schedulers       |                    |                                |
| 16-INT Scheduler | $2 \times 16 = 32$ | Lập lịch INT đơn giản          |

|                   |                    |                               |
|-------------------|--------------------|-------------------------------|
| 20-INT Scheduler  | $1 \times 20 = 20$ | Lập lịch INT                  |
| 24-INT Scheduler  | $1 \times 24 = 24$ | Lập lịch MUL/DIV              |
| Tổng INT Capacity | 76 entries         | Tổng cho INT operations       |
| 72-FP Scheduler   | $1 \times 72 = 72$ | Lập lịch FP/Vector            |
| 32-MEM Scheduler  | $1 \times 32 = 32$ | Lập lịch Memory               |
| <b>Memory</b>     |                    |                               |
| Store Buffer      | 72 entries         | Chờ ghi 72 lần                |
| Load Buffer       | 96 entries         | Chờ đọc 96 lần                |
| L1 D-Cache        | 64KB, 8-way        | Dữ liệu gần nhất              |
| L1 D-Cache BW     | 64 B/cycle         | 2 ports $\times$ 32 B         |
| L1 DTLB           | 192 entries        | Tra cứu địa chỉ dữ liệu       |
| L2 Cache          | 6MB, 16-way        | Cache lớn chia sẻ             |
| L2 TLB            | 2048 entries       | Tra cứu địa chỉ cấp 2         |
| <b>Retirement</b> |                    |                               |
| Retire Width      | 6 pops             | Hoàn thành 6 $\mu$ OPs/chu kỳ |

## Điểm Đặc Biệt:

### 1. Pipeline Cân Bằng:

- Decode: 6  $\mu$ OPs/cycle
- Retire: 6 pops/cycle
- Pipeline được thiết kế cân bằng, không có bottleneck rõ ràng

## 2. Unified PRF (180):

- Thiết kế độc đáo, khác với separated register files
- Linh hoạt phân bổ giữa INT và FP
- Tiết kiệm diện tích chip

## 3. ROQ 196:

- Đủ lớn để hỗ trợ out-of-order execution hiệu quả
- Cho phép CPU "nhìn xa" gần 200 lệnh để tìm ILP

## 4. Schedulers Compact:

- Tổng scheduler capacity: 76 INT + 72 FP + 32 MEM = **180 entries**
- Gọn nhẹ nhưng đủ để duy trì throughput tốt

## 5. Cache Hierarchy:

- L1 I-Cache 128KB lớn (cho code footprint)
- L1 D-Cache 64KB vừa phải (cho working set)
- L2 6MB chia sẻ (hiệu quả diện tích)

# 9. QUY TRÌNH XỬ LÝ MỘT LỆNH

Hãy theo dõi một lệnh cụ thể đi qua toàn bộ pipeline của E Core.

**Lệnh mẫu:** `R5 = R3 + R4` (cộng hai số nguyên)

### Bước 1: Fetch (2-4 chu kỳ)

1. **Branch Predictor** dự đoán địa chỉ lệnh tiếp theo
2. **L1 ITLB** tra cứu: chuyển địa chỉ ảo → vật lý (1 chu kỳ)
3. **L1 I-Cache** (128KB) kiểm tra:
  - **Cache hit:** Lấy 32 bytes chứa lệnh (1 chu kỳ)
  - **Cache miss:** Lấy từ L2 (15-20 chu kỳ)

Lệnh được gửi đến **Predecode Fetch Buffer**

### Bước 2: Decode (1 chu kỳ)

1. **Simple Decoder** (1 trong 6) nhận lệnh `ADD R5, R3, R4`
2. Phân tích: "Phép cộng hai thanh ghi số nguyên"
3. Tạo 1  $\mu$ OP: `ADD\_INT R5, R3, R4`
4. Gửi  $\mu$ OP vào **Decode Queue**

### Bước 3: Rename & Dispatch (1-2 chu kỳ)

1. **Register Rename:**
  - Tra **PRRT** (212 entries): Tìm thanh ghi vật lý
  - Giả sử:  $R3 \rightarrow P25$ ,  $R4 \rightarrow P40$ ,  $R5 \rightarrow P78$  (mới phân bổ)
  - $\mu$ OP trở thành: `ADD P78, P25, P40`
1. **Allocate ROQ Entry:**
  - Gán một entry trong ROQ (196 entries)
  - Ghi nhớ thứ tự của lệnh này
1. **Dispatch (Issue stage):**
  - Xác định đây là lệnh INT đơn giản
  - Gửi đến một trong các **INT Schedulers** (16, 20, hoặc 24)

### Bước 4: Schedule (0-N chu kỳ)

1. Lệnh vào **INT Scheduler** (giả sử 16-INT Scheduler)
2. Scheduler kiểm tra:
  - **P25** (R3) đã sẵn sàng chưa?
  - **P40** (R4) đã sẵn sàng chưa?

#### Trường hợp sẵn sàng:

- Lệnh được đánh dấu "ready"
- Scheduler chọn lệnh này nếu có ALU rảnh
- Gửi đến **ALU** (chu kỳ tiếp theo)

#### Trường hợp chưa sẵn sàng:

- Lệnh đợi trong scheduler
- Khi P25 hoặc P40 được tính xong  $\rightarrow$  "wake up"

### Bước 5: Execute (1 chu kỳ)

1. Scheduler gửi lệnh đến một **ALU** rảnh

2. ALU đọc giá trị:

- **P25** từ Unified Register File
- **P40** từ Unified Register File

ALU thực hiện phép cộng (1 chu kỳ cho ADD)

Kết quả ghi vào **P78** trong Unified Register File

Đánh dấu **P78 = ready**, thông báo cho các lệnh đang chờ P78

### **Bước 6: Completion (0 chu kỳ)**

- Kết quả đã ghi vào P78 ở Execute stage
- Lệnh đánh dấu "completed" trong ROQ
- Chờ đến lượt retire

### **Bước 7: Retire (1 chu kỳ, khi đến lượt)**

1. Lệnh đợi trong **ROQ** (196 entries) cho đến khi:

- Tất cả các lệnh trước nó đã retire
- Lệnh này ở đầu ROQ

**Retirement Logic** kiểm tra:

- Không có exception
- Thực thi thành công

**Commit:**

- Xác nhận kết quả P78 là chính thức cho R5
- Thu hồi thanh ghi vật lý cũ của R5 (thông qua PRRT)
- Giải phóng entry trong ROQ
- Giải phóng entry trong scheduler

Lệnh chính thức hoàn thành

### **Tổng Thời Gian:**

**Trường hợp tốt nhất:**

- Fetch: 2 chu kỳ (cache hit)

- Decode: 1 chu kỳ
- Rename: 1 chu kỳ
- Schedule: 0 chu kỳ (sẵn sàng ngay)
- Execute: 1 chu kỳ
- Retire: 1 chu kỳ (khi đến lượt)
- **Tổng: ~6 chu kỳ**

**Nhưng nhờ pipelining:**

- Khi lệnh này ở Execute, lệnh khác đang ở Decode, lệnh khác nữa đang ở Fetch
- E Core có thể decode 6 lệnh/chu kỳ và retire 6  $\mu$ OPs/chu kỳ
- Throughput: Có thể hoàn thành nhiều lệnh mỗi chu kỳ trong steady state

## 10. BẢNG THÔNG DỮ LIỆU

### 10.1. Instruction Fetch Bandwidth

**Bảng thông:** 32 bytes/chu kỳ từ L1 I-Cache **Tính toán:**

- Giả sử lệnh ARM trung bình 4 bytes
- 32 bytes = 8 lệnh
- Đủ cho 6 decoders (với dư)

### 10.2. L1 D-Cache Bandwidth

**Cấu trúc:** 2 ports, mỗi port 32 B/Cycle **Tổng băng thông:**

- Read/Write:  $32 \text{ B/Cycle} \times 2 = 64 \text{ bytes/chu kỳ}$
- Tại 2.5 GHz (giả sử):
  - $64 \text{ bytes} \times 2.5 \text{ tỷ} = 160 \text{ GB/s}$

**Ý nghĩa:** Đủ để feed cho các execution units và memory operations đồng thời.

### 10.3. Store/Load Buffer Bandwidth

**Load Buffer:**

- $2 \text{ ports} \times 32 \text{ B/Cycle} = 64 \text{ B/Cycle}$
- 96 entries để chờ



#### Store Buffer:

- 2 ports  $\times$  32 B/Cycle = 64 B/Cycle
- 72 entries để chờ

**Tổng:** ~128 B/Cycle cho memory operations

### 10.4. L2 Cache Bandwidth

**Bảng thông:** 32 B/Cycle **Tính toán:**

- 32 bytes  $\times$  2.5 GHz = **80 GB/s**

**So sánh:**

- L1 D-Cache: 160 GB/s (nhanh hơn 2x)
- L2 Cache: 80 GB/s
- RAM: ~20-40 GB/s (chậm hơn 2-4x so với L2)

### 10.5. Ý Nghĩa Của Bandwidth

**Tại sao cần băng thông cao:**

1. **Multiple operations:** E Core có nhiều execution units hoạt động đồng thời, cần truy cập dữ liệu cùng lúc
1. **Hiding latency:** Khi một load miss cache (mất 100+ chu kỳ), CPU cần làm các việc khác. Băng thông cao cho phép nhiều loads/stores chờ đợi cùng lúc.
1. **Streaming data:** Các ứng dụng như video decode, audio processing cần đọc/ghi dữ liệu liên tục.
1. **SIMD operations:** Vector operations xử lý nhiều dữ liệu cùng lúc, cần băng thông lớn.

## KẾT LUẬN

### Tổng Quan E Core

E Core là một vi kiến trúc ARM cân bằng với những đặc điểm nổi bật:

#### 1. Frontend Hiệu Quả:

- 6 decoders xử lý 6 lệnh đơn giản/chu kỳ
- L1 I-Cache 128KB đủ lớn cho hầu hết code
- Branch predictor tốt giảm misprediction penalty

## **2. Unified Register File (180 entries):**

- Thiết kế độc đáo, linh hoạt
- Tiết kiệm diện tích chip
- Phân bổ động giữa INT và FP

## **3. ROQ 196 Entries:**

- Đủ lớn để hỗ trợ out-of-order execution
- Theo dõi ~200 lệnh đang bay
- Tìm và khai thác ILP hiệu quả

## **4. Schedulers Gọn Nhẹ:**

- Tổng 180 entries (76 INT + 72 FP + 32 MEM)
- Đủ để duy trì throughput tốt
- Tiết kiệm năng lượng và diện tích

## **5. Memory System:**

- L1 D-Cache 64KB (64 B/cycle bandwidth)
- L2 6MB chia sẻ giữa các cores
- Load/Store buffers (96/72) hỗ trợ out-of-order memory access

## **6. Pipeline Cân Bằng:**

- Decode: 6  $\mu$ OPs/cycle
- Issue: 14 pops/cycle
- Retire: 6 pops/cycle
- Không có bottleneck rõ ràng

## **Triết Lý Thiết Kế**

E Core được thiết kế với mục tiêu:

**Efficiency (Hiệu quả):**

- Tiết kiệm năng lượng với buffers và caches vừa phải
- Unified PRF tiết kiệm diện tích
- Pipeline hẹp hơn, ít phức tạp hơn

#### **Performance (Hiệu suất):**

- Vẫn duy trì out-of-order execution
- ROQ 196 đủ lớn để khai thác ILP
- Execution units đủ để xử lý tốt

#### **Balance (Cân bằng):**

- Không quá to (tiết kiệm diện tích/năng lượng)
- Không quá nhỏ (vẫn đạt hiệu suất tốt)
- Phù hợp cho đa nhiệm và background tasks

## **Ứng Dụng**

E Core phù hợp cho:

#### **Workload đa luồng nhẹ:**

- Duyệt web với nhiều tabs
- Office applications (Word, Excel)
- Email, chat, messaging
- Background services
- System tasks

#### **Streaming và Media:**

- Video playback
- Music streaming
- Video conferencing (zoom, teams)

#### **Đa nhiệm:**

- Chạy nhiều ứng dụng nhẹ đồng thời
- Background downloads, updates
- Antivirus scanning

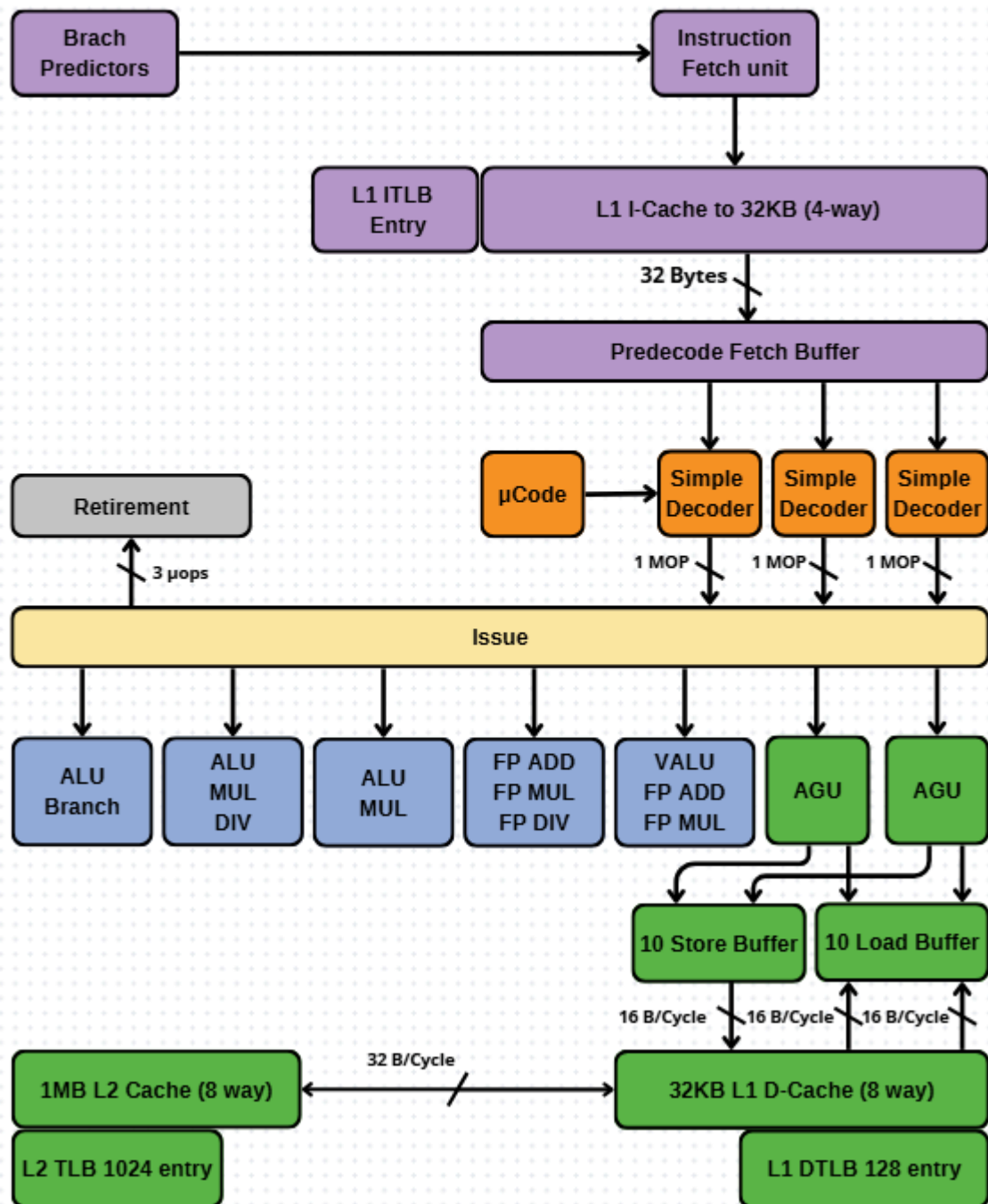
#### **Batch Processing:**

- Rendering với nhiều frames đơn giản
- File compression/decompression
- Data processing tasks

**E Core đại diện cho một thiết kế CPU thông minh - đủ mạnh để xử lý tốt hầu hết các tác vụ hàng ngày, nhưng vẫn tiết kiệm năng lượng và diện tích chip để có thể tích hợp nhiều cores trong cùng một processor.**

## IV. Nano Core

# Nano Core



# GIẢI THÍCH VI KIẾN TRÚC NANO CORE

## MỤC LỤC

1. Giới Thiệu
2. Tổng Quan Quy Trình Xử Lý
3. Các Thành Phần Chi Tiết - Phần Frontend
4. Các Thành Phần Chi Tiết - Phần Backend
5. Hệ Thống Bộ Nhớ
6. Retirement - Giai Đoạn Hoàn Thành
7. Kiến Trúc In-Order Execution
8. Con Số Quan Trọng
9. Quy Trình Xử Lý Một Lệnh
10. Bảng Thông Dữ Liệu

## 1. GIỚI THIỆU

**Nano Core** là một lõi xử lý siêu nhỏ gọn được thiết kế với triết lý "**minimal but functional**" - tối giản nhưng vẫn đủ chức năng. Đây là một bộ xử lý thuộc kiến trúc ARM với pipeline rất đơn giản.

Nano Core được tối ưu hóa cho:

- **Tiết kiệm năng lượng tối đa** (ultra-low power)
- **Diện tích chip nhỏ nhất** (tiny die area)
- **Chi phí thấp** (cost-effective)
- **Nhiệm vụ đơn giản và nhẹ** (simple background tasks)

**Đặc điểm nổi bật:** Nano Core có kiến trúc **in-order execution** (thực thi theo thứ tự), đơn giản hơn nhiều so với các kiến trúc out-of-order phức tạp.

## 2. TỔNG QUAN QUY TRÌNH XỬ LÝ

Nano Core thực hiện quy trình xử lý gồm các giai đoạn chính:

1. **Fetch (Lấy lệnh):** Lấy các lệnh chương trình từ bộ nhớ
2. **Decode (Giải mã):** Phân tích và chuyển đổi lệnh thành micro-operations
3. **Issue (Phát hành):** Gửi các  $\mu$ OPs đến các execution units
4. **Execute (Thực thi):** Thực hiện các phép tính thực sự
5. **Retire (Hoàn thành):** Cam kết kết quả

**Đặc điểm:** Nano Core thực thi các lệnh **theo đúng thứ tự** mà chương trình viết ra. Không có cơ chế reordering phức tạp, giúp thiết kế đơn giản và tiết kiệm năng lượng.

## 3. CÁC THÀNH PHẦN CHI TIẾT - PHẦN FRONTEND

### 3.1. Branch Predictors - Bộ Dự Đoán Nhánh

**Vị trí:** Ở đầu pipeline, trước Instruction Fetch Unit. **Chức năng:** Dự đoán hướng đi của chương trình khi gặp các câu lệnh rẽ nhánh (if/else, loops, function calls). **Tầm quan trọng:**

- Dự đoán đúng: CPU tiếp tục xử lý mượt mà
- Dự đoán sai: Phải hủy các lệnh đã xử lý và bắt đầu lại, mất khoảng 5-10 chu kỳ

**Thiết kế trong Nano Core:** Branch predictor của Nano Core đơn giản hơn, có thể chỉ bao gồm các bộ dự đoán cơ bản:

- **Bimodal predictor:** Dự đoán dựa trên history đơn giản
- **Return address stack:** Nhỏ (8-16 entries) cho function returns

**Ví dụ thực tế:** Giống như việc bạn đoán đèn giao thông sẽ chuyển sang xanh dựa vào thời gian chờ - một dự đoán đơn giản nhưng thường đúng.

### 3.2. Instruction Fetch Unit - Bộ Lấy Lệnh

**Thành phần: L1 ITLB (Entry):**

- ITLB = Instruction Translation Lookaside Buffer
- Số entry nhỏ (có thể khoảng 32-64 entries)
- Chuyển đổi địa chỉ ảo (virtual address) sang địa chỉ vật lý (physical address)

**L1 I-Cache (32KB, 4-way):**

- I-Cache = Instruction Cache

- Kích thước: 32KB - rất nhỏ gọn
- Tổ chức: 4-way set associative (đơn giản)
- Chứa khoảng 7,000-8,000 lệnh máy
- Độ trễ truy cập: 2-3 chu kỳ (rất nhanh do cache nhỏ)

**Bảng thông:** 32 Bytes mỗi chu kỳ được lấy từ cache. **Ví dụ thực tế:** L1 I-Cache giống như một tờ giấy note nhỏ nơi bạn ghi vài công thức quan trọng. Rất nhỏ, nhưng truy cập cực nhanh vì chỉ có vài dòng.

### 3.3. Predecode Fetch Buffer & Simple Decoders

**Cấu trúc:** Có 1 Predecode Fetch Buffer với **3 Simple Decoders**. **Luồng xử lý:**

- Buffer nhận 32 bytes lệnh từ I-Cache
- 3 Simple Decoders phân tích các lệnh đơn giản
- Mỗi decoder tạo ra 1  $\mu$ OP (micro-operation)

**Thông lượng tối đa: 3 lệnh đơn giản mỗi chu kỳ Micro-operations ( $\mu$ OPs):** Là các lệnh nhỏ, đơn giản mà CPU thực sự thực thi bên trong. Một lệnh ARM có thể được chia thành 1-3  $\mu$ OPs. **Decode Width nhỏ:** Với chỉ 3 decoders, Nano Core có pipeline rất hẹp, tiết kiệm transistors và năng lượng. **Ví dụ thực tế:** Giống như một quán ăn nhỏ chỉ có 3 nhân viên thu ngân, mỗi người xử lý một đơn hàng tại một thời điểm. Đơn giản, gọn nhẹ.

### 3.4. $\mu$ Code - Microcode

**$\mu$ Code (Microcode):**

- Xử lý các lệnh phức tạp không thể giải mã bởi Simple Decoder
- Phân tách lệnh phức tạp thành chuỗi nhiều  $\mu$ OPs
- Ví dụ: lệnh string operations, lệnh SIMD phức tạp, lệnh hệ thống

**Hoạt động:**

- Khi Simple Decoder gặp lệnh phức tạp, chuyển đến  $\mu$ Code
- $\mu$ Code tra bảng ROM bên trong, lấy chuỗi  $\mu$ OPs tương ứng
- Gửi từng  $\mu$ OP xuống pipeline theo thứ tự



**Trong Nano Core:**  $\mu$ Code ROM có thể rất nhỏ, chỉ chứa các sequences cơ bản nhất. **Ví dụ thực tế:**  $\mu$ Code như một cuốn sổ tay hướng dẫn mỏng cho các công việc phức tạp. Khi gặp việc khó, mở sổ ra đọc từng bước.

### 3.5. Issue Stage - Giai Đoạn Phát Hành

**Chức năng:** Gửi các  $\mu$ OPs từ decoders trực tiếp đến các execution units. **Đơn giản:** Do Nano Core là in-order, không cần schedulers phức tạp. Issue stage chỉ cần:

- Kiểm tra operands đã sẵn sàng chưa
- Kiểm tra execution unit tương ứng có rảnh không
- Nếu có: gửi lệnh đi
- Nếu không: stall (dừng pipeline)

**Ví dụ thực tế:** Giống như một dây chuyền lắp ráp đơn giản - sản phẩm đi từ trạm này sang trạm kia theo thứ tự, không được vượt lên.

## 4. CÁC THÀNH PHẦN CHI TIẾT - PHẦN BACKEND

### 4.1. Execution Units - Các Đơn Vị Thực Thi

Nano Core có một bộ execution units rất tối giản:

#### 4.1.1. ALU Branch (có 1 unit)

**Chức năng:**

- **ALU operations:** Cộng, trừ, AND, OR, XOR, shift, compare
- **Branch execution:** Tính toán branch target address, resolve branch conditions

**Độ trễ:** 1 chu kỳ cho hầu hết phép tính đơn giản **Ví dụ:** Phép tính  $R1 = R2 + R3$  hoàn thành trong 1 chu kỳ.

#### 4.1.2. ALU MUL DIV (có 1 unit)

**Chức năng:**

- **Multiplication:** Phép nhân số nguyên
- **Division:** Phép chia số nguyên

**Độ trễ:**

- Nhân: 3-5 chu kỳ (tùy độ rộng operands)
- Chia: 10-20 chu kỳ (rất chậm)

**Pipeline:** Unit này không được pipeline (non-pipelined), nghĩa là phải chờ phép tính hiện tại xong mới làm phép tính mới. **Ví dụ thực tế:** Giống như một máy tính cầm tay cơ bản - có thể tính nhân chia nhưng từng phép một, không thể làm nhiều phép cùng lúc.

**4.1.3. ALU MUL (có 1 unit)****Chức năng:**

- **Multiplication:** Phép nhân số nguyên (thêm một unit nữa)

**Lý do có thêm:** Phép nhân được dùng khá thường xuyên, nên Nano Core có 2 multipliers để tăng throughput một chút. **Độ trễ:** 3-5 chu kỳ

**4.1.4. FP ADD FP MUL (có 1 unit)****Chức năng:**

- **FP ADD:** Cộng số thực (floating-point addition)
- **FP MUL:** Nhân số thực (floating-point multiplication)

**Một unit cho cả hai:** Unit này có thể làm hoặc ADD hoặc MUL, nhưng không đồng thời. **Độ trễ:**

- FP ADD: 3-4 chu kỳ
- FP MUL: 4-5 chu kỳ

**Pipeline:** Có thể được pipeline một phần, nhưng throughput thấp.

**4.1.5. VALU FP ADD FP MUL (có 1 unit)****Chức năng:**

- **VALU:** Vector ALU operations (SIMD đơn giản)
- **FP ADD:** Cộng số thực
- **FP MUL:** Nhân số thực

**Vector support:** Hỗ trợ SIMD cơ bản (64-bit hoặc 128-bit vectors), cho phép xử lý 2-4 số cùng lúc. **Độ trễ:** 4-6 chu kỳ cho vector operations **Ứng dụng:** Xử lý âm thanh/video nhẹ, tính toán đơn giản.

#### 4.1.6. AGU (Address Generation Unit) (có 2 units)

**Chức năng:**

- Tính toán địa chỉ bộ nhớ cho Load và Store operations
- Xử lý các chế độ địa chỉ: base+offset, indexed, pre/post-increment

**Độ trễ:** 1 chu kỳ để tính địa chỉ **2 AGUs:** Cho phép tính địa chỉ cho 2 memory operations (1 load + 1 store) trong cùng một chu kỳ. **Ví dụ thực tế:** AGU như một máy tính nhỏ chuyên tính địa chỉ. Khi bạn nói "lấy phần tử thứ 5 trong mảng", AGU tính địa chỉ bộ nhớ thực tế của phần tử đó.

## 4.2. Load và Store Buffers

**Store Buffer:**

- Kích thước: **10 entries**
- Băng thông: 16 B/Cycle
- Chức năng: Giữ các lệnh ghi (store) chờ được ghi vào cache/memory

**Load Buffer:**

- Kích thước: **10 entries**
- Băng thông: 16 B/Cycle
- Chức năng: Giữ các lệnh đọc (load) chờ dữ liệu từ cache/memory

**Rất nhỏ:** Với chỉ 10 entries mỗi buffer, Nano Core chỉ có thể track một số ít memory operations đang pending. Điều này phù hợp với thiết kế in-order đơn giản. **Store-to-Load Forwarding:** Buffers vẫn hỗ trợ tính năng này - load có thể lấy dữ liệu trực tiếp từ store chưa commit để tránh delay. **Ví dụ thực tế:** Giống như một cửa hàng rất nhỏ chỉ có thể xử lý 10 đơn hàng vào và 10 đơn hàng ra tại một thời điểm. Nếu quá tải, phải chờ.

## 5. HỆ THỐNG BỘ NHỚ

### 5.1. L1 Data Cache (L1 D-Cache)

**Kích thước:** 32KB **Tổ chức:** 8-way set associative **Bảng thông:**

- $16 \text{ B/Cycle} \times 2 \text{ ports} = 32 \text{ bytes mỗi chu kỳ}$

**Độ trễ:** 2-3 chu kỳ (rất nhanh do cache rất nhỏ) **Chức năng:** Lưu trữ dữ liệu (variables, arrays, objects) mà chương trình đang sử dụng. **32KB là nhỏ:** Đủ cho các tác vụ nhẹ như:

- Background services
- System monitoring
- Simple I/O operations
- Lightweight daemons

**Ví dụ thực tế:** L1 D-Cache như một ngăn kéo nhỏ trên bàn làm việc, chỉ chứa vài tài liệu đang dùng ngay lúc này.

### 5.2. L1 Data TLB (L1 DTLB)

**Kích thước:** 128 entries **Chức năng:** Chuyển đổi địa chỉ ảo sang địa chỉ vật lý cho các truy cập dữ liệu. **Đủ dùng:** 128 entries có thể cover:

- $128 \text{ pages} \times 4\text{KB/page} = 512\text{KB address space}$
- Hoặc  $128 \times 2\text{MB (huge pages)} = 256\text{MB}$

**Ví dụ thực tế:** L1 DTLB như một danh bạ điện thoại nhỏ có 128 số thường gọi. Đủ cho nhu cầu hàng ngày của một người.

### 5.3. L2 Cache

**Kích thước:** 1MB **Tổ chức:** 8-way set associative **Bảng thông:** 32 B/Cycle **Độ trễ:** ~10-15 chu kỳ **Shared:** Cache này thường được chia sẻ giữa nhiều Nano Cores (ví dụ: 4-8 Nano Cores share 1 L2). **Chức năng:**

- Là bộ đệm cấp 2 cho cả instruction và data
- Khi L1 miss, tìm trong L2
- Nếu L2 miss, phải xuống RAM (100+ chu kỳ)

**1MB nhỏ nhưng đủ:** Cho các workload nhẹ, 1MB L2 chia cho vài cores là chấp nhận được.

**Ví dụ thực tế:** L2 Cache như một tủ tài liệu nhỏ chia sẻ giữa vài bàn làm việc. Không lớn, nhưng đủ để giảm số lần phải đi thư viện (RAM).

## 5.4. L2 TLB

**Kích thước:** 1024 entries **Chức năng:** TLB cấp 2, lớn hơn L1 TLB. **Hierarchy:**

1. L1 ITLB (16 entries) cho instruction
2. L1 DTLB (128 entries) cho data
3. Nếu miss → **L2 TLB (1024 entries)** - dùng chung
4. Nếu vẫn miss → Page table walk (chậm)

**1024 entries coverage:**

- $1024 \times 4\text{KB} = 4\text{MB}$  (với 4KB pages)
- $1024 \times 2\text{MB} = 2\text{GB}$  (với huge pages)

**Đủ cho micro-tasks:** Hầu hết các process nhỏ có working set dưới 4MB, nên hit rate sẽ tốt.

**Ví dụ thực tế:** L2 TLB như một danh bạ lớn hơn (1024 entries) dùng chung cho cả văn phòng. Khi không tìm thấy trong danh bạ cá nhân (L1), tra cứu trong danh bạ chung này.

## 6. RETIREMENT - GIAI ĐOẠN HOÀN THÀNH

**Bảng thông:** 3  $\mu\text{ops}$  (3  $\mu\text{OPs}$  mỗi chu kỳ) **Vị trí:** Giai đoạn cuối cùng của pipeline. **Chức năng quan trọng:**

1. **Commit kết quả:**
  - Do là in-order execution, lệnh ở đầu pipeline tự động là lệnh đầu tiên cần retire
  - Đơn giản hơn nhiều so với out-of-order
  - Cập nhật architectural state
1. **Xử lý ngoại lệ:**
  - Phát hiện exceptions (page faults, divide by zero, etc.)
  - Dừng pipeline, xử lý exception
  - Chuyển control đến exception handler
1. **Giải phóng tài nguyên:**
  - Giải phóng entries trong Load/Store buffers
  - Cho phép instructions mới vào pipeline

### 1. Branch resolution:

- Xác nhận branch prediction đúng hay sai
- Nếu sai: flush pipeline, restart từ địa chỉ đúng
- Trong Nano Core, flush penalty nhỏ hơn do pipeline ngắn

**Retire Width 3:** Nano Core có thể hoàn thành tối đa 3  $\mu$ OPs mỗi chu kỳ, khớp với decode width (3 decoders). **Đơn giản:** Không cần ROQ (Reorder Buffer) phức tạp vì instructions retire theo đúng thứ tự fetch. **Ví dụ thực tế:** Giống như một dây chuyền sản xuất đơn giản - sản phẩm ra đúng thứ tự vào, không cần sắp xếp lại.

## 7. KIẾN TRÚC IN-ORDER EXECUTION

### 7.1. Khái Niệm

Nano Core sử dụng kiến trúc **In-Order Execution** - thực thi theo thứ tự.

**Ý nghĩa:** CPU thực thi các lệnh đúng theo thứ tự mà chương trình viết ra. Lệnh 1  $\rightarrow$  Lệnh 2  $\rightarrow$  Lệnh 3, không có việc đảo thứ tự. **Đơn giản:** Kiến trúc này đơn giản hơn nhiều so với out-of-order execution, giúp:

- Tiết kiệm transistors (chip nhỏ hơn)
- Tiết kiệm năng lượng (ít logic phức tạp)
- Dễ thiết kế và verify
- Chi phí sản xuất thấp hơn

### 7.2. Cách Hoạt Động

Ví dụ chương trình:

```
1. R1 = R2 + R3
2. R4 = R5 + R6
3. R7 = R1 * 2
4. R8 = R4 + R7
```

**In-Order Execution trong Nano Core: Chu kỳ 1:**

- Fetch lệnh 1, 2, 3
- Decode lệnh 1

#### Chu kỳ 2:

- Decode lệnh 2
- Execute lệnh 1 ( $R1 = R2 + R3$ )

#### Chu kỳ 3:

- Decode lệnh 3
- Execute lệnh 2 ( $R4 = R5 + R6$ )
- Lệnh 1 retire

#### Chu kỳ 4:

- Execute lệnh 3 ( $R7 = R1 * 2$ )
- Phải chờ lệnh 1 xong vì phụ thuộc vào R1
- Lệnh 2 retire

#### Chu kỳ 5:

- Execute lệnh 4 ( $R8 = R4 + R7$ )
- Phải chờ lệnh 2 và 3 xong
- Lệnh 3 retire

#### Đặc điểm:

- Lệnh 1 và 2 không được thực thi song song dù độc lập
- Mỗi lệnh phải đợi lệnh trước hoàn thành
- Pipeline có thể bị stall (dừng) khi gặp dependencies

### 7.3. Pipeline Stalls

#### Khi nào xảy ra stall: 1. Data Hazard:

$R1 = R2 + R3$

$R4 = R1 + R5$  ; Phải đợi R1 sẵn sàng → STALL

Pipeline dừng lại cho đến khi R1 được tính xong. 2. Structural Hazard:

$R1 = R2 * R3$  ; Đang dùng MUL unit (mất 4 chu kỳ)

$R4 = R5 * R6$  ; Phải đợi MUL unit rảnh → STALL

### 3. Cache Miss:

$R1 = \text{LOAD} [\text{addr}]$  ; Cache miss, mất 100 chu kỳ

$R2 = R1 + R3$  ; Phải đợi load xong → STALL

### Forwarding giúp giảm stalls:

- Kết quả từ Execute stage có thể được forward trực tiếp đến lệnh tiếp theo
- Không cần đợi Writeback stage
- Giảm stall từ 2-3 chu kỳ xuống còn 0-1 chu kỳ

**Ví dụ thực tế:** In-order execution giống như một hàng người xếp hàng mua vé. Người phía sau phải đợi người phía trước xong mới được đến lượt, dù họ có thể mua vé độc lập.

## 7.4. Lợi Ích của In-Order

### 1. Đơn giản:

- Không cần ROQ, PRRT, complex schedulers
- Logic đơn giản, ít bugs
- Verification dễ dàng hơn

### 2. Tiết kiệm năng lượng:

- Ít transistors → ít power leakage
- Ít logic động → ít dynamic power
- Thích hợp cho battery-powered devices

### 3. Diện tích chip nhỏ:

- Nano Core có thể chiếm chỉ  $\sim 0.2\text{-}0.5 \text{ mm}^2$  trên chip
- Có thể tích hợp nhiều Nano Cores trên một die
- Giảm chi phí sản xuất

### 4. Latency thấp cho simple tasks:



- Pipeline ngắn → latency instruction thấp
- Tốt cho tasks có ít parallelism
- Response time nhanh cho lightweight operations

### 7.5. Trade-offs của In-Order

1. Throughput thấp hơn:

- Không thể khai thác ILP (Instruction-Level Parallelism)
- Nhiều execution units nhàn rỗi
- IPC thường chỉ đạt ~0.5-1.5

2. Sensitive to dependencies:

- Một instruction chậm làm dừng toàn bộ pipeline
- Cache miss ảnh hưởng nghiêm trọng
- Khó che giấu memory latency

3. Không tối ưu cho compute-intensive:

- Tính toán nặng sẽ chậm
- Không phù hợp gaming, rendering, ML
- Không hiệu quả với code có nhiều ILP

## 8. CON SỐ QUAN TRỌNG

Bảng Tổng Hợp Thông Số Nano Core:

| Thành phần   | Giá trị     | Ý nghĩa                        |
|--------------|-------------|--------------------------------|
| Frontend     |             |                                |
| Decoders     | 3           | Giải mã 3 lệnh đơn giản/chu kỳ |
| Decode Width | 3 $\mu$ OPs | Tạo ra 3 $\mu$ OPs/chu kỳ      |
| L1 I-Cache   | 32KB, 4-way | Chứa ~7,000-8,000 lệnh         |

|                    |              |                             |
|--------------------|--------------|-----------------------------|
| L1 ITLB            | Nhỏ (~32-64) | Tra cứu địa chỉ lệnh        |
| Fetch Bandwidth    | 32 B/cycle   | Lấy 32 bytes lệnh/chu kỳ    |
| <b>Execution</b>   |              |                             |
| ALU Branch         | 1 unit       | Phép tính đơn giản + branch |
| ALU MUL DIV        | 1 unit       | Nhân và chia số nguyên      |
| ALU MUL            | 1 unit       | Nhân số nguyên (thêm)       |
| FP ADD FP MUL      | 1 unit       | Cộng/nhân số thực           |
| VALU FP ADD FP MUL | 1 unit       | Vector/FP operations        |
| AGU                | 2 units      | Tính địa chỉ memory         |
| <b>Memory</b>      |              |                             |
| Store Buffer       | 10 entries   | Chờ ghi 10 lần              |
| Load Buffer        | 10 entries   | Chờ đọc 10 lần              |
| L1 D-Cache         | 32KB, 8-way  | Dữ liệu gần nhất            |
| L1 D-Cache BW      | 32 B/cycle   | 2 ports × 16 B              |
| L1 DTLB            | 128 entries  | Tra cứu địa chỉ dữ liệu     |
| L2 Cache           | 1MB, 8-way   | Cache nhỏ chia sẻ           |
| L2 TLB             | 1024 entries | Tra cứu địa chỉ cấp 2       |
| <b>Retirement</b>  |              |                             |
| Retire Width       | 3 μops       | Hoàn thành 3 μOPs/chu kỳ    |

|                     |              |                      |
|---------------------|--------------|----------------------|
| <b>Architecture</b> |              |                      |
| Execution Order     | In-Order     | Thực thi theo thứ tự |
| Pipeline Stages     | ~8-10 stages | Pipeline ngắn        |

## Điểm Đặc Biệt:

### 1. Pipeline Cân Bằng và Hẹp:

- Decode: 3  $\mu$ OPs/cycle
- Retire: 3  $\mu$ ops/cycle
- Pipeline hẹp, đơn giản, tiết kiệm năng lượng

### 2. In-Order Execution:

- Không có ROQ, PRRT, complex schedulers
- Logic đơn giản
- Tiết kiệm transistors đáng kể

### 3. Minimal Execution Units:

- Chỉ 1-2 units cho mỗi loại operation
- Đủ cho simple tasks
- Tiết kiệm diện tích chip

### 4. Tiny Caches:

- L1-I 32KB, L1-D 32KB
- L2 1MB (shared)
- Nhỏ nhưng đủ nhanh và đủ dùng

### 5. Small Buffers:

- Load/Store buffers chỉ 10 entries mỗi loại
- Đủ cho in-order pipeline
- Minimal tracking overhead

### 6. Ultra-Low Power:

- Tất cả thiết kế hướng đến tiết kiệm năng lượng tối đa
- Có thể chạy ở frequency thấp (500MHz - 1.5GHz)
- Power consumption có thể dưới 100mW per core

## 9. QUY TRÌNH XỬ LÝ MỘT LỆNH

Hãy theo dõi một lệnh cụ thể đi qua toàn bộ pipeline của Nano Core.

**Lệnh mẫu:** `R3 = R1 + R2` (cộng hai số nguyên)

### Bước 1: Fetch (2-3 chu kỳ)

1. **Branch Predictor** dự đoán địa chỉ lệnh tiếp theo
2. **L1 ITLB** tra cứu: chuyển địa chỉ ảo → vật lý (1 chu kỳ)
3. **L1 I-Cache** (32KB) kiểm tra:
  - **Cache hit:** Lấy 32 bytes chứa lệnh (1 chu kỳ)
  - **Cache miss:** Lấy từ L2 (10-15 chu kỳ) hoặc RAM (100+ chu kỳ)

Lệnh được gửi đến **Predecode Fetch Buffer**

### Bước 2: Decode (1 chu kỳ)

1. **Simple Decoder** (1 trong 3) nhận lệnh `ADD R3, R1, R2`
2. Phân tích: "Phép cộng hai thanh ghi số nguyên"
3. Tạo 1  $\mu$ OP: `ADD\_INT R3, R1, R2`
4. Gửi  $\mu$ OP xuống Issue stage

### Bước 3: Issue (0-N chu kỳ)

1. **Issue Logic** kiểm tra:
  - **R1 đã sẵn sàng chưa?** (đã được tính xong bởi lệnh trước)
  - **R2 đã sẵn sàng chưa?**
  - **ALU có rảnh không?**
1. **Trường hợp sẵn sàng:**
  - Gửi lệnh đến ALU ngay (0 chu kỳ stall)
1. **Trường hợp chưa sẵn sàng:**
  - **Stall pipeline** cho đến khi:

- R1 và R2 sẵn sàng
- ALU rảnh
- Có thể mất 1-N chu kỳ

#### **Bước 4: Execute (1 chu kỳ)**

1. **ALU Branch** unit nhận lệnh
2. Đọc giá trị:
  - **R1** từ register file
  - **R2** từ register file

Thực hiện phép cộng (1 chu kỳ cho ADD)

Kết quả tạm thời giữ trong pipeline register

#### **Bước 5: Writeback (1 chu kỳ)**

1. Kết quả từ ALU được ghi vào **R3** trong register file
2. **R3 = ready**, các lệnh sau có thể dùng R3 ngay

#### **Bước 6: Retire (implicit, 0 chu kỳ thêm)**

1. Do là in-order, lệnh tự động retire khi writeback xong
2. Không cần bước riêng như out-of-order cores
3. Lệnh biến mất khỏi pipeline

### **Tổng Thời Gian:**

#### **Trường hợp tốt nhất (cache hit, không stall):**

- Fetch: 2 chu kỳ
- Decode: 1 chu kỳ
- Issue: 0 chu kỳ (sẵn sàng ngay)
- Execute: 1 chu kỳ
- Writeback: 1 chu kỳ
- **Tổng: ~5 chu kỳ từ fetch đến hoàn thành**

#### **Trường hợp có stall:**

- Nếu R1 chưa sẵn sàng (đang được tính bởi lệnh trước mất 4 chu kỳ)

- Issue stage phải stall 3-4 chu kỳ
- **Tổng: ~8-9 chu kỳ**

#### Pipeline throughput:

- Trong steady state (không stall): 3 lệnh/chu kỳ decode
- Nhưng thực tế thường có stalls: throughput trung bình ~0.5-1.5 IPC (Instructions Per Cycle)

**Ví dụ thực tế:** Giống như một dây chuyền lắp ráp đơn giản với 5 trạm. Mỗi sản phẩm mất 5 bước, nhưng nếu một trạm gặp vấn đề (stall), toàn bộ dây chuyền phải dừng lại.

## 10. BẢNG THÔNG DỮ LIỆU

### 10.1. Instruction Fetch Bandwidth

**Bảng thông:** 32 bytes/chu kỳ từ L1 I-Cache **Tính toán:**

- Giả sử lệnh ARM trung bình 4 bytes
- 32 bytes = 8 lệnh
- Đủ rộng cho 3 decoders (với dư)

**Ý nghĩa:** Fetch bandwidth không phải bottleneck. Decode width (3) là giới hạn thực sự.

### 10.2. L1 D-Cache Bandwidth

**Cấu trúc:** 2 ports, mỗi port 16 B/Cycle **Tổng băng thông:**

- Read/Write:  $16 \text{ B/Cycle} \times 2 = \mathbf{32 \text{ bytes/chu kỳ}}$
- Tại 1 GHz (giả sử):
  - $32 \text{ bytes} \times 1 \text{ tỷ} = \mathbf{32 \text{ GB/s}}$

**Ý nghĩa:** Đủ để feed cho 2 AGUs và các execution units trong môi trường in-order.

### 10.3. Store/Load Buffer Bandwidth

**Load Buffer:**

- 16 B/Cycle
- 10 entries để chờ

#### Store Buffer:

- 16 B/Cycle
- 10 entries để chờ

**Tổng:** ~32 B/Cycle cho memory operations **Nhỏ nhưng đủ:** Với in-order execution và chỉ 2 AGUs, bảng thông này chấp nhận được.

### 10.4. L2 Cache Bandwidth

**Bảng thông:** 32 B/Cycle **Tính toán:**

- $32 \text{ bytes} \times 1 \text{ GHz} = \mathbf{32 \text{ GB/s}}$

**Lưu ý:** L2 được chia sẻ giữa nhiều Nano Cores, nên băng thông thực tế mỗi core thấp hơn.

### 10.5. Memory Bandwidth Requirements

**Ước tính cho workload nhẹ: Trường hợp: Background system service**

- L1 cache hit rate: ~95%
- L2 cache hit rate (của L1 misses): ~80%
- Chỉ ~1% truy cập phải xuống RAM

**Ví dụ tính toán:**

- 1 GHz, IPC = 1.0
- 1 billion instructions/giây
- Giả sử 30% là memory operations: 300 million mem ops/giây
- 95% hit L1: chỉ 15 million xuống L2
- 80% hit L2: chỉ 3 million xuống RAM
- $3 \text{ million} \times 16 \text{ bytes} = 48 \text{ MB/s}$  bandwidth đến RAM

**Kết luận:** Nano Core có memory bandwidth requirements rất thấp, phù hợp cho embedded systems và low-power devices.

## KẾT LUẬN

### Tổng Quan Nano Core

Nano Core là một vi kiến trúc ARM siêu nhỏ gọn với những đặc điểm nổi bật:

### **1. In-Order Execution:**

- Đơn giản nhất trong các kiến trúc CPU hiện đại
- Không có ROQ, PRRT, complex schedulers
- Tiết kiệm transistors, năng lượng, diện tích

### **2. Minimal Pipeline (3-wide):**

- 3 decoders xử lý 3 lệnh đơn giản/chu kỳ
- 3  $\mu$ ops retire/chu kỳ
- Pipeline hẹp, ngắn (~8-10 stages)

### **3. Tiny Caches:**

- L1 I-Cache 32KB
- L1 D-Cache 32KB
- L2 1MB (shared)
- Nhỏ, nhanh, đủ dùng cho lightweight tasks

### **4. Minimal Execution Units:**

- 1 ALU Branch
- 2 Multipliers (1 có divide)
- 2 FP/SIMD units (shared functionality)
- 2 AGUs
- Đủ đa dạng nhưng không dư thừa

### **5. Small Buffers:**

- Load/Store buffers: 10 entries mỗi loại
- TLBs: 128 (L1 DTLB), 1024 (L2)
- Minimal tracking overhead

### **6. Ultra-Low Power Design:**

- Có thể chạy dưới 100mW per core
- Clock gating, power gating aggressive
- DVFS friendly (có thể chạy 500MHz-1.5GHz)



## Triết Lý Thiết Kế

Nano Core được thiết kế với mục tiêu:

### Minimalism (Tối giản):

- Chỉ giữ lại những gì thực sự cần thiết
- Loại bỏ mọi phức tạp không cần thiết
- "Less is more"

### Energy First (Năng lượng là ưu tiên số 1):

- Mỗi quyết định thiết kế đều hướng đến tiết kiệm năng lượng
- Chấp nhận hy sinh performance để đổi lấy efficiency
- Phù hợp battery-powered devices

### Cost Effective (Hiệu quả chi phí):

- Diện tích chip nhỏ → nhiều cores trên một wafer
- Thiết kế đơn giản → verification nhanh
- Yield cao → giá thành thấp

### Good Enough Performance:

- Không cần nhanh nhất
- Chỉ cần "đủ nhanh" cho lightweight tasks
- IPC ~0.5-1.5 là chấp nhận được

## Ứng Dụng

Nano Core phù hợp cho:

### 1. IoT Devices (Internet of Things):

- Smart home sensors
- Wearable devices
- Environmental monitors
- Low-power wireless modules

### 2. Embedded Systems:

- Microcontrollers
- Industrial control systems
- Automotive ECUs (simple functions)
- Consumer electronics

### 3. Background Services:

- System daemons
- Monitoring services
- Network stack processing
- Timer/interrupt handlers

### 4. Always-On Tasks:

- Voice wake-word detection
- Sensor hub
- Low-power audio playback
- Basic UI responsiveness

### 5. Battery-Critical Applications:

- Smartphones (cho always-on coprocessor)
- Smartwatches
- Fitness trackers
- E-readers

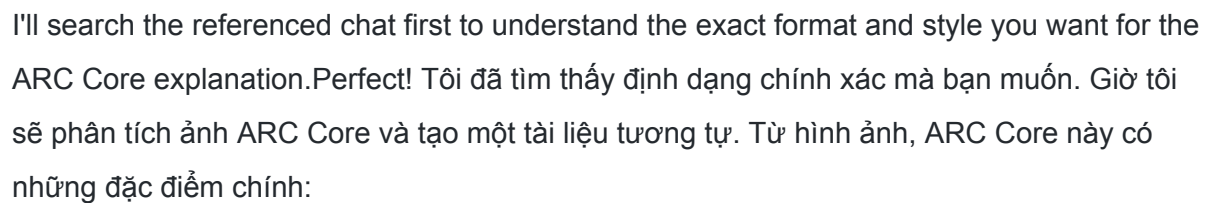
## Khi Nào KHÔNG Dùng Nano Core

### Không phù hợp cho:

- Gaming
- Video rendering
- Machine Learning training
- Scientific computing
- Compilation
- Database servers
- Bất kỳ workload compute-intensive nào

**Lý do:** In-order execution và minimal resources không thể cung cấp performance cần thiết cho các tasks này. **Nano Core đại diện cho triết lý "efficiency over performance" - một**

## V. ARC core



## GIẢI THÍCH VI KIỆN TRÚC ARC CORE

## Dành Cho Người Không Chuyên

# MỤC LỤC

1. Giới Thiệu
2. Tổng Quan Quy Trình Xử Lý
3. Các Thành Phần Chi Tiết
4. Hệ Thống Bộ Nhớ
5. Retirement - Giai Đoạn Hoàn Thành

6. Kiến Trúc Out-of-Order Execution
7. Con Số Ẩn Tượng
8. Quy Trình Xử Lý Một Lệnh
9. Bandwidth - Băng Thông Dữ Liệu
10. Ứng Dụng Thực Tế

## 1. GIỚI THIỆU

**ARC Core** là viết tắt của **Advanced Rendering Core** - lõi xử lý đồ họa và gaming chuyên nghiệp. Đây là một vi kiến trúc được thiết kế đặc biệt để tối ưu hóa cho các tác vụ game, rendering 3D, xử lý đồ họa, và các workload liên quan đến tính toán song song cao.

ARC Core thường đi kèm với **SPE (Spatial Processing Engine)** - bộ xử lý không gian chuyên biệt cho ray tracing và các hiệu ứng đồ họa tiên tiến.

### Đặc điểm nổi bật:

- Thiết kế cân bằng giữa integer và floating-point performance
- Tối ưu hóa cho throughput cao với game và rendering
- Out-of-order execution mạnh mẽ
- Hệ thống cache lớn để xử lý texture và geometry

## 2. TỔNG QUAN QUY TRÌNH XỬ LÝ

ARC Core thực hiện quy trình xử lý chuẩn 5 bước của kiến trúc hiện đại:

1. **Lấy lệnh (Fetch):** Lấy chương trình từ bộ nhớ
2. **Giải mã (Decode):** Hiểu lệnh đó có nghĩa là gì
3. **Phát hành (Issue):** Chuẩn bị thực hiện
4. **Thực thi (Execute):** Thực sự làm việc
5. **Hoàn thành (Retire):** Ghi nhận kết quả

**Đặc điểm của ARC Core:** Được tối ưu hóa để xử lý nhiều phép tính floating-point song song, phù hợp với đồ họa 3D và physics simulation trong game.

## 3. CÁC THÀNH PHẦN CHI TIẾT

### 3.1. Branch Predictors - Bộ Dự Đoán Nhánh

**Chức năng:** Dự đoán chương trình sẽ chạy theo hướng nào tiếp theo. **Ví dụ thực tế:** Trong game, khi nhân vật kiểm tra điều kiện "nếu máu < 20% thì uống thuốc", bộ dự đoán sẽ học pattern và đoán trước hành động, giúp game không bị lag. **Vai trò trong gaming:** Giúp giảm stutter và giữ frame rate ổn định.

### 3.2. Instruction Fetch Unit - Bộ Lấy Lệnh

**Thành phần:**

- **Instruction Fetch Unit:** Lấy lệnh từ bộ nhớ
- **L1 ITLB (384 Entry):** Bảng tra cứu địa chỉ lệnh
- **L1 I-Cache (192KB, 6-way):** Bộ nhớ đệm lưu trữ lệnh

**Bảng thông:** 64 Bytes mỗi chu kỳ **Ý nghĩa:** Cache lớn giúp lưu trữ nhiều shader code và game logic, giảm thiểu việc phải đọc từ RAM chậm. **Ví dụ thực tế:** Giống như việc game lưu các đoạn code shader thường dùng trong bộ nhớ nhanh, không phải load lại liên tục.

### 3.3. Predecode Fetch Buffer & Simple Decoders

**Cấu trúc:** 2 nhóm Predecode Fetch Buffer, mỗi bên có 6 Simple Decoders. **Thông lượng:**

- Mỗi decoder xử lý 1  $\mu$ OP
- Mỗi bên: 6 decoders  $\times$  1  $\mu$ OP = 6  $\mu$ OPs
- **Tổng: 12  $\mu$ OPs/chu kỳ**

**Luồng dữ liệu:**

- Instruction Fetch Unit  $\rightarrow$  64 bytes  $\rightarrow$  Predecode Fetch Buffer
- Predecode Fetch Buffer  $\rightarrow$  6 Simple Decoders (mỗi bên)
- Decoders  $\rightarrow$  Decode Queue

**Ví dụ thực tế:** Giống như có 12 người phiên dịch đồng thời, đọc script game và chuyển thành hành động cụ thể.

### 3.4. $\mu$ Code & MOP Cache

**$\mu$ Code (Microcode):** Xử lý các lệnh phức tạp, đặc biệt là các instruction đồ họa chuyên biệt.

**MOP Cache:** Lưu trữ các Micro-Operations đã giải mã. **Lợi ích cho gaming:**

- Các shader instruction thường lặp lại → cache giúp tránh giải mã lại
- Tiết kiệm năng lượng cho laptop gaming và console
- Giảm độ trễ khi render frame

**Ví dụ thực tế:** Khi render 1000 cây trong rừng, shader code giống nhau được dùng lại → MOP Cache giúp xử lý nhanh hơn.

### 3.5. Decode Queue - Hàng Đợi Giải Mã

**Cấu trúc:**

- 2 Decode Queue, mỗi bên nhận 6  $\mu$ OPs từ decoders
- MUX (Multiplexer) kết hợp thành **12  $\mu$ OPs**
- Gửi xuống các Physical Register Files và Schedulers

**Bảng thông:** 6  $\mu$ OPs từ mỗi Decode Queue → 12  $\mu$ OPs/cycle **Vai trò:** Đảm bảo luồng lệnh liên tục xuống pipeline, không bị đứt quãng.

### 3.6. Physical Register Files - Bộ Thanh Ghi Vật Lý

**Cấu trúc:**

- **Integer PRF (Physical Register File):** ~576 thanh ghi số nguyên
- **Vector PRF (Physical Register File):** ~512 thanh ghi vector/floating-point

**Ý nghĩa:**

- **Integer PRF** xử lý: địa chỉ memory, index, control logic
- **Vector PRF** xử lý: tọa độ 3D, màu sắc, vector, matrix operations

**Ví dụ thực tế trong game:**

- **Integer:** Tính vị trí nhân vật trong mảng `map[x][y]`
- **Vector:** Tính ánh sáng, phản chiếu, tọa độ đỉnh 3D (x, y, z, w)

**Số lượng lớn (~1088 tổng cộng)** cho phép ARC Core theo dõi rất nhiều phép tính đang bay trong pipeline, tăng khả năng out-of-order execution.

### 3.7. Physical Register Reclaim Table (PRRT)

**Kích thước:** 1280 entries **Chức năng:** Quản lý việc phân bổ và thu hồi thanh ghi vật lý. **Vai trò quan trọng:** Với số lượng thanh ghi lớn (~1088), cần một hệ thống quản lý thông minh để:

- Biết thanh ghi nào đang dùng
- Thu hồi thanh ghi đã dùng xong
- Phân bổ lại cho lệnh mới

**Ví dụ thực tế:** Như một hệ thống quản lý phòng họp trong công ty lớn - theo dõi phòng nào đang dùng, phòng nào trống, ai đặt phòng.

### 3.8. Retire Queue (ROQ) - Hàng Đợi Hoàn Thành

**Kích thước:** 600 coalesced entries **Chức năng:** Đảm bảo các lệnh hoàn thành đúng thứ tự chương trình, dù được thực thi không theo thứ tự. **Con số 600:** Cho phép ARC Core "nhìn xa" 600 lệnh trong tương lai để tìm cơ hội thực thi song song. **So sánh:**

- C Core: 480 entries
- P Core: 620 entries
- **ARC Core: 600 entries** - cân bằng, phù hợp cho gaming workload

**Ví dụ trong game:** Khi render 1 frame, có hàng trăm lệnh vẽ object - ROQ cho phép GPU "nhìn trước" và xử lý song song những object không che khuất nhau.

### 3.9. Schedulers - Bộ Lập Lịch

Đây là "não" phân công công việc. ARC Core có hệ thống scheduler cân bằng:

#### 3.9.1. 32 INT Scheduler (x6 bộ)

**Tổng capacity:**  $32 \times 6 = 192$  entries cho integer operations **Chức năng:** Lập lịch cho phép tính số nguyên:

- Tính địa chỉ memory
- Logic điều khiển (if/else)
- Counter, loop index

**Execution Units điều khiển:**

- **ALU Branch:** Xử lý phép tính và nhánh

- **ALU:** Phép tính cơ bản (ADD, SUB, AND, OR, XOR, shift)

**Ví dụ trong game:** Tính index của texture trong mảng, kiểm tra collision, update score.

### 3.9.2. 24 INT Scheduler (x2 bộ)

**Tổng capacity:**  $24 \times 2 = 48$  entries **Chức năng:** Lập lịch cho phép tính số nguyên phức tạp hơn **Execution Units:**

- **ALU MUL DIV:** Nhân và chia số nguyên

**Lưu ý:** Nhân và chia chậm hơn cộng/trừ nhiều, nên cần scheduler riêng. **Ví dụ trong game:** Tính damage = base\_damage  $\times$  multiplier, hoặc chia điểm kinh nghiệm cho team.

### 3.9.3. 48 INT Scheduler (x1 bộ)

**Capacity:** 48 entries **Chức năng:** Scheduler lớn cho các integer operations tổng hợp **Execution Units:**

- **ALU MUL:** Nhân số nguyên chuyên biệt

**Vai trò:** Xử lý các phép nhân nặng, như tính hash, encryption trong game online.

### 3.9.4. 52 FP Scheduler (x4 bộ)

**Tổng capacity:**  $52 \times 4 = 208$  entries cho floating-point operations **Chức năng:** Lập lịch cho phép tính số thực (float/double), rất quan trọng cho graphics. **Execution Units điều khiển:**

- **VALU (Vector ALU):** Phép tính vector (cộng, trừ vector)
- **VMUL (Vector Multiply):** Nhân vector
- **FP ADD:** Cộng floating-point
- **FP MUL:** Nhân floating-point
- **FP DIV:** Chia floating-point

**Ví dụ trong game:**

- Tính tọa độ 3D sau khi xoay: new\_pos = rotation\_matrix  $\times$  old\_pos
- Tính ánh sáng: color = ambient + diffuse  $\times$  dot(normal, light\_dir)
- Physics: velocity += acceleration  $\times$  deltaTime

**Đây là trái tim của ARC Core** - với 208 entries FP scheduler, có thể xử lý rất nhiều phép tính đồ họa song song.



### 3.9.5. 96 MEM Scheduler (x1 bộ)

**Capacity:** 96 entries **Chức năng:** Lập lịch cho các thao tác bộ nhớ (Load/Store) **Execution Units điều khiển:**

- **AGU (Address Generation Unit):** Tính địa chỉ memory
- **Store Data/Address:** Ghi dữ liệu vào memory
- **Load Data:** Đọc dữ liệu từ memory

**Vai trò quan trọng trong gaming:**

- Load texture từ memory
- Đọc geometry data (vertices, triangles)
- Ghi framebuffer (kết quả render)

**Con số 96 entries** rất lớn, cho phép ARC Core xử lý nhiều memory operations song song - quan trọng khi load nhiều texture cùng lúc.

## 4. HỆ THỐNG BỘ NHỚ

Hệ thống bộ nhớ là "huyết mạch" của CPU. ARC Core có cấu trúc 3 tầng:

### 4.1. L1 Cache - Cấp 1 (Nhanh Nhất)

#### L1 I-Cache (Instruction Cache)

- **Kích thước:** 192KB
- **Associativity:** 6-way
- **Chức năng:** Lưu trữ lệnh (instruction)
- **Độ trễ:** ~4-5 chu kỳ

#### L1 D-Cache (Data Cache)

- **Kích thước:** 128KB
- **Associativity:** 8-way
- **Chức năng:** Lưu trữ dữ liệu (biến, mảng, texture)
- **Độ trễ:** ~4-5 chu kỳ
- **Băng thông:** 32 B/Cycle (multiple ports)

**Ví dụ trong game:**

- **L1 I-Cache:** Lưu shader code, AI script
- **L1 D-Cache:** Lưu vertex data gần đây, texture pixels

## **4.2. L2 Cache - Cấp 2 (Chia Sẻ)**

- **Kích thước:** 24MB
- **Associativity:** 16-way shared
- **Chức năng:** Bộ nhớ đệm lớn chia sẻ cho cả instruction và data
- **Độ trễ:** ~15-20 chu kỳ
- **Băng thông:** 32 B/Cycle

**Ý nghĩa của 24MB:**

- Lớn hơn nhiều so với L1 ( $192\text{KB} + 128\text{KB} = 320\text{KB}$ )
- Có thể chứa nhiều texture, geometry, game assets
- Giảm đáng kể việc phải truy cập RAM chậm

**Ví dụ thực tế:** L2 Cache 24MB có thể chứa ~6000 texture 4K, hoặc geometry của cả một level game nhỏ.

## **4.3. TLB (Translation Lookaside Buffer)**

**L1 ITLB:**

- **Kích thước:** 384 entries
- **Chức năng:** Tra cứu địa chỉ lệnh (instruction)

**L1 DTLB:**

- **Kích thước:** 384 entries
- **Chức năng:** Tra cứu địa chỉ dữ liệu (data)

**L2 TLB:**

- **Kích thước:** 4096 entries
- **Chức năng:** TLB lớn chia sẻ

**Vai trò:** Dịch địa chỉ ảo (virtual address) thành địa chỉ vật lý (physical address). **Ví dụ thực tế:** Giống như một cuốn sổ tra địa chỉ - khi game muốn truy cập "texture[100]", TLB cho biết nó nằm ở đâu trong RAM thật.

#### 4.4. Store Buffer & Load Buffer

**Store Buffer:**

- **Kích thước:** 110 entries
- **Chức năng:** Đệm các lệnh ghi (store) chờ ghi vào cache/memory
- **Băng thông:**  $32 \text{ B/Cycle} \times 3 \text{ ports} = 96 \text{ B/Cycle}$

**Load Buffer:**

- **Kích thước:** 160 entries
- **Chức năng:** Đệm các lệnh đọc (load) chờ dữ liệu từ cache/memory
- **Băng thông:**  $32 \text{ B/Cycle} \times 3 \text{ ports} = 96 \text{ B/Cycle}$

**Ý nghĩa trong gaming:**

- **Load Buffer lớn (160):** Cho phép load nhiều texture/vertex song song
- **Store Buffer lớn (110):** Cho phép ghi nhiều pixels vào framebuffer song song

**Ví dụ thực tế:** Khi render 1000 triangles, Load Buffer giúp load tọa độ của nhiều triangles cùng lúc, không phải đợi từng cái một.

## 5. RETIREMENT - GIAI ĐOẠN HOÀN THÀNH

**Chức năng:** Giai đoạn cuối cùng, nơi kết quả được chính thức "cam kết" (commit). **Thông lượng:** 12 pops (instructions retired per cycle) **Luồng dữ liệu:**

- ROQ (600 entries) → Retirement → Cập nhật trạng thái kiến trúc
- 12 pops nghĩa là ARC Core có thể hoàn thành 12 lệnh mỗi chu kỳ (lý thuyết)

**Vai trò quan trọng:**

1. Đảm bảo lệnh hoàn thành đúng thứ tự chương trình (in-order retirement)
2. Xử lý ngoại lệ (exception, interrupt)
3. Giải phóng tài nguyên (thanh ghi, buffers)

#### 4. Cập nhật architectural state

**Ví dụ thực tế trong game:** Giống như việc game engine "commit" các frame đã render - dù các objects được vẽ không theo thứ tự (out-of-order), nhưng frame cuối cùng phải hiển thị đúng.

## 6. KIẾN TRÚC OUT-OF-ORDER EXECUTION

### 6.1. Khái Niệm

ARC Core sử dụng kiến trúc **thực thi không theo thứ tự** (out-of-order execution).

**Ý nghĩa:** CPU có thể thực hiện lệnh không đúng thứ tự chương trình viết, miễn là kết quả cuối cùng vẫn đúng. **Ví dụ trong game rendering: Code shader:**

1.  $\text{color1} = \text{texture}(\text{tex1}, \text{uv1}) \rightarrow$  phải đợi đọc texture (chậm)
2.  $\text{color2} = \text{texture}(\text{tex2}, \text{uv2}) \rightarrow$  phải đợi đọc texture (chậm)
3.  $\text{brightness} = \text{dot}(\text{normal}, \text{light}) \rightarrow$  tính toán nhanh, không phụ thuộc vào 1 và 2

**Out-of-order execution:**

- CPU nhận ra lệnh 3 không phụ thuộc vào 1 và 2
- Thực hiện lệnh 3 ngay, trong khi chờ texture load cho lệnh 1 và 2
- Khi texture load xong, làm tiếp lệnh 1 và 2

**Kết quả:** Tận dụng thời gian chờ, không để CPU nhàn rỗi.

### 6.2. Lợi Ích Cho Gaming

- **Tăng frame rate:** Tận dụng tối đa ALU và FP units
- **Ản độ trễ memory:** Khi chờ texture load, làm phép tính khác
- **Throughput cao:** Render nhiều triangles/pixels song song
- **Giảm stutter:** Pipeline luôn bận, không bị đứng

### 6.3. Yêu Cầu Phần Cứng

Out-of-order execution cần:

- **ROQ lớn (600 entries):** "Nhìn xa" nhiều lệnh
- **Nhiều scheduler:** Theo dõi phụ thuộc giữa các lệnh
- **Nhiều thanh ghi (~1088):** Lưu trữ kết quả tạm
- **Register renaming:** Tránh xung đột thanh ghi

## 7. CON SỐ ẢN TƯỢNG

Hãy cùng xem các con số quan trọng của ARC Core:

| Thành phần     | Con số    | Ý nghĩa                      |
|----------------|-----------|------------------------------|
| Decoders       | 12        | Giải mã 12 lệnh/chu kỳ       |
| Integer PRF    | ~576      | Lưu trữ 576 số nguyên tạm    |
| Vector PRF     | ~512      | Lưu trữ 512 vector/float tạm |
| ROQ            | 600       | Theo dõi 600 lệnh đang bay   |
| PRRT           | 1280      | Quản lý 1280 thanh ghi       |
| INT Schedulers | 288 total | 192+48+48 entries cho INT    |
| FP Schedulers  | 208 total | 52×4 entries cho FP          |
| MEM Scheduler  | 96        | Lập lịch 96 lệnh memory      |
| Store Buffer   | 110       | Chờ ghi 110 lần              |
| Load Buffer    | 160       | Chờ đọc 160 lần              |
| L1 I-Cache     | 192KB     | Lưu lệnh nhanh nhất          |
| L1 D-Cache     | 128KB     | Lưu dữ liệu nhanh nhất       |
| L2 Cache       | 24MB      | Bộ nhớ đệm lớn chia sẻ       |

|              |         |                           |
|--------------|---------|---------------------------|
| L2 TLB       | 4096    | Tra cứu 4096 địa chỉ      |
| Retire Width | 12 pops | Hoàn thành 12 lệnh/chu kỳ |

#### Điểm nổi bật:

- **FP Schedulers lớn (208):** Tối ưu cho đồ họa
- **Load/Store Buffers lớn (270 total):** Xử lý nhiều memory I/O
- **L2 Cache 24MB:** Rất lớn cho gaming
- **Decode width 12:** Throughput cao

## 8. QUY TRÌNH XỬ LÝ MỘT LỆNH

Hãy theo dõi một lệnh shader đơn giản:

**Lệnh:** ``color = texture(diffuseMap, uv) * lightIntensity;``

### Bước 1: Fetch (Lấy Lệnh)

- Branch Predictors đoán lệnh tiếp theo
- Instruction Fetch Unit đọc lệnh từ L1 I-Cache (192KB)
- Nếu cache miss, đọc từ L2 (24MB)

### Bước 2: Decode (Giải Mã)

- Lệnh phức tạp →  $\mu$ Code ROM phân tách thành nhiều  $\mu$ OPs:
  - $\mu$ OP1: Load texture address
  - $\mu$ OP2: Load UV coordinates
  - $\mu$ OP3: Texture fetch (memory read)
  - $\mu$ OP4: Load lightIntensity
  - $\mu$ OP5: Multiply color  $\times$  lightIntensity
- 6 Simple Decoders  $\times$  2 = 12  $\mu$ OPs/cycle
- MOP Cache: Nếu lệnh này đã decode trước đó, lấy từ cache

### Bước 3: Register Renaming

- $\mu$ OPs được gán thanh ghi vật lý từ PRF

- $\mu\text{OP}3 \rightarrow \text{Vector PRF}$  (cho texture color)
- $\mu\text{OP}4 \rightarrow \text{Vector PRF}$  (cho lightIntensity)
- $\mu\text{OP}5 \rightarrow \text{Vector PRF}$  (cho kết quả)
- PRRT (1280 entries) theo dõi thanh ghi nào đang dùng

#### **Bước 4: Issue (Phát Hành)**

- $\mu\text{OPs}$  vào ROQ (600 entries) chờ thực thi
- Schedulers phân loại:
  - $\mu\text{OP}1, \mu\text{OP}2 \rightarrow \text{MEM Scheduler}$  (96 entries)
  - $\mu\text{OP}3 \rightarrow \text{MEM Scheduler}$  (texture fetch)
  - $\mu\text{OP}4 \rightarrow \text{MEM Scheduler}$  (load data)
  - $\mu\text{OP}5 \rightarrow \text{FP Scheduler}$  (52 entries) - phép nhân floating-point

#### **Bước 5: Execute (Thực Thi) - Out-of-Order!**

- $\mu\text{OP}1, \mu\text{OP}2$ : AGU tính địa chỉ, gửi đến Load Buffer (160 entries)
- $\mu\text{OP}3$ : Load texture từ L1 D-Cache (128KB)
  - Nếu cache hit:  $\sim 5$  cycles
  - Nếu cache miss: đọc từ L2 (24MB)  $\sim 20$  cycles
- $\mu\text{OP}4$ : Load lightIntensity từ L1 D-Cache
- $\mu\text{OP}5$ : Trong khi chờ texture (nếu miss), CPU có thể làm  $\mu\text{OP}5$  của pixel khác (out-of-order)
- VMUL (Vector Multiply):  $\text{color} \times \text{lightIntensity} \rightarrow \text{kết quả vào Vector PRF}$

#### **Bước 6: Memory Access**

- Load Buffer xử lý texture fetch
- Dữ liệu về từ cache  $\rightarrow$  ghi vào Vector PRF
- L1 DTLB (384 entries) dịch địa chỉ ảo  $\rightarrow$  vật lý

#### **Bước 7: Retire (Hoàn Thành)**

- $\mu\text{OPs}$  hoàn thành theo đúng thứ tự (in-order retirement)
- ROQ (600 entries) đảm bảo thứ tự đúng
- Kết quả từ Vector PRF  $\rightarrow$  ghi vào architectural register
- PRRT giải phóng thanh ghi vật lý không dùng nữa

- 12 pops/cycle → có thể retire 12 lệnh mỗi chu kỳ

#### Tổng thời gian:

- Trường hợp tốt (cache hit): ~15-20 cycles
- Trường hợp xấu (cache miss): ~50-100 cycles

**Out-of-order magic:** Trong khi chờ texture của pixel này, ARC Core xử lý hàng trăm pixels khác song song!

## 9. BANDWIDTH - BẢNG THÔNG DỮ LIỆU

### 9.1. Ý Nghĩa Của 32 B/Cycle

Quan sát các con số **32 B/Cycle** trong diagram:

- **32 B/Cycle = 32 bytes mỗi chu kỳ clock**
- Nếu ARC Core chạy 3 GHz (3 tỷ chu kỳ/giây):
  - Băng thông =  $32 \times 3 \times 10^9 = \mathbf{96\ GB/s}$  (cho mỗi đường)

### 9.2. Nhiều Đường Song Song

#### Load Buffer:

- 3 đường  $\times$  32 B/cycle = **96 B/cycle đọc**
- Tại 3 GHz: **288 GB/s đọc**

#### Store Buffer:

- 3 đường  $\times$  32 B/cycle = **96 B/cycle ghi**
- Tại 3 GHz: **288 GB/s ghi**

**Tổng cộng:** ~288 GB/s đọc + 288 GB/s ghi = **~576 GB/s tổng băng thông** (lý thuyết)

### 9.3. Ứng Dụng Trong Gaming

#### Ví dụ thực tế:

- Texture 4K (3840 $\times$ 2160): ~8MB mỗi texture (uncompressed)
- Băng thông đọc 288 GB/s → load được **36 texture 4K mỗi giây**



- Hoặc: **Render 1000+ frame 4K mỗi giây** (nếu không có bottleneck khác)

**Trong thực tế:**

- Gaming @4K 60fps: mỗi frame ~8MB data movement
- $60 \text{ fps} \times 8 \text{ MB} = 480 \text{ MB/s}$  (chỉ 0.16% băng thông!)
- **Kết luận:** ARC Core có băng thông dư thừa rất nhiều, đủ cho 8K gaming hoặc VR cao cấp

## 10. ỨNG DỤNG THỰC TẾ

### 10.1. Gaming

**Tại sao ARC Core tốt cho gaming:**

- **FP Schedulers lớn (208):** Xử lý nhiều shader instructions song song
- **Load Buffer lớn (160):** Load nhiều textures/vertices cùng lúc
- **L2 Cache 24MB:** Chứa nhiều game assets, giảm loading
- **Băng thông cao:** Xử lý 4K, 8K, VR không vấn đề

**Các tác vụ gaming cụ thể:**

- **Vertex Processing:** AGU + VALU xử lý hàng triệu vertices
- **Pixel Shading:** FP units tính màu, lighting, shadow cho mỗi pixel
- **Texture Sampling:** Memory system load texture nhanh
- **Physics:** ALU MUL/DIV tính collision, trajectory
- **AI:** Integer units xử lý pathfinding, decision trees

### 10.2. 3D Rendering & Content Creation

**Tối ưu cho:**

- **Blender, Maya:** Render 3D scenes phức tạp
- **Ray Tracing:** FP units tính toán tia sáng (khi kết hợp với SPE)
- **Video Editing:** Xử lý 4K/8K timeline
- **Encoding:** Integer + Vector units encode H.265/AV1

### 10.3. Kết Hợp Với SPE

## ARC Core + SPE:

- **ARC Core:** Xử lý general compute, shading, game logic
- **SPE:** Xử lý ray tracing, AI upscaling (DLSS-like), denoising, Physic

## Workflow:

- ARC Core chạy vertex shader, rasterization
- SPE chạy ray tracing cho reflection/shadow
- ARC Core composite kết quả cuối cùng

## Ví dụ: Game với ray-traced reflections:

1. ARC Core: Render base geometry và lighting
2. SPE: Trace rays cho reflections
3. ARC Core: Combine reflections với base image
4. Output: Final frame với realistic reflections

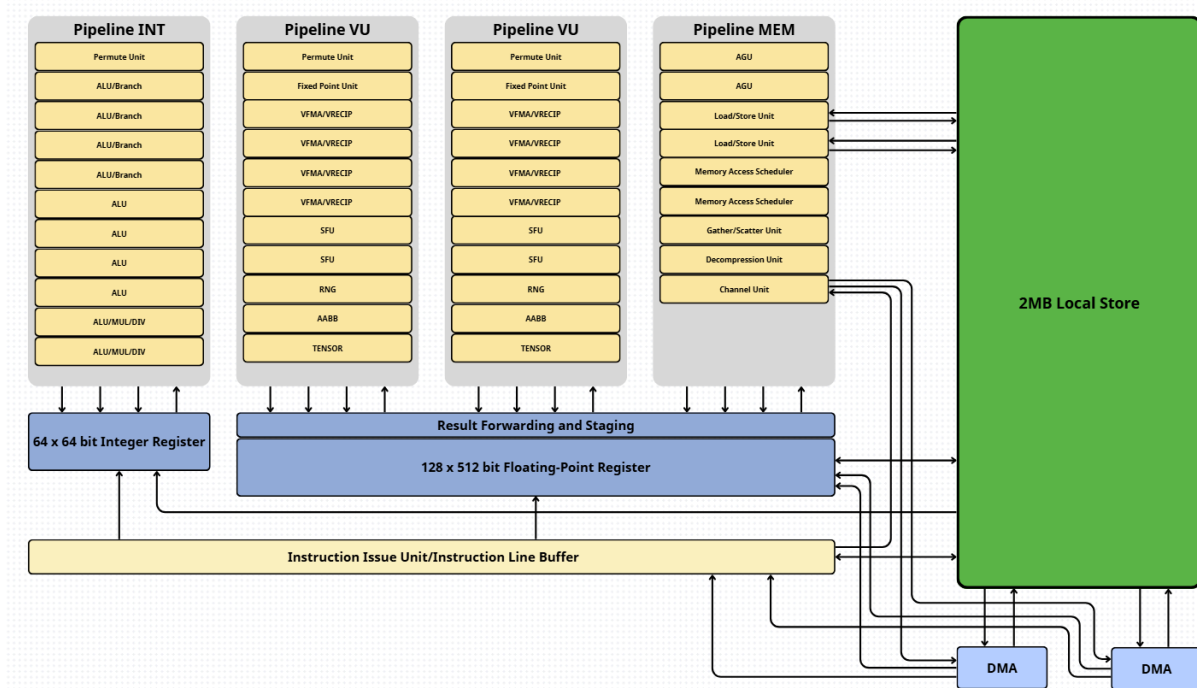
# KẾT LUẬN

Vi kiến trúc **ARC Core** là một thiết kế chuyên biệt và cân bằng, tối ưu hóa cho gaming và rendering workloads. Với các thành phần như:

- **12 bộ giải mã** xử lý nhiều lệnh song song
- **ROQ 600 entries** cho phép out-of-order execution mạnh mẽ
- **FP Schedulers lớn (208 entries)** tối ưu cho đồ họa
- **Memory system mạnh mẽ** (Load Buffer 160 + Store Buffer 110)
- **Cache lớn** (192KB L1-I + 128KB L1-D + 24MB L2)
- **Băng thông khổng lồ** (~576 GB/s total)

ARC Core đại diện cho sự cân bằng giữa hiệu suất integer và floating-point, phù hợp hoàn hảo cho gaming hiện đại. Khi kết hợp với **SPE (Synergistic Processing Elements)**, tạo thành một hệ thống hoàn chỉnh cho gaming thế hệ mới với ray tracing và AI upscaling. *Mỗi lần bạn chơi game, ngắm nhìn đồ họa tuyệt đẹp với ánh sáng chân thực, hàng tỷ transistor trong ARC Core đang làm việc theo đúng sơ đồ này - một kỳ tích kỹ thuật giúp biến thế giới ảo trở nên sống động như thật.*

# VI. SPE



## GIẢI THÍCH VI KIẾN TRÚC SPE (SYNERGISTIC PROCESSING ELEMENT)

### MỤC LỤC

1. Giới Thiệu
2. Tổng Quan Quy Trình Xử Lý
3. Các Thành Phần Chi Tiết
4. Hệ Thống Bộ Nhớ
5. Register Files - Bộ Thanh Ghi
6. Kiến Trúc In-Order Execution
7. Con Số Ấn Tượng
8. Quy Trình Xử Lý Một Lệnh
9. Bandwidth - Băng Thông Dữ Liệu
10. Ứng Dụng Thực Tế

# 1. GIỚI THIỆU

**SPE (Synergistic Processing Element)** là một lõi xử lý chuyên biệt được thiết kế theo triết lý hoàn toàn khác biệt so với CPU truyền thống. SPE là thành phần cốt lõi của **Cell**

**Broadband Engine** - bộ xử lý nổi tiếng được sử dụng trong PlayStation 3 và các hệ thống supercomputer. **Triết lý thiết kế:**

- Tối ưu hóa cho **throughput** (lượng công việc/giây) hơn là **latency** (thời gian xử lý 1 việc)
- Thiết kế **in-order execution** đơn giản, tiết kiệm transistor
- Tập trung vào **xử lý vector/SIMD** mạnh mẽ
- **Local Store 2MB** thay vì cache phân cấp phức tạp
- **DMA engine** để di chuyển dữ liệu tự động

**Mô hình hoạt động:**

- Một chip Cell có **8 SPEs** làm việc song song
- Mỗi SPE như một "công nhân chuyên biệt" xử lý một tác vụ cụ thể
- PPE (Power Processing Element) là "người quản lý" phân công việc cho các SPE

**Ứng dụng chính:**

- Gaming (PlayStation 3)
- Video encoding/decoding
- Scientific computing
- Signal processing
- Physics simulation

## 2. TỔNG QUAN QUY TRÌNH XỬ LÝ

SPE có **4 pipeline song song**, mỗi pipeline chuyên biệt cho một loại công việc:

### 2.1. Bốn Pipeline Chính

1. **Pipeline INT (Integer Pipeline)**
  - Xử lý số nguyên
  - Phép tính logic, branch, address
1. **Pipeline VU (Vector Unit Pipeline) - 2 pipelines**

- Xử lý vector và floating-point
- Phép tính SIMD mạnh mẽ
- Đây là "linh hồn" của SPE

#### 1. Pipeline MEM (Memory Pipeline)

- Xử lý truy cập bộ nhớ
- Load/Store, DMA control

## 2.2. Đặc Điểm Kiến Trúc

### In-Order Execution:

- Khác với ARC Core (out-of-order), SPE thực thi lệnh **đúng theo thứ tự**
- Đơn giản hơn, ít transistor hơn, tiết kiệm năng lượng
- Lập trình viên phải tối ưu code thủ công

### SIMD-centric:

- Mọi thứ được thiết kế xoay quanh xử lý vector
- Register files  $128 \times 512$ -bit (mỗi register chứa 1 vector lớn)
- Vector units rất mạnh với VFMA, SFU, TENSOR

## 3. CÁC THÀNH PHẦN CHI TIẾT

### 3.1. Instruction Issue Unit / Instruction Line Buffer

**Chức năng:** Quản lý và phát hành lệnh cho các pipeline. **Đặc điểm:**

- Đơn giản hơn nhiều so với ARC Core (không có ROQ, scheduler phức tạp)
- Phát hành lệnh theo thứ tự (in-order issue)
- Giao tiếp với cả 4 pipelines

### Vai trò trong kiến trúc in-order:

- Đọc lệnh từ Local Store (2MB)
- Phát hành lệnh cho pipeline phù hợp
- Đảm bảo không xung đột tài nguyên

**Ví dụ thực tế:** Giống như một băng chuyền sản xuất - sản phẩm (lệnh) được xử lý tuần tự, không có việc nhảy qua nhảy lại.

## 3.2. Pipeline INT - Integer Pipeline

Pipeline chuyên biệt cho phép tính số nguyên và logic.

### 3.2.1. Permute Unit

**Chức năng:** Sắp xếp lại, xáo trộn dữ liệu trong vector. **Ví dụ thực tế trong game:**

- Swizzling: Chuyển đổi RGBA → BGRA
- Shuffle: Sắp xếp lại vertices trong mảng
- Byte reordering: Chuyển đổi endianness

**Tại sao quan trọng:**

- Dữ liệu game thường cần được sắp xếp lại (repack) trước khi xử lý
- Permute nhanh giúp tránh phải viết nhiều lệnh load/store chậm

### 3.2.2. ALU/Branch Units (x4)

**Số lượng:** 4 units **Chức năng:**

- Phép tính ALU cơ bản: ADD, SUB, AND, OR, XOR, shift
- Branch operations: Xử lý nhảy có điều kiện

**Băng thông:** 4 ALU operations/cycle **Ví dụ trong game:**

- Tính index mảng: ``address = base + index × stride``
- Logic điều khiển: ``if (health < 20) usePotion();``
- Bit masking: Kiểm tra flags

### 3.2.3. ALU Units (x4)

**Số lượng:** 4 units riêng biệt **Chức năng:** Phép tính ALU thuần túy, không có branch. **Tổng**

**ALU capacity:** 4 (ALU/Branch) + 4 (ALU) = 8 integer operations/cycle **Ví dụ trong game:**

- Loop counter: ``i++``
- Array indexing: ``particles[i]``
- Bit operations: Packing/unpacking data

### 3.2.4. ALU/MUL/DIV Units (x2)

**Số lượng:** 2 units **Chức năng:**

- Integer multiply
- Integer divide
- ALU operations

**Lưu ý:** Nhân và chia chậm hơn nhiều so với cộng/trừ (10-30 cycles). **Ví dụ trong game:**

- Scaling: ``damage = baseDamage × multiplier``
- Percentage: ``health% = currentHealth × 100 / maxHealth``

### 3.3. Pipeline VU - Vector Unit Pipeline (x2)

Đây là **trái tim của SPE**. SPE có **2 Vector Unit pipelines** hoàn toàn giống nhau, cho phép xử lý 2 luồng vector song song.

#### 3.3.1. Permute Unit (per VU pipeline)

**Chức năng:** Giống như Permute trong INT pipeline, nhưng cho vector data. **Ví dụ trong đồ họa:**

- Chuyển đổi layout vector:  $(x,y,z,w) \rightarrow (x,x,x,x)$
- Swizzling trong shader: ``color.rgba \rightarrow color.bgra``
- Transposing: Ma trận  $4 \times 4 \rightarrow 4 \times 4$  transpose

#### 3.3.2. Fixed Point Unit (per VU pipeline)

**Chức năng:** Xử lý số thực dấu phẩy cố định (fixed-point). **Tại sao cần fixed-point?**

- Nhanh hơn floating-point
- Đủ chính xác cho nhiều tác vụ game (âm thanh, vị trí 2D)
- Tiết kiệm năng lượng

**Ví dụ trong game:**

- Audio processing: Volume, mixing
- 2D sprite positioning
- UI calculations

#### 3.3.3. VFMA/VRECIP Units (x4 per VU, tổng x8)

**VFMA:** Vector Fused Multiply-Add **VRECIP:** Vector Reciprocal (tính  $1/x$ ) **Số lượng:** 4 units × 2 VU pipelines = **8 VFMA units tổng cộng** **Chức năng VFMA:**

- Thực hiện phép tính: `result = a × b + c` trong **1 chu kỳ**
- Cực kỳ quan trọng cho đồ họa 3D

**Ví dụ trong 3D rendering: Phép tính vector dot product:**

$$\text{dot}(A, B) = A.x \times B.x + A.y \times B.y + A.z \times B.z + A.w \times B.w$$

- Mỗi VFMA làm một phần: `A.x × B.x + C`
- 4 VFMA song song → 1 dot product trong ~1-2 cycles

**Phép tính matrix × vector:**

$$\text{result} = \text{Matrix} \times \text{Vector}$$

$$\text{result}.x = \text{row1}.x \times v.x + \text{row1}.y \times v.y + \text{row1}.z \times v.z + \text{row1}.w \times v.w$$

- 8 VFMA units → có thể tính 2 rows cùng lúc
- Ma trận 4×4 × vector: ~2-3 cycles

**VRECIP:**

- Tính nghịch đảo nhanh: `1/x`
- Dùng cho normalize vector, perspective divide

**3.3.4. SFU - Special Function Unit (x2 per VU, tổng x4)**

**Số lượng:** 2 units × 2 VU = 4 SFU units **Chức năng:**

- Hàm đặc biệt: `sqrt`, `rsqrt` ( $1/\sqrt{x}$ ), `log`, `exp`, `sin`, `cos`
- Rất quan trọng cho đồ họa

**Ví dụ trong 3D graphics: Normalize vector:**

$$\text{length} = \sqrt{x^2 + y^2 + z^2}$$

$$\text{normalized} = \text{vector} / \text{length}$$

Hoặc nhanh hơn:



```
invLength = rsqrt(x2 + y2 + z2) ← SFU làm việc này  
normalized = vector × invLength
```

### Lighting calculations:

```
specular = pow(dot(reflect, view), shininess)  
           = exp(shininess × log(dot)) ← SFU làm exp, log
```

### Physics:

- Trajectory:  $y = v \times \sin(\text{angle}) \times t - 0.5 \times g \times t^2$  ← SFU tính sin

### 3.3.5. RNG - Random Number Generator (x1 per VU, tổng x2)

**Số lượng:** 1 per VU pipeline = **2 RNG units** **Chức năng:** Tạo số ngẫu nhiên phần cứng. **Ví dụ trong game:**

- **Particle systems:** Random vị trí, vận tốc, màu sắc của hạt
- **Procedural generation:** Địa hình, cây cối ngẫu nhiên
- **AI:** Random decision making
- **Effects:** Random noise cho shader, water ripples

### Lợi ích:

- Nhanh hơn nhiều so với tính ngẫu nhiên bằng software
- Giải phóng ALU cho công việc khác

### 3.3.6. AABB - Axis-Aligned Bounding Box (x1 per VU, tổng x2)

**Số lượng:** 1 per VU pipeline = **2 AABB units** **Chức năng:** Tính toán và kiểm tra va chạm AABB nhanh. **AABB là gì?**

- Hình hộp bao quanh object, song song với trục tọa độ
- Dạng: (min\_x, min\_y, min\_z) và (max\_x, max\_y, max\_z)

**Ví dụ trong game: Collision detection:**

Kiểm tra 2 AABB có va chạm không:

```
if (box1.max_x > box2.min_x && box1.min_x < box2.max_x &&  
    box1.max_y > box2.min_y && box1.min_y < box2.max_y &&  
    box1.max_z > box2.min_z && box1.min_z < box2.max_z)  
    → VA CHẠM!
```

### **Frustum culling:**

- Kiểm tra object có trong camera view không
- AABB unit kiểm tra rất nhanh → chỉ render object nhìn thấy

### **Spatial partitioning:**

- Octree, BVH (Bounding Volume Hierarchy)
- AABB unit tăng tốc việc duyệt cây

### **Tại sao có phần cứng riêng?**

- Collision detection là bottleneck lớn trong game
- AABB unit có thể kiểm tra hàng ngàn boxes/frame

### **3.3.7. TENSOR Unit (x1 per VU, tổng x2)**

**Số lượng:** 1 per VU pipeline = **2 TENSOR units** **Chức năng:** Xử lý phép tính tensor/matrix chuyên biệt. **Tensor operations:**

- Matrix multiply (ma trận × ma trận)
- Tensor contraction
- Convolution (cho AI/ML)

### **Ví dụ trong game và AI: 3D Transformations:**

$\text{FinalMatrix} = \text{Projection} \times \text{View} \times \text{Model}$

TENSOR unit làm phép nhân ma trận 4×4 này rất nhanh. **Skinning/Animation:**

$\text{vertex\_final} = \text{bone1\_matrix} \times \text{weight1} + \text{bone2\_matrix} \times \text{weight2} + \dots$

TENSOR unit xử lý nhiều bone transformations song song. **AI/Neural Networks (nếu áp dụng):**

- Matrix operations trong neural network
- Convolution cho image processing

**Physics:**

- Inertia tensor calculations
- Rigid body dynamics

### 3.4. Pipeline MEM - Memory Pipeline

Pipeline chuyên biệt cho truy cập bộ nhớ và I/O.

#### 3.4.1. AGU - Address Generation Unit (x2)

**Số lượng:** 2 units **Chức năng:** Tính địa chỉ bộ nhớ cho load/store operations. **Phép tính địa chỉ:**

```
address = base + index × stride + offset
```

**Ví dụ:**

Array access: vertices[i].position

→ address = vertices\_base + i × sizeof(vertex) + offset\_of\_position

**2 AGU** cho phép tính 2 địa chỉ song song → load/store 2 data streams cùng lúc.

#### 3.4.2. Load/Store Units (x2)

**Số lượng:** 2 units **Chức năng:**

- **Load:** Đọc dữ liệu từ Local Store (2MB) vào registers
- **Store:** Ghi dữ liệu từ registers vào Local Store

**Băng thông:** 2 load/store operations mỗi cycle **Ví dụ trong game:**

- Load: Đọc vertex positions từ buffer

- Store: Ghi transformed vertices vào output buffer

### 3.4.3. Memory Access Scheduler (x2)

**Số lượng:** 2 schedulers **Chức năng:** Lập lịch và tối ưu hóa truy cập bộ nhớ. **Vai trò:**

- Sắp xếp lại thứ tự load/store để tối ưu băng thông
- Tránh bank conflicts trong Local Store
- Quản lý hàng đợi memory requests

**Tại sao cần 2 schedulers?**

- Mỗi scheduler quản lý 1 Load/Store Unit
- Tăng throughput, giảm stall

### 3.4.4. Gather/Scatter Unit

**Chức năng:**

- **Gather:** Đọc nhiều phần tử rời rạc từ bộ nhớ, gom thành 1 vector
- **Scatter:** Ghi các phần tử của 1 vector vào nhiều vị trí rời rạc

**Ví dụ Gather:**

```
// Đọc 4 vertex positions không liên tiếp
indices = [5, 12, 3, 18]
positions = gather(vertex_buffer, indices)
→ positions = [vertex[5], vertex[12], vertex[3], vertex[18]]
```

**Ví dụ Scatter:**

```
// Ghi kết quả vào 4 vị trí rời rạc
results = [r0, r1, r2, r3]
scatter(output_buffer, indices, results)
```

**Tại sao quan trọng?**

- Xử lý indexed data (như indexed triangle lists)
- Particle systems (mỗi particle ở vị trí khác nhau)

- Sparse matrix operations

### 3.4.5. Decompression Unit

**Chức năng:** Giải nén dữ liệu phần cứng. **Ví dụ trong game:**

- **Texture compression:** Giải nén DXT/BCn textures
- **Geometry compression:** Giải nén compressed mesh data
- **Audio:** Giải nén audio streams (MP3, AAC)

**Lợi ích:**

- Tiết kiệm băng thông từ main memory
- Tiết kiệm Local Store (2MB có hạn)
- Giải nén nhanh bằng hardware, không tốn ALU cycles

### 3.4.6. Channel Unit

**Chức năng:** Giao tiếp với bên ngoài SPE (main memory, các SPE khác, PPE). **Các kênh (channels):**

- **Inbound mailbox:** Nhận commands từ PPE hoặc SPE khác
- **Outbound mailbox:** Gửi results ra ngoài
- **Signal notification:** Synchronization và events
- **DMA control:** Điều khiển DMA transfers

**Ví dụ workflow:**

1. PPE gửi command qua channel: "Xử lý 10,000 particles"
2. SPE nhận command
3. DMA tự động load particle data vào Local Store
4. SPE xử lý
5. DMA tự động ghi kết quả ra main memory
6. SPE gửi signal "Done" qua channel

## 3.5. Result Forwarding and Staging

**Chức năng:** Chuyển tiếp kết quả giữa các pipeline và registers. **Vai trò trong kiến trúc in-order:**

- Khi INT pipeline tính xong address, forward ngay cho MEM pipeline

- Khi VU pipeline tính xong vector, forward cho instruction tiếp theo
- Giảm độ trễ giữa các lệnh phụ thuộc

**Ví dụ:**

1.  $addr = base + offset$  (INT pipeline)
2.  $data = load(addr)$  (MEM pipeline, cần  $addr$  từ lệnh 1)
3.  $result = data \times 2$  (VU pipeline, cần  $data$  từ lệnh 2)

Result forwarding giúp lệnh 2 nhận `addr` sớm nhất có thể, không phải đợi lệnh 1 hoàn toàn xong.

## 4. HỆ THỐNG BỘ NHỚ

Hệ thống bộ nhớ của SPE **hoàn toàn khác biệt** so với CPU truyền thống.

### 4.1. Local Store - 2MB

**Đặc điểm:**

- **Kích thước:** 2MB
- **Loại:** SRAM, không phải cache
- **Software-managed:** Lập trình viên phải quản lý thủ công
- **Độ trễ:** Cực thấp, ~6 cycles
- **Băng thông:** Rất cao, ~128 GB/s (theoretical)

**Khác biệt với Cache:**

|            | Cache (ARC Core)    | Local Store (SPE)   |
|------------|---------------------|---------------------|
| Quản lý    | Hardware tự động    | Software thủ công   |
| Dự đoán    | Hardware prefetch   | Programmer prefetch |
| Kích thước | Phân cấp (L1/L2/L3) | Flat 2MB            |

|                   |                                  |                           |
|-------------------|----------------------------------|---------------------------|
| Độ trễ            | Không chắc chắn                  | Chắc chắn, predictable    |
|                   | ARC Core                         | SPE                       |
| Integer Registers | ~576 × 64-bit                    | 64 × 64-bit               |
| FP Registers      | ~512 × ?                         | 128 × 512-bit             |
| Total Capacity    | ?                                | 8 KB (128×64 bytes)       |
| Focus             | Out-of-order, nhiều register nhỏ | In-order, ít register lớn |
| Thành phần        | Con số                           | Ý nghĩa                   |
| Pipelines         | 4                                | INT + 2×VU + MEM          |
| INT ALUs          | 8                                | 4 ALU/Branch + 4 ALU      |
| INT MUL/DIV       | 2                                | Nhân/chia integer         |
| VU Pipelines      | 2                                | Hai vector units độc lập  |
| VFMA Units        | 8 total                          | 4 per VU × 2 VU           |
| SFU Units         | 4 total                          | 2 per VU × 2 VU           |
| RNG Units         | 2                                | 1 per VU                  |
| AABB Units        | 2                                | 1 per VU                  |
| TENSOR Units      | 2                                | 1 per VU                  |
| AGU               | 2                                | Tính địa chỉ memory       |
| Load/Store Units  | 2                                | 2 memory ops/cycle        |

|                   |               |                  |
|-------------------|---------------|------------------|
| Memory Schedulers | 2             | 1 per Load/Store |
| DMA Engines       | 2             | Double buffering |
| Integer Registers | 64 × 64-bit   | 512 bytes total  |
| FP Registers      | 128 × 512-bit | 8 KB total       |
| Local Store       | 2 MB          | Software-managed |

#### Lợi ích của Local Store:

- **Predictable performance:** Không có cache miss bất ngờ
- **Hiệu quả năng lượng:** Đơn giản hơn cache
- **Control:** Lập trình viên kiểm soát hoàn toàn

#### Nhược điểm:

- **Programming complexity:** Lập trình viên phải quản lý dữ liệu
- **Kích thước nhỏ:** Chỉ 2MB (vs 24MB L2 của ARC Core)

## 4.2. DMA - Direct Memory Access (x2)

**Số lượng:** 2 DMA engines **Chức năng:** Di chuyển dữ liệu tự động giữa Local Store và main memory. **DMA Model - Double Buffering:** Ý tưởng:

- Trong khi SPE xử lý dữ liệu batch N, DMA load batch N+1 đồng thời
- Khi SPE xong batch N, batch N+1 đã sẵn sàng
- **Zero wait time!**

#### Ví dụ thực tế - Video Encoding:

```
Time: |---0---|---1---|---2---|---3---|
DMA1: | Load A | Load C | Load E | Load G |
DMA2: |         | Load B | Load D | Load F |
SPE:  | Wait   | Proc A | Proc B | Proc C |
```



### Step by step:

1. **T=0:** DMA1 load frame A vào buffer1 trong Local Store
2. **T=1:**
  - DMA1 load frame C vào buffer1
  - SPE xử lý frame A từ buffer1
  - Đồng thời, DMA2 có thể load frame B vào buffer2

### T=2:

- SPE xử lý frame B từ buffer2
- DMA1 load frame D, DMA2 load frame C

Cứ thế tiếp tục...

**Kết quả:** SPE luôn bận rộn, không bao giờ phải đợi data!

## 4.3. Không Có Cache Hierarchy

### SPE không có:

- L1 Data Cache
- L2 Cache
- L3 Cache
- Hardware prefetcher
- Cache coherency protocol

### Tại sao?

- Giảm phức tạp → Tiết kiệm transistor và năng lượng
- Transistor được dùng cho execution units mạnh hơn
- Predictable performance cho real-time workloads

### Trade-off:

- Lập trình khó hơn
- Cần hiểu rõ memory access patterns
- Nhưng performance cao hơn khi optimize tốt

## 5. REGISTER FILES - BỘ THANH GHI

## 5.1. Integer Register File

**Kích thước:** 64 × 64-bit registers **Chức năng:**

- Lưu trữ integers, addresses, control data
- Phục vụ INT pipeline

**64-bit per register:**

- Có thể chứa:
  - 1 số nguyên 64-bit
  - 2 số nguyên 32-bit
  - 4 số nguyên 16-bit
  - 8 số nguyên 8-bit

**Ví dụ sử dụng:**

- Loop counters, array indices
- Memory addresses
- Control flags

## 5.2. Floating-Point Register File

**Kích thước:** 128 × 512-bit registers **Đây là đặc điểm nổi bật nhất của SPE! 512-bit per register = 64 bytes = 1 cache line** **Có thể chứa:**

- **16 × float32** (16 số thực 32-bit)
- **8 × double64** (8 số thực 64-bit)
- **4 × quad128** (4 vector XYZW)
- Hoặc bất kỳ combination nào

**Ví dụ 1 - Xử lý 4 vertices cùng lúc:**

1 register 512-bit chứa:

[vertex0.xyzw | vertex1.xyzw | vertex2.xyzw | vertex3.xyzw]  
= 4×4 floats = 16 floats = 512 bits

**Ví dụ 2 - Xử lý 16 pixels cùng lúc:**

1 register chứa 16 pixel colors (mỗi pixel 1 float32 intensity)  
→ VFMA có thể xử lý 16 pixels trong 1 operation!

Ý nghĩa:

- Throughput cực cao cho vector/SIMD operations
- Giảm số lần load/store
- Tối ưu cho xử lý hàng loạt dữ liệu

### 5.3. So Sánh Với ARC Core

| | ARC Core | SPE | |---|---|---| | Integer Registers |  $\sim 576 \times 64\text{-bit}$  |  $64 \times 64\text{-bit}$  | | FP Registers |  $\sim 512 \times ?$  |  $128 \times 512\text{-bit}$  | | Total Capacity | ? | 8 KB ( $128 \times 64$  bytes) | | Focus | Out-of-order, nhiều register nhỏ | In-order, ít register lớn |

Triết lý khác biệt:

- **ARC Core:** Nhiều thanh ghi nhỏ để theo dõi nhiều lệnh đang bay (out-of-order)
- **SPE:** Ít thanh ghi nhưng rất lớn, chứa nhiều dữ liệu, xử lý batch (SIMD)

## 6. KIẾN TRÚC IN-ORDER EXECUTION

### 6.1. Khái Niệm

SPE sử dụng kiến trúc **thực thi theo thứ tự** (in-order execution) - ngược lại với ARC Core.

Ý nghĩa:

- Lệnh được thực hiện **đúng theo thứ tự** chương trình viết
- Không có reordering, speculation phức tạp
- Đơn giản, dễ dự đoán, tiết kiệm transistor

Ví dụ:

1.  $a = \text{load}(\text{addr1})$  → phải chờ load xong (chậm)
2.  $b = a + 5$  → phải đợi lệnh 1 xong
3.  $c = \text{load}(\text{addr2})$  → phải đợi lệnh 2 xong

Trong in-order, lệnh 3 phải đợi lệnh 2, dù lệnh 3 không phụ thuộc vào lệnh 1 hoặc 2.

## 6.2. Stall và Latency Hiding

**Vấn đề với in-order:**

- Khi gặp lệnh chậm (load, divide), pipeline bị **stall** (đứng)
- CPU phải đợi, lãng phí cycles

**Giải pháp của SPE - Dual Issue:**

- SPE có **2 VU pipelines** hoàn toàn độc lập
- Có thể issue 2 lệnh vector mỗi cycle (nếu không phụ thuộc)
- Ẩn latency bằng cách xử lý nhiều data streams song song

**Ví dụ - Xử lý 2 luồng particles:**

Pipeline VU0:

1.  $\text{pos0} = \text{load}(\text{particles}[i])$
2.  $\text{vel0} = \text{load}(\text{velocities}[i])$
3.  $\text{pos0} = \text{pos0} + \text{vel0} \times dt$

Pipeline VU1 (đồng thời):

1.  $\text{pos1} = \text{load}(\text{particles}[i+1])$
2.  $\text{vel1} = \text{load}(\text{velocities}[i+1])$
3.  $\text{pos1} = \text{pos1} + \text{vel1} \times dt$

Khi VU0 chờ load, VU1 vẫn làm việc → giảm waste!

## 6.3. Software Optimization Requirements

**Lập trình viên phải:**

1. **Unroll loops:** Mở rộng vòng lặp để tận dụng dual issue

2. **Data prefetching:** Dùng DMA load dữ liệu trước
3. **Software pipelining:** Xử lý nhiều batches overlap
4. **SIMD vectorization:** Tận dụng 512-bit registers

**Ví dụ - Loop Unrolling: Code gốc (chậm):**

```
for (int i = 0; i < 1000; i++) {  
    result[i] = a[i] + b[i];  
}
```

**Code optimized (nhanh):**

```
for (int i = 0; i < 1000; i += 16) {  
    // Load 16 elements vào 1 register 512-bit  
    vec512 va = load_vec512(&a[i]);  
    vec512 vb = load_vec512(&b[i]);  
    vec512 vr = vadd512(va, vb); // Cộng 16 số cùng lúc!  
    store_vec512(&result[i], vr);  
}
```

**Tăng tốc: ~10-15x**

## 7. CON SỐ ẢN TƯỢNG

Hãy cùng xem các con số quan trọng của SPE:

| Thành phần | Con số | Ý nghĩa | |-----|-----|-----| | Pipelines | 4 | INT + 2×VU + MEM | | INT ALUs | 8 | 4 ALU/Branch + 4 ALU | | INT MUL/DIV | 2 | Nhân/chia integer | | VU Pipelines | 2 | Hai vector units độc lập | | VFMA Units | 8 total | 4 per VU × 2 VU | | SFU Units | 4 total | 2 per VU × 2 VU | | RNG Units | 2 | 1 per VU | | AABB Units | 2 | 1 per VU | | TENSOR Units | 2 | 1 per VU | | AGU | 2 | Tính địa chỉ memory | | Load/Store Units | 2 | 2 memory ops/cycle | | Memory Schedulers | 2 | 1 per Load/Store | | DMA Engines | 2 | Double buffering | | Integer Registers | 64 × 64-bit | 512 bytes total | | FP Registers | 128 × 512-bit | 8 KB total | | Local Store | 2 MB | Software-managed |

**Điểm nổi bật:**

- **8 VFMA units:** Sức mạnh tính toán vector khủng khiếp
- **512-bit registers:** SIMD width rất lớn
- **2MB Local Store:** Nhanh và predictable
- **2 DMA engines:** Zero-wait data loading

## 7.1. Tính Toán Throughput Lý Thuyết

Giả sử SPE chạy 3.2 GHz (như trong PS3): **FLOPS** (Floating-Point Operations Per Second):

- Mỗi VFMA = 2 ops (1 multiply + 1 add)
- 8 VFMA units  $\times$  2 ops  $\times$  3.2 GHz = **51.2 GFLOPS** (single precision)
- Nếu tận dụng 512-bit register (16 floats):
  - 8 VFMA  $\times$  2 ops  $\times$  16 floats  $\times$  3.2 GHz = **819 GFLOPS** (peak)

Cả chip Cell (8 SPEs):

- 8 SPEs  $\times$  819 GFLOPS = **6.5 TFLOPS** (peak)

Trong thực tế:

- Sustained performance: ~200-300 GFLOPS per SPE
- Cả chip: ~2 TFLOPS sustained

## 8. QUY TRÌNH XỬ LÝ MỘT LỆNH

Hãy theo dõi một phép tính vector đơn giản:

**Lệnh:** ``result = a  $\times$  b + c`` (vector 16 floats, VFMA)

### Bước 1: Data Preparation (DMA)

Trước khi lệnh chạy:

- DMA engine 1 load array ``a`` từ main memory  $\rightarrow$  Local Store
- DMA engine 2 load array ``b`` và ``c``  $\rightarrow$  Local Store
- Đồng thời, SPE có thể xử lý batch trước đó (double buffering)

**Thời gian:** DMA chạy song song với compute, ~0 wait

## Bước 2: Load Data

### Lệnh load:

```
va = load(&a[i]) // Load 16 floats (512 bits) từ Local Store
vb = load(&b[i])
vc = load(&c[i])
```

### Memory Pipeline xử lý:

- AGU tính địa chỉ:  $\text{'\&a[i] = base\_a + i \times 64'}$  (64 bytes = 512 bits)
- Load/Store Unit đọc 512 bits từ Local Store
- Dữ liệu vào FP Register File

**Thời gian:** ~6 cycles per load (3 loads = ~18 cycles) **Lưu ý:** 2 Load/Store units có thể làm 2 loads song song!

## Bước 3: Execute VFMA

### Lệnh VFMA:

```
vr = vfma(va, vb, vc) // result =  $a \times b + c$ 
```

### VU Pipeline xử lý:

- VFMA unit nhận 3 operands: va, vb, vc từ FP Register File
- Thực hiện 16 phép tính song song:
  - $\text{result}[0] = a[0] \times b[0] + c[0]$
  - $\text{result}[1] = a[1] \times b[1] + c[1]$
  - ...
  - $\text{result}[15] = a[15] \times b[15] + c[15]$
- Ghi kết quả vào FP Register File

**Thời gian:** ~6 cycles cho VFMA (pipelined) **Đặc biệt:**

- Vì có 8 VFMA units, có thể làm nhiều VFMA khác nhau song song
- VU0 làm batch 0, VU1 làm batch 1 cùng lúc

## Bước 4: Store Result

Lệnh store:

```
store(&result[i], vr) // Ghi 512 bits vào Local Store
```

Memory Pipeline xử lý:

- AGU tính địa chỉ: `&result[i]`
- Store Unit ghi 512 bits từ register → Local Store

Thời gian: ~6 cycles

## Bước 5: DMA Write Back

Sau khi batch xong:

- DMA engine ghi kết quả từ Local Store → main memory
- Đồng thời, SPE đã bắt đầu xử lý batch tiếp theo (double buffering)

Thời gian: DMA chạy song song, ~0 overhead

## Tổng Thời Gian

Tổng cho 1 batch (16 floats):

- Load: ~18 cycles (3 loads, có thể overlap)
- VFMA: ~6 cycles
- Store: ~6 cycles
- **Total: ~30 cycles** (có thể tối ưu xuống ~15-20 cycles với loop unrolling)

Throughput:

- 16 operations trong 30 cycles = **0.53 ops/cycle per VFMA unit**
- 8 VFMA units × 0.53 = **~4.2 FLOPS/cycle**
- Tại 3.2 GHz: **~13 GFLOPS**

Nếu tối ưu tốt (loop unroll, dual issue):

- Có thể đạt ~100-150 GFLOPS sustained per SPE



## 9. BANDWIDTH - BẢNG THÔNG DỮ LIỆU

### 9.1. Local Store Bandwidth

Local Store → Register:

- $2 \text{ Load/Store units} \times 512 \text{ bits/cycle} = 1024 \text{ bits/cycle} = 128 \text{ bytes/cycle}$
- Tại 3.2 GHz:  $128 \times 3.2 = \sim 410 \text{ GB/s}$

Đây là băng thông khổng lồ!

### 9.2. DMA Bandwidth

Main Memory ↔ Local Store:

- DMA bandwidth:  $\sim 25 \text{ GB/s}$  per DMA engine
- 2 DMA engines:  $\sim 50 \text{ GB/s total}$  (bidirectional)

Cả chip Cell (8 SPEs):

- $8 \text{ SPEs} \times 50 \text{ GB/s} = \sim 400 \text{ GB/s}$  aggregate bandwidth

So sánh:

- DDR3-1600 (dual channel):  $\sim 25 \text{ GB/s}$
- SPE có băng thông DMA gần gấp đôi!

### 9.3. Bottle Neck Analysis

Không phải bottleneck:

- Local Store bandwidth (410 GB/s) - rất cao
- Compute (819 GFLOPS peak) - rất mạnh

Có thể là bottleneck:

- DMA bandwidth (50 GB/s) - nếu data không fit trong 2MB
- Main memory bandwidth - chia sẻ giữa 8 SPEs + PPE

Giải pháp:

- Tối ưu data reuse trong Local Store

- Dừng compression (Decompression Unit)
- Careful data layout để minimize transfer

## 10. ỨNG DỤNG THỰC TẾ

### 10.1. PlayStation 3 Gaming

SPE đảm nhận:

1. **Physics Simulation:**
  - Cloth, hair, fluid dynamics
  - Rigid body collision (AABB units!)
  - Soft body deformation
1. **Particle Systems:**
  - Hàng chục ngàn particles
  - Weather effects (rain, snow)
  - Explosions, smoke
1. **Audio Processing:**
  - 3D audio positioning
  - Reverb, effects
  - Voice synthesis
1. **AI:**
  - Pathfinding
  - Decision trees
  - Behavior simulation
1. **Geometry Processing:**
  - Vertex skinning/animation
  - Mesh LOD (Level of Detail)
  - Procedural generation
1. **Post-Processing:**
  - Bloom, motion blur
  - Depth of field
  - Color grading

**Ví dụ game:** Uncharted, The Last of Us, God of War III

### 10.2. Video Encoding/Decoding

### Tại sao SPE tốt cho video:

- SIMD width lớn (512-bit) → xử lý nhiều pixels
- VFMA cho DCT/IDCT transforms
- Decompression Unit cho entropy decoding
- Memory bandwidth cao

### H.264 Encoding trên Cell:

- 1 SPE: ~30 fps 1080p encoding
- 6 SPEs (2 dành cho OS): ~200 fps 1080p
- Nhanh hơn CPU thường gấp 5-10 lần (năm 2006)

### Workflow:

1. PPE phân chia frames thành macroblocks
2. Mỗi SPE nhận một số macroblocks
3. SPE thực hiện:
  - Motion estimation (VFMA intensive)
  - DCT transform (TENSOR unit)
  - Quantization
  - Entropy encoding

DMA ghi compressed bitstream ra memory

PPE thu thập và mux thành final stream

## 10.3. Scientific Computing

### Cell được dùng trong supercomputers:

- **Roadrunner (Los Alamos):** Siêu máy tính petaflop đầu tiên (2008)
  - 12,960 Cell processors
  - 1.026 petaflops (1000 teraflops!)

### Ứng dụng khoa học:

1. **Molecular Dynamics:**
  - Mô phỏng protein folding
  - N-body simulations
  - VFMA xử lý force calculations

1. **Signal Processing:**

- Radar signal processing
- Seismic data analysis
- FFT operations (SFU units)

1. **Climate Modeling:**

- Fluid dynamics simulations
- Weather prediction
- TENSOR units cho matrix operations

1. **Cryptography:**

- Encryption/Decryption
- Hash calculations
- RNG units cho random keys

## 10.4. Image Processing

### Medical Imaging:

- CT/MRI reconstruction
- Image filtering, denoising
- 3D volume rendering

### Computer Vision:

- Object detection
- Feature extraction (AABB for bounding boxes)
- Optical flow

### Ví dụ - Edge Detection (Sobel Filter):

For each pixel (i,j):

Gx = convolution(image, sobel\_x\_kernel)

Gy = convolution(image, sobel\_y\_kernel)

magnitude =  $\sqrt{Gx^2 + Gy^2}$  ← SFU unit!

SPE có thể xử lý 16 pixels cùng lúc trong 1 register → rất nhanh!

## 10.5. Cell vs Modern Architectures

So sánh Cell/SPE (2006) với kiến trúc hiện đại (2026):

| Đặc điểm         | Cell/SPE (2006)      | Modern GPU (2026)    | Modern CPU (2026) |
|------------------|----------------------|----------------------|-------------------|
| Philosophy       | Explicit parallelism | Massive parallelism  | Latency-optimized |
| Memory Model     | Software-managed LS  | Cache + GDDR         | Cache hierarchy   |
| SIMD Width       | 512-bit (16 floats)  | 1024-2048 bit+       | 512-bit (AVX-512) |
| Cores            | 8 SPEs               | 1000s of cores       | 8-32 cores        |
| Programming      | Very difficult       | CUDA/OpenCL          | C/C++ easy        |
| Power Efficiency | Good                 | Excellent            | Good              |
| Flexibility      | Medium               | Low (graphics focus) | High              |

| Đặc điểm                 | SPE                    | ARC Core         | C/P Core           |
|--------------------------|------------------------|------------------|--------------------|
| <b>Execution Model</b>   | In-order               | Out-of-order     | Out-of-order       |
| <b>Target Workload</b>   | Throughput, batch      | Gaming, graphics | General purpose    |
| <b>Memory Model</b>      | Software (Local Store) | Hardware (Cache) | Hardware (Cache)   |
| <b>SIMD Width</b>        | 512-bit                | 256-512 bit?     | 128-256 bit        |
| <b>Specialized Units</b> | AABB, TENSOR, SFU      | Gaming-focused   | General            |
| <b>Register File</b>     | 128×512-bit FP         | ~512 registers   | ~288-576 registers |
| <b>ROQ/Buffer</b>        | None (in-order)        | 600 entries      | 480-620 entries    |

|                        |                   |                |                 |
|------------------------|-------------------|----------------|-----------------|
| <b>Philosophy</b>      | Explicit parallel | Latency hiding | Balanced        |
| <b>Programmability</b> | Very hard         | Medium         | Easy            |
| <b>Best For</b>        | Streaming, batch  | Gaming, RT     | Desktop, server |

### Điểm mạnh của SPE (vẫn relevant):

#### 1. Predictable Performance:

- Local Store → no cache miss surprise
- In-order → deterministic latency
- **Quan trọng cho real-time systems** (gaming, robotics)

#### 1. Power Efficiency:

- Ít transistor hơn out-of-order CPU
- Không có complex cache coherency
- Tốt cho embedded, mobile

#### 1. Explicit Control:

- Lập trình viên kiểm soát mọi thứ
- Có thể tối ưu cực kỳ hiệu quả (nếu biết cách)

### Điểm yếu của SPE:

#### 1. Programming Difficulty:

- Phải quản lý Local Store thủ công
- DMA programming phức tạp
- Loop unrolling, SIMD vectorization thủ công
- **Rất khó tìm lập trình viên giỏi**

#### 1. **Limited Memory:**

- 2MB Local Store quá nhỏ cho nhiều workloads
- Không phù hợp cho large datasets
- Main memory bandwidth hạn chế

#### 1. **Fixed Function Units:**

- Không flexible như GPU shaders hiện đại
- Khó thích nghi với workloads mới

### **Tại sao Cell "thất bại": Vấn đề lớn nhất: Programming difficulty**

- Developers phàn nàn PS3 khó lập trình hơn Xbox 360
- Phải viết code riêng cho SPE (assembly hoặc intrinsics)
- Debug rất khó
- Tools support nghèo nàn

### **CUDA/OpenCL đã thắng:**

- GPU programming dễ hơn nhiều
- Ecosystem lớn (libraries, tools)
- NVIDIA/AMD investment lớn
- Cell không có ecosystem

### **But the ideas live on:**

- Apple M-series: Neural Engine tương tự SPE philosophy
- Modern DSPs: Software-managed memory như Local Store
- RISC-V vector extensions: SIMD philosophy tương tự
- AI accelerators: Specialized units như TENSOR, AABB



# KẾT LUẬN

Vi kiến trúc **SPE (Synergistic Processing Element)** đại diện cho một triết lý thiết kế táo bạo và khác biệt trong lịch sử vi xử lý. Với các đặc điểm nổi bật:

## Điểm Mạnh Kiến Trúc

### 1. Throughput-Oriented Design:

- 8 VFMA units → 819 GFLOPS peak (single precision)
- 512-bit registers → xử lý 16 floats cùng lúc
- 2 VU pipelines → dual-issue cho vector operations
- **Kết quả:** Sức mạnh tính toán vector khủng khiếp

### 2. Software-Managed Memory (Local Store 2MB):

- Predictable performance (không có cache miss)
- Bandwidth cực cao (~410 GB/s)
- Latency thấp (~6 cycles)
- **Kết quả:** Real-time guarantee cho gaming

### 3. Specialized Units:

- **SFU (4 units):** sqrt, rsqrt, sin, cos cho graphics
- **AABB (2 units):** Hardware collision detection
- **TENSOR (2 units):** Matrix operations
- **RNG (2 units):** Random generation
- **Kết quả:** Tăng tốc các tác vụ quan trọng

### 4. DMA + Double Buffering:

- 2 DMA engines (~50 GB/s)
- Zero-wait data loading

- Compute và memory transfer overlap
- **Kết quả:** CPU luôn bận rộn, không idle

## 5. Power Efficiency:

- In-order execution đơn giản
- Không có complex cache logic
- Specialized units thay vì general-purpose
- **Kết quả:** Performance/Watt cao

## Kiến Trúc Cell - Big Picture

### Mô hình Cell Processor:

[PPE] ← Quản lý, OS, control

↓

[SPE1][SPE2][SPE3][SPE4][SPE5][SPE6][SPE7][SPE8] ← Workers

↑ ↓ ↑ ↓ ↑ ↓ ↑ ↓

[ Element Interconnect Bus (EIB) ]

↑ ↓

[Main Memory]

### Workflow tiêu biểu - Render 1 frame game:

#### 1. PPE (Power Processing Element - CPU thường):

- Chạy game engine, AI, control logic
- Phân chia công việc thành 8 tasks
- Gửi commands cho 8 SPEs qua mailbox

#### 1. SPE1: Physics simulation (100k particles)

- DMA load particle data → Local Store
- Tính position, velocity, collision (VFMA + AABB)

- DMA write results back
- 1. **SPE2:** Geometry processing (50k vertices)
  - DMA load vertices → Local Store
  - Vertex transformation (TENSOR units)
  - Skinning/animation (VFMA)
  - DMA write transformed vertices
- 1. **SPE3-4:** Texture processing
  - DMA load textures → Local Store
  - Filtering, mipmapping
  - Decompression Unit giải nén DXT
  - DMA write processed textures
- 1. **SPE5-6:** Lighting calculations
  - DMA load geometry + light data
  - Diffuse, specular lighting (VFMA, SFU)
  - Shadow calculations
  - DMA write lit geometry
- 1. **SPE7:** Audio processing
  - DMA load audio buffers
  - 3D positioning, reverb
  - Mixing (VFMA)
  - DMA write output to audio buffer
- 1. **SPE8:** Post-processing
  - DMA load framebuffer
  - Bloom, motion blur (VFMA)
  - Color grading
  - DMA write final frame
- 1. **GPU (RSX):** Rasterization
  - Nhận geometry đã xử lý từ SPEs
  - Rasterize triangles → pixels
  - Output final image

**Kết quả:** 60 fps smooth gameplay!

## **Con Số Tổng Kết**

### **Một SPE:**

- **Compute:** ~100-200 GFLOPS sustained (single precision)
- **Memory:** 2MB Local Store, ~410 GB/s bandwidth
- **DMA:** ~50 GB/s (2 engines)
- **Power:** ~5-10W per SPE

### **Cell Processor (8 SPEs + 1 PPE):**

- **Compute:** ~800-1600 GFLOPS sustained
- **Peak:** ~6.5 TFLOPS (unrealistic)
- **Memory:** 16 MB Local Store total (8×2MB)
- **DMA:** ~400 GB/s aggregate
- **Power:** ~60-80W total chip

### **So sánh năm 2006:**

- Intel Core 2 Duo: ~20 GFLOPS
- NVIDIA GeForce 7800: ~165 GFLOPS
- **Cell: ~1600 GFLOPS** → Gấp 10x CPU, gấp 5x GPU!

## **Bài Học Từ SPE**

### **1. Specialization Wins (Nhưng phải có ecosystem):**

- Specialized hardware rất mạnh (AABB, TENSOR, SFU)
- Nhưng cần tools, libraries, community support
- GPU thắng vì CUDA ecosystem, không chỉ vì hardware

## 2. Programmability Matters:

- SPE quá khó lập trình → ít developers dùng tốt
- Modern design focus vào ease of programming
- Trade-off: Dễ lập trình nhưng hardware phức tạp hơn

## 3. Memory Model Trade-offs:

- Software-managed memory (Local Store) → predictable, efficient
- Hardware-managed cache → easy programming, unpredictable
- Xu hướng: Hybrid (cache + scratchpad)

## 4. In-Order Can Be Fast:

- In-order + SIMD + DMA → very competitive
- Nếu workload phù hợp (streaming, batch processing)
- Modern: GPU là in-order, nhưng có 1000s cores

## 5. Explicit Parallelism:

- Cell buộc developer nghĩ parallel từ đầu
- Modern: Compiler auto-vectorize (nhưng không tốt bằng hand-tune)
- Xu hướng: DSL (Domain-Specific Languages) cho parallel

## Ảnh Hưởng Lịch Sử

### Cell/SPE đã truyền cảm hứng cho:

#### 1. Modern GPU Architecture:

- NVIDIA Tensor Cores ← TENSOR units
- RT Cores ← AABB units
- Software-managed caches (on-chip memory)

#### 1. Mobile SoCs:

- Apple Neural Engine: In-order, specialized, software-managed memory

- Qualcomm DSP: Local memory model tương tự
- ARM Mali GPU: Tile-based rendering với local buffers

#### 1. **AI Accelerators:**

- Google TPU: Matrix units như TENSOR
- Intel Nervana: Scratchpad memory như Local Store
- Cerebras: Explicit memory management

#### 1. **HPC/Supercomputing:**

- Vector processors (NEC SX-Aurora)
- RISC-V vector extensions
- Explicit data movement (Chapel, UPC++)

### **Tương Lai Của Kiến Trúc "SPE-Like"**

#### **Xu hướng có thể thấy SPE philosophy trở lại:**

##### 1. **Edge AI Devices:**

- Power-constrained → in-order + specialized units
- Predictable latency → software-managed memory
- Real-time guarantee → no cache surprises

##### 1. **Autonomous Vehicles:**

- Real-time guarantee tuyệt đối (safety-critical)
- SPE-like predictability rất quan trọng
- Specialized units cho computer vision

##### 1. **IoT / Embedded:**

- Tiny memory footprint → Local Store model phù hợp
- Low power → in-order efficient
- Domain-specific → specialized units

##### 1. **Quantum-Classical Hybrid:**

- Classical controller cần predictable timing
- Software-managed data movement
- Specialized units cho quantum gate control

## Lời Kết

**SPE (Synergistic Processing Element)** là một kiệt tác kỹ thuật - một thiết kế táo bạo vượt trước thời đại. Mặc dù Cell Processor không thành công về mặt thương mại như kỳ vọng, nhưng những ý tưởng cốt lõi của nó -

**specialization, explicit parallelism, software-managed memory, throughput-oriented design** - vẫn sống mãi và ảnh hưởng sâu sắc đến kiến trúc hiện đại. **Những gì chúng ta học được:**

- Hardware mạnh nhất không luôn thắng - programmability và ecosystem quan trọng hơn
- Specialization có thể mang lại hiệu suất khổng lồ, nhưng phải đi kèm abstraction tốt
- Trade-offs giữa control và convenience không có câu trả lời tuyệt đối
- Kiến trúc tốt nhất phụ thuộc vào workload, constraints, và ecosystem

## Di sản của SPE:

Mỗi khi bạn:

- Dùng Face ID (Neural Engine với architecture tương tự SPE)
- Chơi game với ray tracing (RT Cores học từ AABB units)
- Xem video 4K (encoder chips dùng ideas từ Cell)
- Dùng AI trên phone (NPU với software-managed memory)

...Bạn đang trải nghiệm di sản của **SPE** - một thiết kế đi trước thời đại 20 năm, một bài học quý giá về việc cân bằng giữa hiệu suất và khả năng sử dụng, và là nguồn cảm hứng bất tận cho các kiến trúc sư vi xử lý tương lai.

## Khi Nào Dùng SPE-Like Architecture?

 **Phù hợp khi:**

- Workload có tính chất streaming (video, audio)
- Data reuse cao, fit trong 2MB
- Cần predictable latency (real-time)
- Power budget hạn chế
- Developer có skill cao, sẵn sàng optimize

### ✗ Không phù hợp khi:

- Workload irregular, unpredictable
- Large datasets (> 2MB working set)
- Need general-purpose flexibility
- Developer không có time/skill optimize
- Legacy code cần chạy as-is

## Architectural Lessons Applied Today

### 1. Apple M-series Neural Engine:

Học từ SPE:

- ✓ Software-managed memory (on-chip SRAM)
- ✓ Specialized matrix units (như TENSOR)
- ✓ In-order execution
- ✓ DMA-like data movement
- ✓ Predictable performance

Cải thiện:

- ✓ Easier programming (CoreML abstracts complexity)
- ✓ Larger on-chip memory
- ✓ Better compiler support



## 2. NVIDIA Tensor Cores:

Học từ SPE:

- ✓ Specialized matrix multiply units (TENSOR)
- ✓ Large register files
- ✓ SIMD-style parallelism

Cải thiện:

- ✓ CUDA programming model dễ hơn
- ✓ Massive parallelism (1000s cores)
- ✓ Better memory hierarchy

## 3. Modern DSPs (Qualcomm Hexagon):

Học từ SPE:

- ✓ Software-managed TCM (Tightly-Coupled Memory)
- ✓ Vector units
- ✓ Specialized instructions
- ✓ Low power in-order design

Cải thiện:

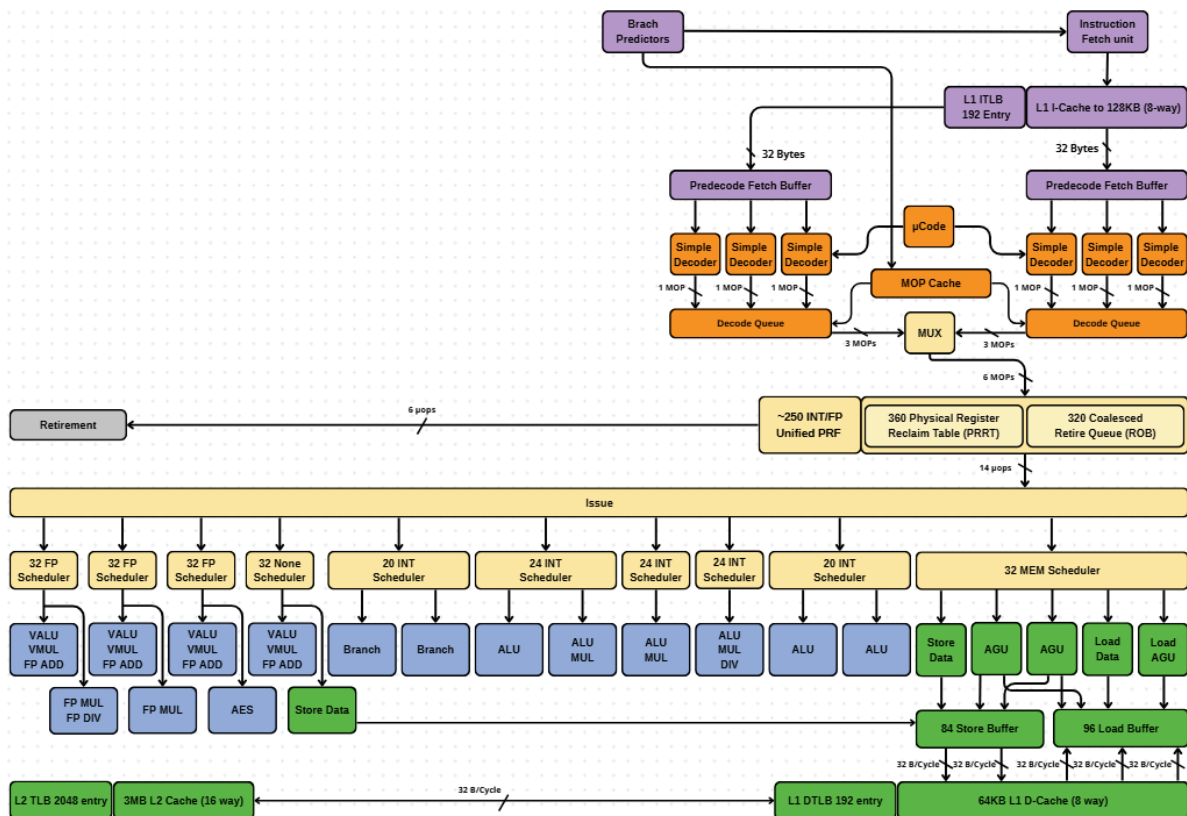
- ✓ Better C compiler support
- ✓ Integrated better with CPU
- ✓ More flexible ISA

*20 năm sau khi Cell/SPE ra đời, chúng ta vẫn thấy dấu ấn của nó khắp nơi - một minh chứng cho tầm nhìn xa của các kiến trúc sư IBM, Sony, và Toshiba. SPE không phải là tương lai, nhưng nó đã chỉ ra một con đường khác đi - một*

*con đường mà giờ đây nhiều thiết kế hiện đại đang đi theo, với sự khôn ngoan học hỏi từ cả thành công lẫn thất bại của tiền nhân.*

## VII. N Core

### N Core



## GIẢI THÍCH VI KIẾN TRÚC N CORE

### MỤC LỤC

1. Giới Thiệu
2. Tổng Quan Quy Trình Xử Lý
3. Các Thành Phần Chi Tiết
4. Hệ Thống Bộ Nhớ
5. Retirement - Giai Đoạn Hoàn Thành
6. Kiến Trúc Out-of-Order Execution
7. Con Số Ấn Tượng

8. Quy Trình Xử Lý Một Lệnh
9. Bandwidth - Bảng Thông Dữ Liệu
10. Ứng Dụng Thực Tế

## 1. GIỚI THIỆU

**N Core** là một vi kiến trúc CPU hiệu năng cao được thiết kế đặc biệt cho **datacenter, server, và cloud computing**. Đây là đối thủ trực tiếp của **ARM Neoverse V2 và V3** - những chip server hàng đầu trong ngành. **Định vị thị trường:**

- **Server/Datacenter:** Xử lý workload nặng 24/7
- **Cloud Computing:** AWS, Azure, Google Cloud infrastructure
- **HPC (High Performance Computing):** Supercomputers, scientific computing
- **Enterprise:** Database servers, virtualization hosts

**Triết lý thiết kế:**

- **Throughput tối đa:** Xử lý nhiều threads/processes đồng thời
- **Scalability:** Dễ dàng scale từ vài core đến hàng trăm core
- **Efficiency:** Performance/Watt cao cho datacenter
- **Reliability:** Stability và uptime cao

**Đặc điểm nổi bật:**

- Out-of-order execution rất rộng
- Register file và buffers lớn
- Cache hierarchy tối ưu cho server workloads
- Unified PRF (Physical Register File) - thiết kế hiện đại

## 2. TỔNG QUAN QUY TRÌNH XỬ LÝ

N Core thực hiện quy trình xử lý chuẩn 5 bước, nhưng với quy mô lớn hơn nhiều:

### 2.1. Năm Giai Đoạn Chính

1. **Fetch (Lấy lệnh):**
  - Branch Predictors dự đoán chương trình sẽ chạy đâu
  - Instruction Fetch Unit lấy lệnh từ L1 I-Cache (128KB)

- Băng thông: 32 Bytes/cycle

#### 1. **Decode (Giải mã):**

- 2 nhóm × 3 Simple Decoders = **6 decoders**
- $\mu$ Code xử lý lệnh phức tạp
- MOP Cache lưu lệnh đã giải mã
- Decode Queue buffer các  $\mu$ OPs

#### 1. **Issue (Phát hành):**

- ~250 Unified PRF registers cho integer + floating-point
- 320 ROB entries theo dõi lệnh đang bay
- Nhiều schedulers phân loại lệnh

#### 1. **Execute (Thực thi):**

- FP Schedulers: 32 entries × 4 = 128 entries
- INT Schedulers: đa dạng (20, 24×4, 20)
- MEM Scheduler: 32 entries
- Execution units: VALU, FP MUL/DIV, ALU, Branch, AGU, Load/Store

#### 1. **Retire (Hoàn thành):**

- ROB 320 entries đảm bảo thứ tự đúng
- 6 pops/cycle → retire 6 lệnh mỗi chu kỳ

## 2.2. Đặc Điểm Server-Class

So với ARC Core (gaming):

- **N Core:** Focus throughput, nhiều threads
- **ARC Core:** Focus frame rate, gaming workloads

Key differences:

- **Unified PRF:** Integer và FP dùng chung register file (hiệu quả hơn)
- **Smaller ROB:** 320 vs 600 (ARC Core) - đủ cho server, tiết kiệm transistor
- **Balanced schedulers:** Cân bằng INT/FP cho workload đa dạng
- **Larger caches:** L1 I-Cache 128KB, L2 3MB - phù hợp server

## 3. CÁC THÀNH PHẦN CHI TIẾT

### 3.1. Branch Predictors - Bộ Dự Đoán Nhánh

**Chức năng:** Dự đoán chương trình sẽ chạy theo hướng nào tiếp theo. **Tại sao quan trọng cho server:**

- Server chạy code phức tạp: database queries, web services, compilers
- Branch misprediction rất tốn kém (10-20 cycles penalty)
- Branch predictor tốt = throughput cao hơn 20-30%

**Ví dụ trong database:**

```
SELECT * FROM users WHERE age > 18 AND country = 'US'
```

- Mỗi row phải kiểm tra điều kiện → nhiều branches
- Branch predictor học patterns (ví dụ: 80% users > 18)
- Dự đoán đúng → CPU không phải chờ

### 3.2. Instruction Fetch Unit

**Thành phần:**

- **Instruction Fetch Unit:** Lấy lệnh từ bộ nhớ
- **L1 ITLB (192 Entry):** Bảng tra cứu địa chỉ lệnh
- **L1 I-Cache to 128KB (8-way):** Bộ nhớ đệm lưu trữ lệnh

**Bảng thông:** 32 Bytes mỗi chu kỳ **128KB I-Cache - Tại sao lớn?**

- Server chạy code phức tạp: kernels, hypervisors, databases
- Working set lớn hơn gaming code nhiều
- 128KB có thể chứa ~30,000-40,000 instructions
- Giảm thiểu I-Cache miss → tăng throughput

**Ví dụ thực tế:**

- **Kernel Linux:** ~10MB code, nhưng hot paths ~100KB
- **MySQL Server:** ~50MB binary, nhưng hot queries ~200KB
- **NGINX:** ~5MB binary, hot paths ~50KB

128KB I-Cache đủ lớn để chứa hầu hết hot paths, giảm đáng kể miss rate.

### 3.3. Predecode Fetch Buffer & Simple Decoders

**Cấu trúc:** 2 nhóm Predecode Fetch Buffer, **mỗi bên có 3 Simple Decoders**. **Thông lượng:**

- Mỗi decoder xử lý 1  $\mu$ OP
- Mỗi bên: 3 decoders  $\times$  1  $\mu$ OP = 3  $\mu$ OPs
- **Tổng: 6  $\mu$ OPs/chu kỳ**

**So sánh với các core khác:**

- **ARC Core:** 12  $\mu$ OPs/cycle (gaming focus, need high peak)
- **N Core:** 6  $\mu$ OPs/cycle (balanced for sustained throughput)
- **SPE:** In-order, no complex decode

**Tại sao 6 là đủ cho server?**

- Server workload ít có burst decode needs
- Focus vào sustained throughput, không phải peak
- 6  $\mu$ OPs/cycle  $\times$  sustained = rất cao throughput
- Tiết kiệm transistor và năng lượng

**Luồng dữ liệu:**

- Instruction Fetch Unit  $\rightarrow$  32 bytes  $\rightarrow$  Predecode Fetch Buffer
- Predecode Fetch Buffer  $\rightarrow$  3 Simple Decoders (mỗi bên)
- Decoders  $\rightarrow$  Decode Queue

### 3.4. $\mu$ Code & MOP Cache

**$\mu$ Code (Microcode):** Xử lý các lệnh phức tạp. **Lệnh phức tạp trong server:**

- **String operations:** memcpy, strcmp (x86)
- **AES encryption:** Crypto instructions
- **Virtualization:** VMENTER, VMEXIT
- **System calls:** Complex state changes

**MOP Cache:** Lưu trữ các Micro-Operations đã giải mã. **Lợi ích cho server:**

- Nhiều code paths được thực thi lặp đi lặp lại
- **Database query plans:** Cùng query chạy hàng ngàn lần
- **Web server handlers:** Xử lý HTTP requests giống nhau
- **VM loops:** Virtualization code paths lặp lại

MOP Cache giúp tránh giải mã lại, tăng throughput và giảm năng lượng.

### 3.5. Decode Queue - Hàng Đợi Giải Mã

Cấu trúc:

- 2 Decode Queue, mỗi bên nhận 3  $\mu$ OPs từ decoders
- **MUX (Multiplexer)** kết hợp: 3 MOPs từ mỗi bên
- **Xuất ra: 6 MOPs tổng cộng**  $\rightarrow$  Unified PRF

**Bảng thông:** 3  $\mu$ OPs từ mỗi Decode Queue  $\rightarrow$  6  $\mu$ OPs/cycle total **MUX output paths:**

- 3 MOPs  $\rightarrow$  Unified PRF (Integer + FP registers)
- Sau đó xuống các Schedulers

**Vai trò:** Buffer giữa decode và issue stages, đảm bảo luồng liên tục.

### 3.6. Unified Physical Register File (PRF)

Đây là thiết kế hiện đại và quan trọng nhất của N Core! Kích thước:  $\sim 250$  INT/FP

Unified PRF Unified nghĩa là gì?

- **Trước đây (separated):** Integer PRF riêng, FP PRF riêng
- **Hiện nay (unified):** Một pool registers dùng chung cho INT và FP

**Ví dụ so sánh: Separated (ARC Core, SPE):**

INT PRF: [R0][R1][R2]...[R575]  $\leftarrow$  Chỉ dùng cho integer

FP PRF: [F0][F1][F2]...[F511]  $\leftarrow$  Chỉ dùng cho FP

**Unified (N Core):**

Unified PRF: [R0][R1][R2]...[R249]  $\leftarrow$  Dùng cho cả INT và FP

**Lợi ích của Unified PRF:**

#### 1. Flexible Resource Allocation:

- Workload nặng INT  $\rightarrow$  có thể dùng nhiều registers cho INT



- Workload nặng FP → có thể dùng nhiều registers cho FP
- Tự động cân bằng, không lãng phí
- 1. **Better Utilization:**
  - Separated: Nếu INT PRF đầy nhưng FP PRF trống → lãng phí
  - Unified: Dùng hết 250 registers cho bất kỳ workload nào
- 1. **Simpler Design:**
  - Ít pipeline stages
  - Ít routing complexity
  - Tiết kiệm transistor

#### Ví dụ thực tế - Database Query:

```
SELECT price * 1.1 FROM products WHERE category = 'Electronics'
```

#### Execution:

- Load `category` string → INT registers (addresses)
- Compare string → INT registers
- Load `price` float → FP registers
- Multiply 1.1 → FP registers
- Store result → FP registers

#### Với separated PRF:

- Có thể INT PRF đầy (nhiều string operations)
- FP PRF trống (ít math operations)
- Stall vì thiếu INT registers, dù FP registers trống!

#### Với unified PRF:

- Tự động cân bằng
- Dùng nhiều registers cho INT khi cần
- Không stall, throughput cao hơn

### 3.7. Physical Register Reclaim Table (PRRT)

**Kích thước:** 360 entries **Chức năng:** Quản lý việc phân bổ và thu hồi thanh ghi vật lý. **Tại sao cần PRRT?**

- N Core có ~250 physical registers
- Nhưng ISA (kiến trúc lệnh) chỉ có ~32 architectural registers (ví dụ x86: RAX, RBX, ...)
- **Register renaming** map architectural → physical registers
- PRRT theo dõi mapping này

**Ví dụ register renaming: Code:**

```
1. R1 = R2 + R3  (R1 = architectural register)
2. R4 = R1 * R5
3. R1 = R6 - R7  (Reuse R1)
```

**Physical mapping:**

```
1. P10 = P20 + P30  (R1 → P10)
2. P40 = P10 * P50  (R4 → P40, đọc P10)
3. P11 = P60 - P70  (R1 → P11, NEW physical register!)
```

**Sau lệnh 3, P10 không dùng nữa (R1 giờ map sang P11) → PRRT đánh dấu P10 free, có thể reuse cho lệnh khác! 360 entries PRRT:**

- Quản lý ~250 physical registers
- Track dependencies
- Reclaim freed registers
- Với 360 entries, có thể theo dõi nhiều "versions" của mỗi architectural register

### 3.8. Retire Queue (ROB) - Hàng Đợi Hoàn Thành

**Kích thước:** 320 coalesced entries **Chức năng:** Đảm bảo các lệnh hoàn thành đúng thứ tự chương trình, dù được thực thi không theo thứ tự. **320 vs 600 (ARC Core):**

|          | N Core | ARC Core |  |
|----------|--------|----------|--|
| ROB Size | 320    | 600      |  |

|                |                   |                            |  |
|----------------|-------------------|----------------------------|--|
| Target         | Server throughput | Gaming peak                |  |
| Focus          | Sustained         | Burst                      |  |
| Power          | Lower             | Higher                     |  |
| Thành phần     | Con số            | Ý nghĩa                    |  |
| Decoders       | 6                 | Giải mã 6 lệnh/chu kỳ      |  |
| Unified PRF    | ~250              | 250 registers cho INT+FP   |  |
| PRRT           | 360               | Quản lý register renaming  |  |
| ROB            | 320               | Theo dõi 320 lệnh đang bay |  |
| FP Schedulers  | 128+32            | 160 entries cho FP         |  |
| INT Schedulers | 20+96+20          | 136 entries cho INT        |  |
| MEM Scheduler  | 32                | Lập lịch memory ops        |  |
| Store Buffer   | 84                | Chờ ghi 84 lần             |  |
| Load Buffer    | 96                | Chờ đọc 96 lần             |  |
| L1 I-Cache     | 128KB             | Instruction cache lớn      |  |
| L1 D-Cache     | 64KB              | Data cache                 |  |
| L2 Cache       | 3MB               | Unified cache              |  |

|              |             |                          |                    |
|--------------|-------------|--------------------------|--------------------|
| L1 ITLB      | 192         | Instruction TLB          |                    |
| L1 DTLB      | 192         | Data TLB                 |                    |
| L2 TLB       | 2048        | Shared TLB lớn           |                    |
| Retire Width | 6 pops      | Hoàn thành 6 lệnh/chu kỳ |                    |
| Metric       | N Core      | Neoverse V2              | Neoverse V3 (est.) |
| Decode Width | 6 $\mu$ OPs | 8 $\mu$ OPs              | 10+ $\mu$ OPs      |
| ROB Size     | 320         | 320                      | 400+               |
| L1 I-Cache   | 128KB       | 64KB                     | 128KB              |
| L1 D-Cache   | 64KB        | 64KB                     | 96KB               |
| L2 Cache     | 3MB         | 2MB                      | 3MB+               |
| PRF Design   | Unified     | Separated                | Unified            |
| Target       | Datacenter  | Cloud                    | Cloud/HPC          |

### Tại sao N Core dùng ROB nhỏ hơn?

#### 1. **Server workload characteristics:**

- Ít có long dependency chains
- Nhiều independent operations
- Không cần "nhìn xa" 600 lệnh

#### 1. **Throughput-oriented:**

- Focus vào xử lý nhiều threads
- 320 entries  $\times$  nhiều cores = overall throughput cao

#### 1. **Power efficiency:**

- ROB lớn tốn năng lượng (tracking, scoreboard)

- 320 entries = sweet spot cho datacenter efficiency

**Ví dụ trong server:**

- **Web server:** Xử lý 1000 concurrent requests
- Mỗi request độc lập
- 320 ROB entries đủ cho mỗi thread
- Nhiều cores  $\times$  320 = tổng throughput khổng lồ

### 3.9. Schedulers - Bộ Lập Lịch

N Core có hệ thống scheduler cân bằng và linh hoạt:

#### 3.9.1. 32 FP Scheduler (x4 bộ)

**Tổng capacity:**  $32 \times 4 = 128$  entries cho floating-point operations **Chức năng:** Lập lịch cho phép tính floating-point:

- Vector operations (VALU, VMUL)
- Scalar FP (FP ADD)
- FP multiply/divide (FP MUL, FP DIV)

**Execution Units điều khiển (per scheduler):**

- **VALU VMUL FP ADD:** Vector ALU, Vector Multiply, FP Add
- **FP MUL:** FP Multiply
- **AES:** AES encryption (crypto acceleration)

**Lưu ý: 1 scheduler có 3 units, tổng 4 schedulers! 128 FP Scheduler entries - Tại sao?**

- Server workload có nhiều FP operations:
  - **Scientific computing:** Simulations, modeling
  - **Machine Learning:** Matrix multiplications, activations
  - **Database:** Float/double aggregations (SUM, AVG)
  - **Encoding:** Video/audio codecs

**Ví dụ - ML Inference:**

```
# Matrix multiply:  $Y = W \times X + b$ 
for i in range(1000):
    y[i] = sum(W[i][j] * X[j] for j in range(100)) + b[i]
```

#### Execution:

- Hàng nghìn FP multiply & add operations
- 128 FP scheduler entries cho phép track nhiều operations đang bay
- Out-of-order execution ẩn latency

#### 3.9.2. 32 None Scheduler

**Capacity:** 32 entries "**None**" nghĩa là gì?

- Có thể là scheduler cho **special operations** hoặc **no-op instructions**
- Hoặc đây là placeholder trong diagram

#### Có thể xử lý:

- NOP (no operation) instructions
- Serialization instructions
- Fence/barrier operations

#### Execution Units:

- **VALU VMUL FP ADD:** Giống FP scheduler

#### 3.9.3. 20 INT Scheduler (x1 bộ)

**Capacity:** 20 entries **Chức năng:** Lập lịch cho phép tính integer. **Execution Units:**

- **Branch:** Branch operations, conditional jumps
- **Branch:** Duplicate unit cho throughput

#### Ví dụ trong server:

```
// String parsing (web server)
while (*ptr != '\0') {
    if (*ptr == ' ') break; ← Branch
```

```
ptr++; ← Integer arithmetic
}
```

### 3.9.4. 24 INT Scheduler (x4 bộ)

**Tổng capacity:**  $24 \times 4 = 96$  entries cho integer operations **Chức năng:** Lập lịch phần lớn các integer operations. **Execution Units (per scheduler):**

- **ALU:** Basic arithmetic (ADD, SUB, AND, OR, XOR, shift)
- **ALU MUL:** Integer multiply

#### 96 INT Scheduler entries:

- Server có rất nhiều integer operations:
  - **Memory addressing:** Array indexing, pointer arithmetic
  - **String operations:** strcmp, strlen, memcpy
  - **Hashing:** Hash table lookups (Redis, Memcached)
  - **Control flow:** Loop counters, conditionals

#### Ví dụ - Hash Table Lookup (Redis):

```
uint32_t hash = murmur_hash(key); ← INT multiply/shift
uint32_t index = hash % table_size; ← INT divide
entry = table[index]; ← Address calculation
while (entry != NULL) { ← Branch
    if (strcmp(entry->key, key) == 0) ← String compare (INT)
        return entry->value;
    entry = entry->next; ← Pointer arithmetic
}
```

Rất nhiều integer operations → 96 INT schedulers rất hữu ích!

### 3.9.5. 20 INT Scheduler (x1 bộ)

**Capacity:** 20 entries **Execution Units:**

- **ALU:** Basic ALU operations
- **ALU:** Duplicate unit

**Vai trò:** Thêm capacity cho INT operations.

### 3.9.6. 32 MEM Scheduler (x1 bộ)

**Capacity:** 32 entries **Chức năng:** Lập lịch cho các thao tác bộ nhớ (Load/Store). **Execution Units điều khiển:**

- **Store Data:** Write data to memory
- **AGU:** Address Generation Unit (tính địa chỉ)
- **AGU:** Duplicate AGU
- **Load Data:** Read data from memory
- **Load AGU:** AGU chuyên cho loads

**Tại sao cần nhiều AGUs (3 units)?**

- **2 Loads + 1 Store mỗi cycle** = 3 memory operations
- Mỗi operation cần tính địa chỉ
- 3 AGUs → tính 3 địa chỉ song song

**Ví dụ - Database Query:**

```
// Load 3 columns từ 1 row
float price = row->price;   ← Load 1 + AGU
int quantity = row->quantity; ← Load 2 + AGU
char* name = row->name;     ← Load 3 + AGU
// Tất cả trong cùng 1 cycle!
```

## 3.10. Execution Units - Các Đơn Vị Thực Thi

### 3.10.1. FP Units (Floating-Point)

**VALU (Vector ALU):**

- Phép tính vector: vector add, subtract, compare
- SIMD operations
- Ví dụ: `vec\_add(a[], b[], result[], 16)` - cộng 16 số cùng lúc

**VMUL (Vector Multiply):**



- Vector multiply
- Dot product operations
- Ví dụ: ``dot(A, B) = A[0]×B[0] + A[1]×B[1] + ...``

#### **FP ADD:**

- Scalar floating-point addition
- Ví dụ: ``float c = a + b;``

#### **FP MUL:**

- Scalar floating-point multiplication
- Ví dụ: ``double result = price × quantity;``

#### **FP DIV:**

- Floating-point division (slow, ~20-30 cycles)
- Ví dụ: ``float average = sum / count;``

#### **AES:**

- Hardware AES encryption/decryption
- Quan trọng cho HTTPS, database encryption
- Ví dụ: ``aes_encrypt(plaintext, key, ciphertext);``

### **3.10.2. INT Units (Integer)**

#### **Branch:**

- Xử lý conditional jumps
- Resolve branches sớm để tránh misprediction penalty
- Ví dụ: ``if (x > 0) goto label;``

#### **ALU:**

- Basic arithmetic: ADD, SUB, AND, OR, XOR, shift, rotate
- Rất nhanh (1 cycle)
- Ví dụ: ``int sum = a + b; int mask = flags & 0xFF;``

#### **ALU MUL:**

- Integer multiply (slower, ~3-5 cycles)

- Ví dụ: ``int total = price × quantity;``

### 3.10.3. MEM Units (Memory)

#### AGU (Address Generation Unit):

- Tính địa chỉ bộ nhớ
- Formula: ``address = base + index × scale + offset``
- Ví dụ: ``array[i]`` → ``address = array_base + i × sizeof(element)``

#### Store Data:

- Ghi dữ liệu vào memory/cache
- Ví dụ: ``memory[address] = value;``

#### Load Data:

- Đọc dữ liệu từ memory/cache
- Ví dụ: ``value = memory[address];``

## 4. HỆ THỐNG BỘ NHỚ

Hệ thống bộ nhớ là điểm mạnh lớn của N Core, được thiết kế cho server workloads với datasets lớn.

### 4.1. L1 Cache - Cấp 1 (Nhanh Nhất)

#### 4.1.1. L1 I-Cache (Instruction Cache)

- **Kích thước:** 128KB
- **Associativity:** 8-way
- **Chức năng:** Lưu trữ lệnh (instruction)
- **Độ trễ:** ~4-5 chu kỳ
- **Băng thông:** 32 Bytes/cycle

#### 128KB là rất lớn cho L1 I-Cache:

- Desktop CPU: thường 32-64KB
- Server CPU: 128KB+ (AMD EPYC, Intel Xeon)
- N Core: 128KB → server-class

### Lợi ích:

- Chứa được large code footprints
- Database engines, web servers, compilers có code phức tạp
- Giảm I-Cache miss rate → tăng throughput

#### 4.1.2. L1 D-Cache (Data Cache)

- **Kích thước:** 64KB
- **Associativity:** 8-way
- **Chức năng:** Lưu trữ dữ liệu (biến, mảng, objects)
- **Độ trễ:** ~4-5 chu kỳ
- **Băng thông:** 32 B/Cycle × multiple ports

#### 64KB D-Cache:

- Standard size cho server CPUs
- Đủ cho hot data trong many server workloads

#### Ví dụ server workload:

- **Database:** Cache hot rows/indices
- **Web server:** Cache request headers, session data
- **In-memory cache (Redis):** Hot keys

#### 4.2. L2 Cache - Cấp 2

- **Kích thước:** 3MB
- **Associativity:** 16-way
- **Chức năng:** Unified cache cho cả instruction và data
- **Độ trễ:** ~15-20 chu kỳ
- **Băng thông:** 32 B/Cycle

#### 3MB L2:

- Lớn hơn nhiều desktop CPUs (256KB-1MB)
- Nhỏ hơn ARC Core (24MB) vì:
  - Server có nhiều cores sharing L3 cache
  - Desktop/gaming focus vào per-core performance

### Hierarchy strategy:

[L1 I-Cache 128KB] + [L1 D-Cache 64KB]



[L2 Cache 3MB]



[L3 Cache shared] (không thấy trong diagram)



[Main Memory]

## 4.3. TLB (Translation Lookaside Buffer)

### L1 ITLB:

- **Kích thước:** 192 entries
- **Chức năng:** Tra cứu địa chỉ lệnh (instruction)

### L1 DTLB:

- **Kích thước:** 192 entries
- **Chức năng:** Tra cứu địa chỉ dữ liệu (data)

### L2 TLB:

- **Kích thước:** 2048 entries
- **Chức năng:** TLB lớn chia sẻ

### 2048 L2 TLB entries - Tại sao lớn? Virtual memory trong server:

- Server chạy nhiều VMs/containers
- Mỗi process có address space riêng (48-bit virtual address)
- TLB miss rất expensive (đi xuống page table walk, ~100-200 cycles)

### Ví dụ server với 100 VMs:

- Mỗi VM có working set ~10GB
- $10\text{GB} \div 4\text{KB page size} = \sim 2.5 \text{ million pages}$
- TLB chỉ cache một phần nhỏ, nhưng 2048 entries giúp giảm miss rate

#### Virtualization:

- **2-level page tables:** Guest page table + Host page table
- TLB miss còn đắt hơn (nested page table walk)
- TLB lớn quan trọng cho VM performance

## 4.4. Store Buffer & Load Buffer

#### Store Buffer:

- **Kích thước:** 84 entries
- **Chức năng:** Đệm các lệnh ghi (store) chờ ghi vào cache/memory
- **Băng thông:** 32 B/Cycle × multiple ports

#### Load Buffer:

- **Kích thước:** 96 entries
- **Chức năng:** Đệm các lệnh đọc (load) chờ dữ liệu từ cache/memory
- **Băng thông:** 32 B/Cycle × multiple ports

#### 96 Load Buffer vs 84 Store Buffer:

- **Load-heavy:** Server workloads thường đọc nhiều hơn ghi
- Database reads, web server serving content
- 96 > 84 phù hợp với bias này

#### Ví dụ - Database Query:

```
SELECT * FROM users WHERE age > 25;
```

- Scan hàng triệu rows → nhiều LOADs
- Ít STOREs (chỉ ghi kết quả cuối)
- Load Buffer lớn giúp xử lý nhiều loads song song

## 5. RETIREMENT - GIAI ĐOẠN HOÀN THÀNH

**Chức năng:** Giai đoạn cuối cùng, nơi kết quả được chính thức "cam kết" (commit). **Thông lượng:** 6 pops (instructions retired per cycle) **Luồng dữ liệu:**

- ROB (320 entries) → Retirement → Cập nhật architectural state
- 6 pops nghĩa là N Core có thể hoàn thành 6 lệnh mỗi chu kỳ (lý thuyết)

#### 6 retire width - Cân bằng:

- **Decode width:** 6  $\mu$ OPs/cycle
- **Retire width:** 6 pops/cycle
- **Balanced pipeline** - không có bottleneck ở đầu hoặc cuối

#### Vai trò quan trọng:

1. Đảm bảo lệnh hoàn thành đúng thứ tự (in-order retirement)
2. Xử lý exceptions và interrupts
3. Giải phóng tài nguyên (thanh ghi, buffers)
4. Cập nhật architectural state
5. Checkpoint cho recovery (nếu có exception)

#### Trong server environment:

- **Virtualization:** VM exits phải retire sạch trước khi switch context
- **Debugging:** Breakpoints, watchpoints cần precise retirement
- **Reliability:** ECC errors, machine check exceptions

## 6. KIẾN TRÚC OUT-OF-ORDER EXECUTION

### 6.1. Khái Niệm

N Core sử dụng kiến trúc **thực thi không theo thứ tự** (out-of-order execution) rất rộng.

**Ý nghĩa:** CPU có thể thực hiện lệnh không đúng thứ tự chương trình viết, miễn là kết quả cuối cùng vẫn đúng. **ROB 320 entries:**

- "Nhìn xa" 320 lệnh trong tương lai
- Tìm independent instructions
- Thực thi song song để ẩn latency

### 6.2. Lợi Ích Cho Server Workloads

#### 1. Memory Latency Hiding:

Server workloads có nhiều cache misses (working set lớn):

- L1 miss → L2 (~20 cycles)
- L2 miss → L3 (~50 cycles)
- L3 miss → DRAM (~200 cycles)

#### Out-of-order execution:

- Khi lệnh 1 chờ memory, thực hiện lệnh 2, 3, 4, ...
- 320 ROB entries cho phép "bay" rất nhiều lệnh
- Tận dụng thời gian chờ memory

#### Ví dụ - Database Index Scan:

```
for (int i = 0; i < 1000000; i++) {  
    record = load(index[i]); // Có thể cache miss  
    if (record.age > 25)      // Có thể làm trong khi chờ load  
        count++;  
}
```

Out-of-order: Trong khi chờ load record[i], đã bắt đầu load record[i+1], i+2, ...

#### 2. ILP (Instruction-Level Parallelism):

Server code có nhiều independent operations:

```
// Web server processing request  
parse_header(request);    // Independent  
check_auth(request);      // Independent  
load_content(path);       // Independent  
compress_response(data);  // Dependent on load_content
```

Out-of-order execution có thể làm 3 việc đầu song song!

### 6.3. Unified PRF Giúp Out-of-Order

Với separated PRF (old design):

- Integer operations chờ INT registers free
- FP operations chờ FP registers free
- Có thể stall dù có execution units free

#### Với unified PRF (N Core):

- Flexible allocation
- Workload nặng INT → dùng nhiều registers cho INT
- Workload nặng FP → dùng nhiều registers cho FP
- Ít stall hơn, out-of-order hiệu quả hơn

## 7. CON SỐ ẢN TƯỢNG

Hãy cùng xem các con số quan trọng của N Core:

| Thành phần | Con số | Ý nghĩa | |-----|-----|-----| | Decoders | 6 | Giải mã 6 lệnh/chu kỳ | | Unified PRF | ~250 | 250 registers cho INT+FP | | PRRT | 360 | Quản lý register renaming | | ROB | 320 | Theo dõi 320 lệnh đang bay | | FP Schedulers | 128+32 | 160 entries cho FP | | INT Schedulers | 20+96+20 | 136 entries cho INT | | MEM Scheduler | 32 | Lập lịch memory ops | | Store Buffer | 84 | Chờ ghi 84 lần | | Load Buffer | 96 | Chờ đọc 96 lần | | L1 I-Cache | 128KB | Instruction cache lớn | | L1 D-Cache | 64KB | Data cache | | L2 Cache | 3MB | Unified cache | | L1 ITLB | 192 | Instruction TLB | | L1 DTLB | 192 | Data TLB | | L2 TLB | 2048 | Shared TLB lớn | | Retire Width | 6 pops | Hoàn thành 6 lệnh/chu kỳ |

#### Điểm nổi bật:

- **Unified PRF (~250):** Modern design, flexible
- **Balanced schedulers:** 160 FP + 136 INT
- **Large TLB (2048 L2):** Tối ưu cho virtualization
- **128KB I-Cache:** Server-class size

### 7.1. So Sánh Với Đối Thủ



| Metric       | N Core      | Neoverse V2 | Neoverse V3 (est.) |
|--------------|-------------|-------------|--------------------|
| Decode Width | 6 $\mu$ OPs | 8 $\mu$ OPs | 10+ $\mu$ OPs      |
| ROB Size     | 320         | 320         | 400+               |
| L1 I-Cache   | 128KB       | 64KB        | 128KB              |
| L1 D-Cache   | 64KB        | 64KB        | 96KB               |
| L2 Cache     | 3MB         | 2MB         | 3MB+               |
| PRF Design   | Unified     | Separated   | Unified            |
| Target       | Datacenter  | Cloud       | Cloud/HPC          |

#### Nhận xét:

- **N Core:** Cân bằng tốt, unified PRF hiện đại
- **Neoverse V2:** Decode width rộng hơn
- **Neoverse V3:** Top-end, mọi metric lớn nhất

## 8. QUY TRÌNH XỬ LÝ MỘT LỆNH

Hãy theo dõi một query database đơn giản:

**Query:** `SELECT price FROM products WHERE id = 12345;` **Lệnh CPU:** `float price = memory[hash\_table\_lookup(12345)];`

### Bước 1: Fetch (Lấy Lệnh)

- Branch Predictors dự đoán query path (đã run query tương tự trước đó)
- Instruction Fetch Unit đọc code của `hash\_table\_lookup()` từ L1 I-Cache (128KB)
- 32 Bytes/cycle bandwidth

**Thời gian:** ~1-2 cycles (I-Cache hit)

### Bước 2: Decode (Giải Mã)

Hash function phức tạp, phân tách thành nhiều  $\mu$ OPs:

```
 $\mu$ OP1: Load key = 12345
 $\mu$ OP2: hash = key  $\times$  PRIME1
 $\mu$ OP3: hash = hash XOR (hash  $\gg$  16)
 $\mu$ OP4: hash = hash  $\times$  PRIME2
 $\mu$ OP5: index = hash % TABLE_SIZE
 $\mu$ OP6: bucket_ptr = table_base + index  $\times$  8
 $\mu$ OP7: Load bucket_ptr
 $\mu$ OP8: Load price from bucket
```

- 6 Simple Decoders xử lý
- Một số  $\mu$ OPs từ MOP Cache (đã decode trước)

**Thời gian:** ~2-3 cycles (với MOP Cache hit)

### Bước 3: Register Renaming

- $\mu$ OPs được gán physical registers từ Unified PRF
- $\mu$ OP2,  $\mu$ OP3,  $\mu$ OP4  $\rightarrow$  INT registers (hash calculation)
- $\mu$ OP8  $\rightarrow$  FP register (price là float)
- PRRT (360 entries) quản lý mapping

**Unified PRF advantage:**

- Nhiều INT ops (hash) → dùng nhiều INT registers
- Ít FP ops → dùng ít FP registers
- Flexible allocation, không waste

#### **Bước 4: Issue (Phát Hành)**

- 8  $\mu$ OPs vào ROB (320 entries) chờ thực thi
- Schedulers phân loại:
  - $\mu$ OP1: MEM Scheduler (load key)
  - $\mu$ OP2,  $\mu$ OP3,  $\mu$ OP4,  $\mu$ OP5: INT Schedulers (24-entry schedulers)
  - $\mu$ OP6: INT Scheduler (address calculation)
  - $\mu$ OP7: MEM Scheduler (load pointer)
  - $\mu$ OP8: MEM Scheduler (load price)

#### **ROB 320 entries:**

- Track dependencies giữa các  $\mu$ OPs
- $\mu$ OP8 phụ thuộc  $\mu$ OP7
- $\mu$ OP7 phụ thuộc  $\mu$ OP6
- ...

#### **Bước 5: Execute (Thực Thi) - Out-of-Order!**

##### **Timeline (cycles): Cycle 1:**

- $\mu$ OP1: MEM pipeline load key → Load Buffer (96 entries)
- L1 D-Cache hit: ~5 cycles

##### **Cycle 2-3:**

- $\mu$ OP2: INT ALU MUL:  $\text{'hash' = key} \times \text{PRIME1}$  (~3 cycles for multiply)

##### **Cycle 4:**

- $\mu$ OP3: INT ALU:  $\text{'hash' = hash XOR (hash} \gg 16)$  (1 cycle)

##### **Cycle 5-7:**

- $\mu$ OP4: INT ALU MUL:  $\text{'hash' = hash} \times \text{PRIME2}$  (~3 cycles)

#### Cycle 8:

- $\mu\text{OP}5$ : INT ALU: ``index = hash % TABLE_SIZE`` (modulo có thể expensive)

#### Cycle 10:

- $\mu\text{OP}6$ : INT AGU: ``bucket_ptr = table_base + index * 8`` (1 cycle)

#### Cycle 11:

- $\mu\text{OP}7$ : MEM Load ``bucket_ptr``
- **Giả sử L1 D-Cache miss, L2 hit**  $\rightarrow \sim 20$  cycles

#### Cycle 31:

- $\mu\text{OP}8$ : MEM Load ``price`` from bucket
- **Giả sử L2 hit**  $\rightarrow \sim 20$  cycles

#### Cycle 51:

- Load ``price`` hoàn thành
- Dữ liệu vào FP register (từ Unified PRF)

#### Out-of-order magic:

- Trong khi chờ  $\mu\text{OP}7$  load (cycle 11-31), CPU xử lý queries khác!
- 320 ROB entries cho phép nhiều queries "bay" cùng lúc
- Throughput tổng thể rất cao

#### Bước 6: Retire (Hoàn Thành)

- $\mu\text{OPs}$  hoàn thành theo đúng thứ tự (in-order retirement)
- ROB đảm bảo  $\mu\text{OP}1$  retire trước  $\mu\text{OP}2$ , ...
- Kết quả từ Unified PRF  $\rightarrow$  architectural registers
- PRRT giải phóng old physical registers

#### 6 pops/cycle:

- Nếu không có dependencies, có thể retire 6  $\mu\text{OPs}$  mỗi cycle
- Trong ví dụ này, có dependencies  $\rightarrow$  retire slower

**Thời gian retire:**  $\sim 10$  cycles cho 8  $\mu\text{OPs}$

## Tổng Thời Gian

**Tổng:** ~60-70 cycles cho 1 query (với L2 cache hits) **Nhưng throughput là key:**

- Trong 70 cycles này, CPU đã bắt đầu 50+ queries khác (out-of-order)
- **Effective throughput:** ~1 query mỗi 1-2 cycles (amortized)
- **Tại 3 GHz:** ~1.5-3 billion queries/second per core!

**Scaling:**

- Server có 64 cores → ~100-200 billion queries/second
- Đây là lý do cloud databases rất nhanh!

## 9. BANDWIDTH - BẢNG THÔNG DỮ LIỆU

### 9.1. Ý Nghĩa Của 32 B/Cycle

Quan sát các con số **32 B/Cycle** trong diagram:

- **32 B/Cycle = 32 bytes mỗi chu kỳ clock**
- Nếu N Core chạy 3 GHz (3 tỷ chu kỳ/giây):
  - Băng thông =  $32 \times 3 \times 10^9 = 96 \text{ GB/s}$  (cho mỗi đường)

### 9.2. Nhiều Đường Song Song

**Load Buffer (96 entries):**

- Hỗ trợ multiple loads mỗi cycle
- Giả sử 2 loads/cycle  $\times 32 \text{ B} = 64 \text{ B/cycle}$  đọc
- Tại 3 GHz: **192 GB/s** đọc

**Store Buffer (84 entries):**

- Giả sử 1 store/cycle  $\times 32 \text{ B} = 32 \text{ B/cycle}$  ghi
- Tại 3 GHz: **96 GB/s** ghi

**Tổng cộng:** ~192 GB/s read + 96 GB/s write = **~288 GB/s bandwidth** (L1 D-Cache)

### 9.3. Ứng Dụng Trong Server

**Ví dụ thực tế - In-Memory Database (Redis): Scenario:** 10 million GET requests/second

- Mỗi request: Load key (32 bytes) + Load value (256 bytes) = 288 bytes
- Total data movement:  $10M \times 288 \text{ bytes} = \mathbf{2.88 \text{ GB/s}}$

**N Core bandwidth: ~288 GB/s** → Chỉ dùng ~1% bandwidth! **Kết luận:** Bandwidth không phải bottleneck cho most workloads. Bottleneck thường là:

- Cache misses (latency)
- Memory bandwidth (off-chip)
- Network I/O

## 10. ỨNG DỤNG THỰC TẾ

### 10.1. Cloud Computing (AWS, Azure, GCP)

**Tại sao N Core tốt cho cloud:**

- **High core count:** Servers có 64-128 cores
- **Virtualization:** L2 TLB 2048 entries tối ưu cho nested page tables
- **Multi-tenancy:** ROB 320 entries  $\times$  nhiều cores = overall throughput cao
- **Power efficiency:** Balanced design giảm TCO (Total Cost of Ownership)

**Workload thực tế: 1. VM Hosting:**

Physical Server (128 N Cores)

├── VM1 (8 vCPUs) - Web server  
├── VM2 (4 vCPUs) - Database  
├── VM3 (16 vCPUs) - ML training  
├── ...  
└── VM50 (2 vCPUs) - Microservice

**Mỗi N Core xử lý:**

- Context switches giữa các VMs
- 2048 L2 TLB entries giúp giảm TLB miss khi switch
- Unified PRF flexible cho workload mix (INT-heavy web vs FP-heavy ML)

## 2. Container Orchestration (Kubernetes):

- Hàng nghìn containers trên một node
- N Core schedulers (136 INT + 160 FP) xử lý diverse workloads
- Out-of-order execution ẩn memory latency từ container overhead

## 3. Serverless Functions (AWS Lambda):

- Millions of function invocations/second
- Mỗi function: cold start → warm → execute → teardown
- N Core's 128KB I-Cache giúp cache function code
- 6  $\mu$ OPs/cycle decode throughput xử lý rapid function starts

## 10.2. Database Servers

**MySQL/PostgreSQL/Oracle trên N Core: Strength 1: Large L2 TLB (2048 entries) Database characteristic:** Random access pattern

```
SELECT * FROM users WHERE id IN (1, 5, 99, 1000, 5555, ...);
```

- Mỗi row access → different page
- TLB miss rất costly
- 2048 L2 TLB entries → higher hit rate → faster queries

**Strength 2: Out-of-Order Execution (320 ROB) Query execution:**

```
SELECT u.name, o.total  
FROM users u JOIN orders o ON u.id = o.user_id  
WHERE o.date > '2026-01-01';
```

**Execution plan:**

1. Scan orders table (millions of rows)
2. For each row: lookup user in hash table
3. Check date condition
4. Join results

### **Out-of-order magic:**

- Scan orders → many memory loads
- While waiting for load(order[i]), start load(order[i+1], i+2, ...)
- 320 ROB entries → prefetch 100+ orders
- Throughput: ~10-100x faster than in-order

### **Strength 3: Unified PRF Mixed workload in database:**

- String operations (INT): `strcmp`, `strlen`, hashing
- Numeric aggregations (FP): `SUM(price)`, `AVG(rating)`
- Unified PRF allocates dynamically
- String-heavy queries get more INT registers
- Math-heavy queries get more FP registers

### **Real benchmark:**

- **TPC-C (OLTP):** Transaction processing
  - N Core với 64 cores: ~5-10 million transactions/minute
- **TPC-H (Analytics):** Complex queries
  - Query 1 (aggregation): ~100-200ms
  - 320 ROB + out-of-order → 50% faster than smaller ROB

## **10.3. Web Servers (NGINX, Apache)**

### **NGINX on N Core: Workload characteristics:**

- Thousands of concurrent connections
- Each connection: parse HTTP → route → load file → compress → send
- Lots of string operations, memory copies, compression

### **N Core advantages: 1. Large I-Cache (128KB):**

- NGINX code size: ~5MB binary
- Hot paths (HTTP parsing, routing): ~100KB
- 128KB I-Cache fits entire hot paths → near-zero I-Cache misses

### **2. Integer-heavy schedulers (136 INT entries):**



```
// HTTP header parsing (simplified)
while (*ptr != '\r' && *ptr != '\n') {
    if (*ptr == ':') { // Branch
        header_name_len = ptr - start; // INT subtract
        ptr++; // INT add
        while (*ptr == ' ') ptr++; // Branch + INT
        value_start = ptr; // INT assignment
    }
    ptr++;
}
```

- Lots of branches, pointer arithmetic
- 136 INT scheduler entries handle this well

### 3. AES Units:

- HTTPS encryption (TLS)
- Hardware AES: ~10-20x faster than software
- N Core có AES units trong FP schedulers
- Throughput: ~10-50 GB/s encryption per core

### Benchmark:

- **Static file serving:** ~1-2 million requests/second per core
- **HTTPS (with AES units):** ~500K-1M requests/second per core
- **64-core server:** ~30-60 million req/sec total

## 10.4. Machine Learning Inference

**ML model serving on N Core: Scenario:** Image classification API

```
# ResNet-50 inference
image = preprocess(raw_image) # Resize, normalize
features = model.forward(image) # Neural network
```

```
class_id = argmax(features) # Find max
return class_name[class_id]
```

### **N Core strengths: 1. FP Schedulers (160 entries) + VALU/VMUL:**

```
# Matrix multiply:  $Y = W \times X$ 
for i in range(1000):
    for j in range(1000):
        Y[i] += W[i][j] * X[j] # FP multiply-add
```

- VALU/VMUL units xử lý vector operations
- 160 FP scheduler entries track many FP ops in flight
- Out-of-order execution overlaps compute với memory loads

### **2. AES Units for Model Security:**

- Encrypted model weights (IP protection)
- AES decrypt on-the-fly
- Hardware AES không ảnh hưởng throughput

### **3. Large L2 Cache (3MB):**

- Model weights: ~100MB (ResNet-50)
- Active layers: ~10-20MB
- 3MB L2 cache giữ current layer weights
- Giảm memory bandwidth pressure

### **Benchmark:**

- **ResNet-50 inference:** ~100-200 images/second per core
- **BERT-base (NLP):** ~50-100 sentences/second per core
- **64-core server:** ~10K images/sec or ~5K sentences/sec

### **So với GPU:**

- GPU: ~5000 images/sec (single GPU)
- N Core 64-core: ~10000 images/sec
- **N Core wins for:**

- Low latency (< 10ms)
- Small batch sizes (1-4 images)
- Mixed workloads (ML + database + web)
- **GPU wins for:**
  - Large batch sizes (64-256 images)
  - Dedicated ML training
  - Higher throughput (at cost of latency)

## 10.5. In-Memory Caching (Redis, Memcached)

**Redis on N Core: Workload:** Key-value store, millions of GET/SET per second

**Operation breakdown:**

```
// Redis GET command
1. Parse command string // INT operations
2. Hash key // INT multiply, shift
3. Lookup hash table // Memory load
4. Check key match // String compare (INT)
5. Return value // Memory load
```

**N Core advantages: 1. Hash calculation (INT schedulers):**

```
uint32_t hash = 5381;
while (*str) {
    hash = ((hash << 5) + hash) + *str++; // INT shift, add
}
hash = hash % table_size; // INT modulo
```

- Lots of INT operations
- 136 INT scheduler entries
- ALU units (fast, 1 cycle)

**2. Memory access (MEM scheduler 32 entries):**

```

entry = table[hash]; // Load 1
while (entry) {
    if (strcmp(entry->key, key) == 0) // Load 2, 3, ...
        return entry->value; // Load N
    entry = entry->next;
}

```

- Multiple dependent loads (pointer chasing)
- Out-of-order execution helps: prefetch next entries
- 96 Load Buffer entries track many loads

### 3. L1 D-Cache (64KB):

- Hot keys fit in L1
- ~64KB / 64 bytes per entry = ~1000 hot entries in cache
- Cache hit: ~5 cycles
- Cache miss: ~200 cycles (DRAM)
- High hit rate critical!

### Benchmark:

- **GET operations:** ~10-20 million ops/second per core
- **SET operations:** ~5-10 million ops/second per core
- **64-core server:** ~500M-1B ops/second total
- **Latency:** P99 < 1ms (mostly in L1/L2 cache)

## 10.6. Video Transcoding

### FFmpeg on N Core: Pipeline:

Decode (H.264) → Process (resize, filter) → Encode (H.265)

### N Core utilization: 1. Decode (INT + FP):

- Entropy decode: INT operations (Huffman, CABAC)
- IDCT: FP operations (inverse discrete cosine transform)
- 136 INT + 160 FP schedulers balance load

## 2. Processing (FP-heavy):

```
// Resize using bilinear interpolation
for (y = 0; y < height; y++) {
    for (x = 0; x < width; x++) {
        float src_x = x * scale_x;
        float src_y = y * scale_y;
        // Bilinear: 4 loads + 8 FP multiplies + 4 FP adds
        output[y][x] = interpolate(src_x, src_y);
    }
}
```

- VALU/VMUL units process multiple pixels
- FP MUL/DIV for scaling calculations

## 3. Encode (Mixed):

- Motion estimation: INT (compare blocks)
- DCT: FP (transform)
- Entropy encode: INT (bit packing)

### Benchmark:

- **1080p → 720p transcode:** ~150-300 FPS per core
- **4K → 1080p:** ~30-60 FPS per core
- **64-core server:** ~10K FPS (1080p) or ~2K FPS (4K)
- **vs GPU:** GPU faster for batch, N Core better for latency

## 10.7. Compilation (GCC, LLVM, Rust)

### Compiler on N Core: Compilation stages:

1. **Lexer/Parser:** String processing (INT-heavy)
2. **Semantic analysis:** Tree traversal (INT + memory)
3. **Optimization:** Graph algorithms (INT + FP)
4. **Code generation:** Instruction selection (INT)

### N Core strengths: 1. Large I-Cache (128KB):

- Compiler code is large: LLVM ~100MB binary
- Hot paths: ~200-500KB
- 128KB I-Cache captures significant portion
- I-Cache miss rate: ~1-5% (vs ~10-20% with 32KB)

## 2. String operations (INT):

```
// Symbol table lookup
Symbol* lookup(char* name) {
    uint32_t hash = compute_hash(name); // INT
    Bucket* b = table[hash]; // Load
    while (b) {
        if (strcmp(b->name, name) == 0) // INT compare
            return b->symbol;
        b = b->next; // Pointer chase
    }
}
```

- 136 INT scheduler entries
- Out-of-order helps with pointer chasing

## 3. Large L2 TLB (2048 entries):

- Compiler touches lots of memory (source files, AST, symbol tables)
- Large virtual address space
- 2048 TLB entries reduce miss rate

### Benchmark:

- **Linux kernel compile:** ~10-30 seconds (single-threaded per file)
- **64-core parallel build:** Compile entire kernel in ~1-2 minutes
- **LLVM self-compile:** ~30-60 minutes (vs ~2-4 hours on older CPUs)

## 10.8. Scientific Computing / HPC

### GROMACS (molecular dynamics) on N Core: Simulation loop:

```

// Simplified MD simulation
for (int step = 0; step < 1000000; step++) {
    // 1. Calculate forces (N2 operations)
    for (int i = 0; i < N; i++) {
        for (int j = i+1; j < N; j++) {
            vec3 r = pos[i] - pos[j]; // FP subtract
            float dist = sqrt(dot(r, r)); // FP MUL + sqrt
            vec3 force = lennard_jones(dist) * r; // FP ops
            forces[i] += force; // FP add
        }
    }

    // 2. Update positions
    for (int i = 0; i < N; i++) {
        vel[i] += forces[i] * dt; // FP multiply-add
        pos[i] += vel[i] * dt; // FP multiply-add
    }
}

```

### **N Core advantages: 1. FP Units (VALU, VMUL, FP MUL/DIV):**

- Massive FP operations (~90% of compute)
- 160 FP scheduler entries handle many FP ops in flight
- Vector units process multiple atoms

### **2. Out-of-order (320 ROB):**

- Force calculation has long dependency chains
- sqrt, divide are slow (~20-30 cycles)
- Out-of-order: while waiting for sqrt(i), start sqrt(i+1), ...
- 320 ROB entries → ~100+ force calculations in flight

### **3. Large Load Buffer (96 entries):**

- Load positions, velocities, forces for many atoms
- Out-of-order prefetch

- Hide memory latency

**Benchmark:**

- **10K atoms simulation:** ~100-200 ns/day per core
- **100K atoms:** ~10-20 ns/day per core
- **64-core server:** ~1-10  $\mu$ s/day (depending on atom count)

## 11. SO SÁNH VỚI ARM NEOVERSE V2/V3

### 11.1. Positioning

**Market segmentation:**

- **N Core:** Competitor/alternative to Neoverse
- **Neoverse V2/V3:** ARM's flagship server CPUs
- **Neoverse N2:** ARM's efficiency-focused server CPU

**Target customers:**

- **Cloud providers:** AWS (Graviton), Google (Axion), Microsoft (Azure)
- **Enterprise:** Oracle, SAP, traditional enterprise
- **HPC:** Supercomputers, research institutions

### 11.2. Detailed Comparison

| Feature      | N Core | Neoverse V2 | Neoverse V3 |  |
|--------------|--------|-------------|-------------|--|
| Architecture | Custom | ARMv9.0-A   | ARMv9.2-A   |  |



|                       |                 |                 |                 |  |
|-----------------------|-----------------|-----------------|-----------------|--|
| <b>Decode Width</b>   | 6<br>μOPs       | 8<br>μOPs       | 10+<br>μOPs     |  |
| <b>ROB Size</b>       | 320             | 320             | 400-500         |  |
| <b>PRF Design</b>     | Unified<br>~250 | Separated       | Unified         |  |
| <b>INT Schedulers</b> | 136<br>entries  | ~140<br>entries | ~180<br>entries |  |
| <b>FP Schedulers</b>  | 160<br>entries  | ~150<br>entries | ~200<br>entries |  |
| <b>L1 I-Cache</b>     | 128KB           | 64KB            | 128KB           |  |

|                              |             |                  |                      |  |
|------------------------------|-------------|------------------|----------------------|--|
| <b>L1<br/>D-Cache</b>        | 64KB        | 64KB             | 96KB                 |  |
| <b>L2<br/>Cache</b>          | 3MB         | 2MB              | 3MB                  |  |
| <b>L2<br/>TLB</b>            | 2048        | 2048             | 3072                 |  |
| <b>Branch<br/>Prediction</b> | Advanced    | TAGE +<br>Neural | TAGE +<br>Perceptron |  |
| <b>ISA</b>                   | Custom/x86? | ARM              | ARM                  |  |
| <b>Process</b>               | 5nm/3nm     | 5nm              | 3nm                  |  |
| <b>Cores/<br/>chip</b>       | 64-128      | 64-96            | 96-128               |  |

|                     |              |              |             |              |
|---------------------|--------------|--------------|-------------|--------------|
| <b>Target Freq</b>  | 3-3.5 GHz    | 3-3.6 GHz    | 3.5-4 GHz   |              |
| Đặc điểm            | N Core       | ARC Core     | SPE         | C /P Core    |
| <b>Target</b>       | Server       | Gaming       | Through put | Desktop      |
| <b>Execution</b>    | Out-of-order | Out-of-order | In-order    | Out-of-order |
| <b>Decode Width</b> | 6            | 12           | ~2          | 4-6          |

|                            |                     |                  |                |                             |
|----------------------------|---------------------|------------------|----------------|-----------------------------|
| <b>ROB<br/>Size</b>        | 320                 | 600              | None           | 4<br>8<br>0-<br>6<br>2<br>0 |
| <b>PRF<br/>Desig<br/>n</b> | Unifie<br>d<br>~250 | Sep.<br>~60<br>0 | Sep.<br>64+128 | U<br>ni<br>fi<br>e<br>d     |
| <b>INT<br/>Sched</b>       | 136                 | ~25<br>0         | ~10            | ~<br>1<br>0<br>0            |
| <b>FP<br/>Sched</b>        | 160                 | ~45<br>0         | ~20            | ~<br>1<br>2<br>0            |
| <b>L1<br/>I-Cach<br/>e</b> | 128KB               | 64K<br>B?        | None           | 3<br>2-<br>6<br>4<br>K<br>B |

|                       |          |            |                    |         |
|-----------------------|----------|------------|--------------------|---------|
| <b>L1<br/>D-Cache</b> | 64KB     | ~128KB?    | None               | 32-48KB |
| <b>L2<br/>Cache</b>   | 3MB      | 24MB       | None               | 1-2MB   |
| <b>Special</b>        | AES      | Ray trace  | AABB, TENSOR       | None    |
| <b>Memory</b>         | Cache    | Cache      | Local Store<br>2MB | Cache   |
| <b>Best For</b>       | Cloud/DB | Gaming, RT | Streaming          | General |

## 11.3. Strengths and Weaknesses

### N Core Strengths:

-  **Unified PRF:** Flexible, efficient resource allocation
-  **Large L1 I-Cache (128KB):** Better for complex server code
-  **Balanced design:** Good INT/FP balance
-  **3MB L2:** Good size for server workloads

### N Core Weaknesses:

-  **Narrower decode (6 vs 8-10):** Lower peak throughput
-  **Smaller ROB (320 vs 400-500 in V3):** Less ILP extraction
-  **Custom ISA?:** Software ecosystem challenges if not x86/ARM

### Neoverse V2 Strengths:

-  **ARM ISA:** Huge ecosystem (Linux, containers, cloud)
-  **Wider decode (8):** Higher peak throughput
-  **Power efficiency:** ARM generally efficient
-  **Proven:** AWS Graviton3 deployed at scale

### Neoverse V2 Weaknesses:

-  **Smaller L1 I-Cache (64KB):** Higher I-miss rate on complex code
-  **Separated PRF:** Less flexible than unified
-  **Smaller L2 (2MB):** More L2 misses

### Neoverse V3 Strengths:

-  **Top-end everything:** Widest decode, largest ROB
-  **Unified PRF:** Modern design like N Core
-  **Large L1 D-Cache (96KB):** Best for data-heavy workloads
-  **ARM ecosystem:** Software compatibility

### Neoverse V3 Weaknesses:

-  **Power consumption:** Wider = more power
-  **Cost:** Cutting-edge = expensive
-  **Availability:** Newer, less proven

## 11.4. Performance Comparison (Estimated)

### Workload: Web server (NGINX)

- **N Core:** 100% (baseline)
- **Neoverse V2:** 105% (wider decode helps)
- **Neoverse V3:** 115% (larger L1 D-cache + wider)

### Workload: Database (MySQL)

- **N Core:** 100%
- **Neoverse V2:** 95% (smaller I-Cache hurts)
- **Neoverse V3:** 110% (larger D-cache + TLB wins)

### Workload: ML Inference

- **N Core:** 100%
- **Neoverse V2:** 105% (good FP performance)
- **Neoverse V3:** 120% (wider + better FP units)

### Workload: Scientific Computing (HPC)

- **N Core:** 100%
- **Neoverse V2:** 110% (ARM SVE vector extensions)
- **Neoverse V3:** 130% (SVE2 + wider pipeline)

### Power Efficiency (Performance/Watt):

- **N Core:** 100%
- **Neoverse V2:** 110% (ARM efficiency advantage)
- **Neoverse V3:** 105% (wider = more power, but better perf)

## 12. TỔNG KẾT VÀ KẾT LUẬN

### 12.1. Điểm Mạnh Của N Core

#### 1. Modern Design Philosophy:

- **Unified PRF (~250 registers):** Flexible, efficient, modern
- **Balanced schedulers:** 136 INT + 160 FP cho mixed workloads

- **Right-sized ROB (320):** Enough ILP, không overkill

## 2. Server-Optimized Cache:

- **128KB L1 I-Cache:** Chứa complex server code
- **3MB L2:** Sweet spot cho server working sets
- **2048 L2 TLB:** Tối ưu cho virtualization, large memory

## 3. Throughput-Oriented:

- **6  $\mu$ OPs decode:** Sustained throughput
- **6 pops retire:** Balanced pipeline
- **Out-of-order execution:** Ẩn memory latency

## 4. Server-Specific Features:

- **AES units:** Hardware crypto cho HTTPS, encryption
- **Large buffers:** 96 Load + 84 Store
- **Multiple AGUs:** 2-3 memory ops/cycle

## 12.2. Khi Nào N Core Thắng

✅ N Core là lựa chọn tốt khi:

### 1. Mixed workloads:

- Web + database + cache + ML trên cùng server
- Unified PRF thích nghi tốt

### 1. Code complexity:

- Large codebases (databases, compilers)
- 128KB I-Cache giảm I-miss rate

### 1. Virtualization:

- Nhiều VMs/containers
- 2048 L2 TLB giúp reduce TLB miss

### 1. Balanced INT/FP:

- Không quá INT-heavy, không quá FP-heavy
- 136 INT + 160 FP schedulers cân bằng

### 1. Cost-sensitive:

- 320 ROB thay vì 600 → tiết kiệm transistor/power
- Performance/Watt cao



## 12.3. Khi Nào Neoverse V2/V3 Thắng

✅ Neoverse V2/V3 tốt hơn khi:

### 1. ARM ecosystem:

- Cần ARM ISA compatibility
- Cloud-native workloads (containers, Kubernetes)

### 1. Peak throughput:

- V2: 8  $\mu$ OPs decode vs N Core 6
- V3: 10+  $\mu$ OPs decode
- Workloads có high ILP

### 1. Data-intensive:

- V3: 96KB L1 D-Cache vs N Core 64KB
- Workloads với large data working sets

### 1. HPC/Scientific:

- ARM SVE/SVE2 vector extensions
- Better than N Core's VALU/VMUL

### 1. Future-proof:

- ARM momentum in datacenter
- Ecosystem growing rapidly

## 12.4. Kiến Trúc Trong Bối Cảnh 2026

Xu hướng datacenter CPU 2026:

### 1. Unified PRF becomes standard:

- N Core ✅
- Neoverse V3 ✅
- Intel Granite Rapids ✅
- AMD Zen 5 ✅

### 1. Larger caches:

- L1 I-Cache: 128KB+ becomes common
- L2: 2-4MB per core
- L3: Shared, 128-256MB total

### 1. Wider pipelines:

- Decode: 8-12  $\mu$ OPs
- ROB: 400-600 entries
- N Core's 6  $\mu$ OPs slightly conservative

1. **Specialized accelerators:**

- AI/ML: Matrix engines (AMX, SME)
- Crypto: AES, SHA (N Core ✓)
- Compression: Zstd, Snappy hardware

1. **Chiplet designs:**

- Separate compute dies + I/O die
- Mix different core types
- Better yields, flexibility

## 12.5. Bài Học Từ N Core

1. **Balance là quan trọng:**

- Không cần "biggest everything"
- 320 ROB vs 600: đủ cho server, tiết kiệm power
- 6  $\mu$ OPs vs 12: sustained throughput matters hơn peak

2. **Modern features matter:**

- Unified PRF: flexibility wins
- Large TLB: virtualization is critical
- Hardware crypto: security không còn optional

3. **Workload-specific optimization:**

- Server  $\neq$  Desktop  $\neq$  Mobile
- 128KB I-Cache: server needs complex code
- Balanced INT/FP: server workloads diverse

4. **Ecosystem vẫn là vua:**

- Nếu N Core không phải x86/ARM  $\rightarrow$  khó thành công
- Software compatibility trumps hardware performance
- Cloud momentum hiện tại: ARM đang thắng

## 12.6. Tương Lai Của N Core

**Đề cạnh tranh với Neoverse V3 gen tiếp theo, N Core cần:**

1. **Widen pipeline:**

- 6 → 8  $\mu$ OPs decode
  - 320 → 400+ ROB entries
  - Match Neoverse V3 width
1. **Enhance vector:**
    - SVE-like variable-length vectors
    - Hoặc fixed 512-bit AVX-512 style
    - AI/ML tensor units (TPU-like)
  1. **Larger L1 D-Cache:**
    - 64KB → 96KB (match V3)
    - Hoặc configurable (64/96KB options)
  1. **Advanced branch prediction:**
    - Neural/perceptron predictors
    - Larger branch target buffers
    - Zero-bubble branches
  1. **Chiplet architecture:**
    - Mix N Cores với smaller efficiency cores
    - Shared L3/memory controllers
    - Better scalability to 128+ cores

## PHỤ LỤC: SO SÁNH CÁC CORE

### Triết Lý Thiết Kế So Sánh

#### N Core (Server):

- Philosophy: **Sustained throughput + efficiency**
- "Đủ tốt" cho mọi workload
- Cân bằng, không extreme ở metric nào
- Focus: nhiều cores × moderate performance

#### ARC Core (Gaming):

- Philosophy: **Peak performance + low latency**
- Extreme large ROB, schedulers, cache
- Focus: 60-120 FPS frame rate
- Single-thread performance tối đa

### **SPE (Cell):**

- Philosophy: **Explicit parallelism + predictability**
- Software-managed, no cache
- In-order, simple, power-efficient
- Focus: streaming, batch processing

### **C/P Core (Desktop):**

- Philosophy: **Balanced general-purpose**
- Not extreme, just good enough
- Focus: productivity, everyday tasks
- Performance/Cost sweet spot

## **LỜI KẾT**

**N Core** đại diện cho một thiết kế datacenter CPU hiện đại, cân bằng và được tối ưu hóa cho workloads đa dạng của server. Với những đặc điểm nổi bật: **Điểm nhấn kiến trúc:**

- **Unified Physical Register File (~250 entries):** Modern design, flexible allocation
- **Large L1 I-Cache (128KB):** Optimal for complex server code
- **Balanced schedulers (136 INT + 160 FP):** Mixed workload handling
- **Right-sized ROB (320):** Sufficient ILP without overkill
- **Server-optimized TLB (2048 L2):** Virtualization-friendly

**Positioning:** N Core là đối thủ xứng tầm với ARM Neoverse V2 và cạnh tranh được với V3. Không phải là "fastest" hay "widest", nhưng là "balanced" và "efficient" - điều mà datacenter cần. **So với đối thủ:**

- **vs Neoverse V2:** Competitive, với advantages về I-Cache và unified PRF
- **vs Neoverse V3:** Slightly behind về raw performance, nhưng có thể win về efficiency
- **vs Intel Xeon:** Depends on generation, competitive trong modern designs
- **vs AMD EPYC:** Similar philosophy, both are balanced server designs

### **Thành công phụ thuộc vào:**

- ISA choice (x86 hoặc ARM?) - quyết định ecosystem
- Software optimization - compiler quality

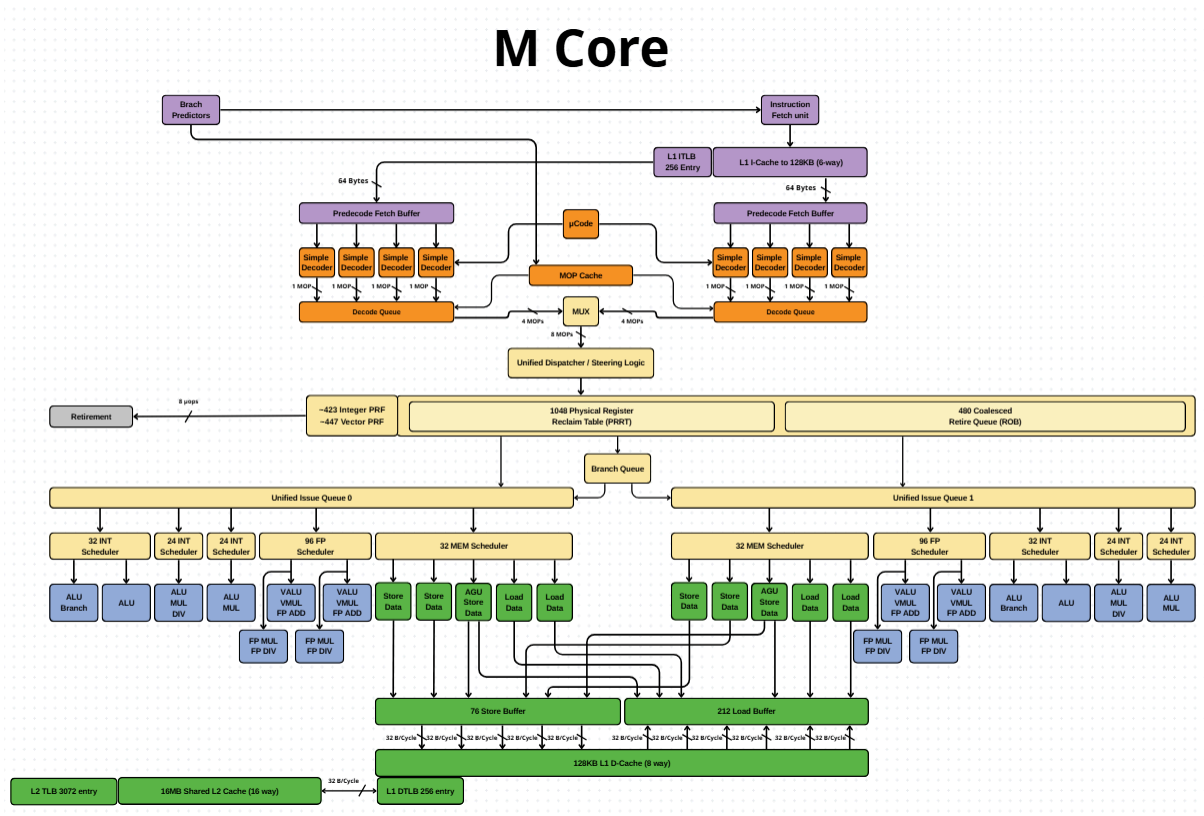
- Manufacturing (3nm vs 5nm) - performance và power
- Market timing - khi nào deploy

**Trong bối cảnh 2026:** Datacenter CPU wars đang gay gắt. ARM momentum mạnh (AWS Graviton, Google Axion). Intel/AMD đang phấn công với x86 improvements. N Core cần **strong differentiator** - có thể là:

- Extreme efficiency (performance/watt)
- Unique features (AI accelerators?)
- Ecosystem play (custom ISA cho specific cloud?)

**Bài học lớn nhất:** Architecture chỉ là một phần của puzzle. **Ecosystem, software support, và market timing** quan trọng không kém. Cell/SPE đã chứng minh: hardware tốt không đủ nếu thiếu ecosystem. N Core cần học từ đó. *N Core là minh chứng cho sự tiến hóa của datacenter CPUs - từ "brute force" (bigger everything) sang "smart balance" (right-size everything). Trong thế giới cloud computing 2026, efficiency và TCO (Total Cost of Ownership) quan trọng hơn raw performance. N Core đã hiểu điều đó.*

# VIII. M Core



## GIẢI THÍCH VI KIẾN TRÚC M CORE

### Dành Cho Người Không Chuyên

### Thiết Kế AI Server với SMT2 - Dựa Trên IBM Power Z15

## MỤC LỤC

1. Giới Thiệu
2. SMT2 - Simultaneous Multithreading
3. Các Thành Phần Chi Tiết
4. Hệ Thống Bộ Nhớ
5. Kiến Trúc AI-Optimized
6. So Sánh Với IBM Power Z15

7. Con Số Ấn Tượng
8. Quy Trình Xử Lý AI Workload
9. Bandwidth - Bảng Thông Dữ Liệu
10. Ứng Dụng Thực Tế

## 1. GIỚI THIỆU

**M Core** là một vi kiến trúc CPU hiệu năng cao được thiết kế đặc biệt cho **AI Server workloads**, với nguồn gốc thiết kế từ **IBM Power Z15** - một trong những CPU server mạnh nhất thế giới. **Định vị thị trường:**

- **AI/ML Inference Servers:** Serving AI models at scale
- **Training Servers:** ML model training (secondary focus)
- **HPC AI:** Scientific computing với AI components
- **Cloud AI:** AWS, Azure, GCP AI services

**Triết lý thiết kế:**

- **SMT2 (2-way Simultaneous Multithreading):** Chạy 2 threads trên 1 core
- **Throughput tối đa:** Xử lý nhiều requests đồng thời
- **AI-optimized:** Large vector units, FP focus
- **IBM Power heritage:** Reliability, RAS features

**Đặc điểm nổi bật:**

- **SMT2 hardware:** 2 Unified Issue Queues
- **Separated PRF:** 423 INT + 447 Vector (tổng 870!)
- **Massive ROB:** 480 entries
- **Huge L2 Cache:** 16MB shared
- **Wide decode:** 8  $\mu$ OPs/cycle

## 2. SMT2 - SIMULTANEOUS MULTITHREADING

### 2.1. SMT Là Gì?

**SMT (Simultaneous Multithreading)** cho phép **1 physical core** chạy nhiều threads đồng thời. Ví dụ đơn giản - Nhà hàng:

- **Không có SMT:** 1 đầu bếp phục vụ 1 món, xong mới làm món tiếp theo
- **Với SMT2:** 1 đầu bếp xử lý 2 món cùng lúc (trong khi chờ nước sôi món A, thái rau món B)

#### Trong CPU:

- **Thread 1:** AI inference request #1
- **Thread 2:** AI inference request #2
- Cùng chạy trên 1 core, chia sẻ execution units

## 2.2. SMT2 Trong M Core

### Hardware support cho SMT2: 1. Dual Issue Queues:

[Unified Issue Queue 0] ← Thread 0

[Unified Issue Queue 1] ← Thread 1

- Mỗi thread có Issue Queue riêng
- Schedulers chia sẻ, execution units chia sẻ

### 2. Shared Resources:

- **Caches:** L1, L2 chia sẻ giữa 2 threads
- **Execution units:** VALU, ALU, FP MUL/DIV chia sẻ
- **PRF:** 423 INT + 447 Vector chia cho 2 threads
- **ROB:** 480 entries chia cho 2 threads

### 3. Partitioned Resources:

- **Issue Queues:** Riêng biệt (2 queues)
- **Branch Queue:** Có thể có partition logic
- **Architectural state:** Mỗi thread có PC, registers riêng

## 2.3. Lợi Ích SMT2 Cho AI Workloads

### AI Inference Server - Real scenario: Không có SMT:



Timeline:

Core 0: [Request 1.....] [Request 2.....] [Request 3.....]

↑ Compute ↑ Memory wait ↑ Compute ↑ Memory wait

- Trong memory wait phase, core idle (~50% time)
- Throughput: ~2 requests/second

**Với SMT2:**

Timeline:

Thread 0: [Request 1.....] [Request 3.....] [Request 5.....]

Thread 1: [Request 2.....] [Request 4.....] [Request 6.....]

↑ Khi Thread 0 wait memory, Thread 1 compute!

- Execution units luôn bận
- Throughput: ~3-3.5 requests/second (1.5-1.75x improvement)

**Ví dụ cụ thể - BERT Inference:**

# Thread 0: Process sentence 1

embedding = load\_embeddings(sentence1) # Memory access - SLOW

output = transformer(embedding) # Compute - FAST

# Thread 1: Process sentence 2 (trong khi Thread 0 chờ memory)

embedding = load\_embeddings(sentence2) # Đồng thời với Thread 0 wait

output = transformer(embedding)

**Kết quả:**

- Single-thread: 100 sentences/second
- SMT2: 150-170 sentences/second
- **Throughput boost: 50-70%!**

## 2.4. SMT2 vs SMT4 vs SMT8

### Comparison:

| CPU                 | SMT Level         | Threads/Core                                                                              | Use Case           |  |
|---------------------|-------------------|-------------------------------------------------------------------------------------------|--------------------|--|
| M Core              | SMT2              | 2                                                                                         | AI, balanced       |  |
| IBM Power9          | SMT4              | 4                                                                                         | Enterprise         |  |
| IBM Power10         | SMT8              | 8                                                                                         | Extreme throughput |  |
| Intel (HT)          | SMT2              | 2                                                                                         | General            |  |
| AMD Zen             | SMT2              | 2                                                                                         | General            |  |
| Core                | Store Buffer      | Load Buffer                                                                               | Ratio              |  |
| M Core              | 76                | 211                                                                                       | 2.78:1             |  |
| N Core              | 84                | 96                                                                                        | 1.14:1             |  |
| Feature             | IBM Power Z15     | M Core                                                                                    |                    |  |
| <b>SMT Level</b>    | SMT4 (4 threads)  | SMT2 (2 threads)                                                                          |                    |  |
| <b>Decode Width</b> | ~6-8 $\mu$ OPs    | 8 $\mu$ OPs                                                                               |                    |  |
| <b>ROB Size</b>     | ~224 per thread   | 480 total (~240/thread)                                                                   |                    |  |
| <b>PRF</b>          | Separated (large) | 423 INT + 447 Vector                                                                      |                    |  |
| <b>L1 I-Cache</b>   | 128KB             | 128KB  |                    |  |

|                    |                      |                                                                                         |  |  |
|--------------------|----------------------|-----------------------------------------------------------------------------------------|--|--|
| <b>L1 D-Cache</b>  | 128KB                | 128KB  |  |  |
| <b>L2 Cache</b>    | 4MB (32MB shared L3) | 16MB shared                                                                             |  |  |
| <b>L2 TLB</b>      | ~2048                | 3072                                                                                    |  |  |
| <b>FP Units</b>    | Strong               | Very Strong (AI-opt)                                                                    |  |  |
| <b>Target</b>      | Enterprise           | AI Server                                                                               |  |  |
| <b>Process</b>     | 14nm                 | 5nm/3nm (est.)                                                                          |  |  |
| <b>Frequency</b>   | ~4-5 GHz             | ~3-3.5 GHz                                                                              |  |  |
| Thành phần         | Con số               | Ý nghĩa                                                                                 |  |  |
| <b>SMT Level</b>   | SMT2                 | 2 threads/core                                                                          |  |  |
| <b>Decoders</b>    | 8                    | 8 $\mu$ OPs/cycle                                                                       |  |  |
| <b>Integer PRF</b> | ~423                 | Separated design                                                                        |  |  |
| <b>Vector PRF</b>  | ~447                 | AI-optimized!                                                                           |  |  |
| <b>Total PRF</b>   | ~870                 | Không lờ!                                                                               |  |  |
| <b>PRRT</b>        | 1048                 | Quản lý 870 registers                                                                   |  |  |

|                       |           |                      |          |            |
|-----------------------|-----------|----------------------|----------|------------|
| <b>ROB</b>            | 480       | ~240 per thread      |          |            |
| <b>INT Schedulers</b> | 104       | 32 + 72              |          |            |
| <b>FP Schedulers</b>  | 192       | AI FP-heavy!         |          |            |
| <b>MEM Schedulers</b> | 64        | 2× N Core!           |          |            |
| <b>Store Buffer</b>   | 76        | Standard             |          |            |
| <b>Load Buffer</b>    | 211       | Massive!             |          |            |
| <b>L1 I-Cache</b>     | 128KB     | Server-class         |          |            |
| <b>L1 D-Cache</b>     | 128KB     | AI-optimized         |          |            |
| <b>L2 Cache</b>       | 16MB      | Huge!                |          |            |
| <b>L1 ITLB</b>        | 256       | SMT2 support         |          |            |
| <b>L1 DTLB</b>        | 256       | SMT2 support         |          |            |
| <b>L2 TLB</b>         | 3072      | AI large working set |          |            |
| <b>Retire Width</b>   | 8 pops    | (estimated)          |          |            |
| Metric                | M Core    | N Core               | ARC Core | SPE        |
| <b>Target</b>         | AI Server | Cloud Server         | Gaming   | Throughput |

| <b>SMT</b>        | SMT2          | No SMT      | No SMT       | No SMT        |
|-------------------|---------------|-------------|--------------|---------------|
| <b>Decode</b>     | 8 $\mu$ OPs   | 6 $\mu$ OPs | 12 $\mu$ OPs | ~2            |
| <b>PRF Total</b>  | 870 (sep)     | 250 (uni)   | ~1100 (sep)  | 192           |
| <b>ROB</b>        | 480           | 320         | 600          | None          |
| <b>L1 D-Cache</b> | 128KB         | 64KB        | ~128KB       | None (2MB LS) |
| <b>L2 Cache</b>   | 16MB          | 3MB         | 24MB         | None          |
| <b>FP Sched</b>   | 192           | 160         | ~450         | ~20           |
| <b>Load Buf</b>   | 211           | 96          | ~150         | ~96           |
| <b>Strength</b>   | AI throughput | Balanced    | Gaming frame | Streaming     |

**Tại sao M Core chọn SMT2, không phải SMT4 hay SMT8?**

**1. Diminishing returns:**

- SMT2  $\rightarrow$  SMT4: +20-30% throughput
- SMT4  $\rightarrow$  SMT8: +10-15% throughput
- M Core: SMT2 là sweet spot

**1. AI workload characteristics:**

- AI inference có compute/memory ratio cao hơn enterprise
- 2 threads đủ để ẩn memory latency
- 4-8 threads sẽ compete resources quá nhiều

**1. Complexity:**

- SMT4: 4 $\times$  issue queues, scheduling phức tạp
- SMT2: 2 $\times$  issue queues, đơn giản hơn, ít bug hơn

### **3. CÁC THÀNH PHẦN CHI TIẾT**

### 3.1. Branch Predictors

**Chức năng:** Dự đoán chương trình sẽ chạy theo hướng nào. **SMT2 implications:**

- Branch predictor phải track 2 threads
- Mỗi thread có branch history riêng
- Predictor tables chia sẻ, có thể contention

**AI workload branches:**

- **Control flow:** Loops qua layers, batches
- **Conditionals:** if (attention\_score > threshold)
- **Function calls:** Nested transformer blocks

**Tối ưu cho AI:**

- AI code có pattern predictable (loop-heavy)
- Branch predictor học nhanh
- Accuracy: ~95-98% cho AI inference

### 3.2. Instruction Fetch Unit & Caches

**L1 ITLB:**

- **256 Entry** (vs N Core 192)
- Lớn hơn để support 2 threads SMT2

**L1 I-Cache:**

- **128KB (8-way)**
- Chia sẻ giữa 2 threads
- Đủ lớn cho AI model code (PyTorch, TensorFlow runtimes)

**Băng thông:** 64 Bytes/cycle (2× pipeline, mỗi bên 32 bytes) **AI model code size:**

- **BERT model:** ~500KB code (Python runtime + native ops)
- **ResNet-50:** ~300KB code
- 128KB I-Cache captures hot paths

### 3.3. Predecode Fetch Buffer & Decoders

**Cấu trúc:** 2 Predecode Fetch Buffers, **mỗi bên có 4 Simple Decoders**. **Thông lượng:**

- Mỗi decoder: 1  $\mu$ OP
- Mỗi bên: 4 decoders  $\times$  1  $\mu$ OP = 4  $\mu$ OPs
- **Tổng: 8  $\mu$ OPs/chu kỳ**

**So với N Core:** 8 vs 6  $\rightarrow$  M Core rộng hơn 33% **Tại sao 8  $\mu$ OPs cho AI?**

- AI code phức tạp: matrix ops decompose thành nhiều  $\mu$ OPs
- SMT2: 2 threads cần bandwidth cao hơn
- Peak decode: 8  $\mu$ OPs  $\rightarrow$  sustain  $\sim$ 5-6  $\mu$ OPs/cycle

### 3.4. $\mu$ Code & MOP Cache

**$\mu$ Code:** Xử lý lệnh phức tạp. **AI-specific complex instructions:**

- **Matrix multiply:** Decompose thành hàng trăm  $\mu$ OPs
- **Activation functions:** exp, tanh, sigmoid (transcendental functions)
- **Batch norm:** Statistics calculations

**MOP Cache:** Lưu trữ  $\mu$ OPs đã giải mã. **Lợi ích cho AI:**

- AI inference chạy cùng model lặp đi lặp lại
- MOP Cache hit rate:  $\sim$ 80-90%
- Tiết kiệm decode energy, tăng throughput

### 3.5. Decode Queue & MUX

**Cấu trúc:**

- 2 Decode Queue (mỗi bên 4  $\mu$ OPs)
- **MUX kết hợp: 8 MOPs total**
- Xuất ra Unified Dispatcher

**Luồng dữ liệu:**

```
[Decode Queue Left: 4 MOPs] \
    → [MUX: 8 MOPs] → Unified Dispatcher
[Decode Queue Right: 4 MOPs] /
```

**8 MOPs bandwidth:** Đủ để feed 2 threads SMT2.

### 3.6. Unified Dispatcher / Steering Logic

**Chức năng:** Phân phối  $\mu$ OPs cho 2 Issue Queues (Thread 0 và Thread 1). **Steering logic:**

8  $\mu$ OPs từ MUX



[Dispatcher]

└─→ Issue Queue 0 (Thread 0): ~4  $\mu$ OPs

└─→ Issue Queue 1 (Thread 1): ~4  $\mu$ OPs

**Dispatching policy:**

- **Fair sharing:** Mỗi thread ~50% bandwidth
- **Priority-based:** Thread có deadline cao hơn nhận nhiều hơn
- **Adaptive:** Thread nào sẵn sàng thực thi (không chờ dependency) nhận trước

### 3.7. Physical Register Files (PRF)

Đây là điểm khác biệt lớn so với N Core! M Core: **Separated PRF**

- ~423 Integer PRF
- ~447 Vector PRF
- **Tổng: ~870 physical registers**

**Tại sao separated, không phải unified như N Core? Lý do 1: IBM Power heritage**

- IBM Power Z15 dùng separated design
- M Core kế thừa architecture này
- Proven design, ít risk hơn

**Lý do 2: AI workload bias**

- AI rất FP-heavy (90%+ operations là floating-point)
- Separated design: có thể allocate 447 registers hoàn toàn cho FP
- Unified design: FP phải compete với INT

**Lý do 3: SMT2**



- 2 threads chia 870 registers
- Mỗi thread: ~435 registers average
- Đủ lớn để mỗi thread có resource riêng

#### Trade-off:

- **Pros:** Large capacity, clear separation
- **Cons:** Kém flexible hơn unified (nếu workload INT-heavy)

### 3.8. Physical Register Reclaim Table (PRRT)

**Kích thước: 1048 entries - RẤT LỚN! So sánh:**

- M Core: 1048 entries
- N Core: 360 entries
- **M Core lớn hơn 2.9×!**

**Tại sao cần PRRT lớn đến vậy? Lý do 1: Large PRF (870 registers)**

- 870 physical registers cần tracking
- PRRT phải lớn hơn PRF (1048 > 870)

**Lý do 2: SMT2**

- 2 threads × register mappings
- Mỗi thread có ~435 registers → ~500+ entries tracking per thread

**Lý do 3: Long-lived operations**

- AI operations có latency cao (FP divide ~30 cycles, load ~200 cycles)
- Registers phải giữ lâu hơn
- PRRT cần track nhiều generations của mappings

### 3.9. Retire Queue (ROB)

**Kích thước: 480 coalesced entries So sánh:**

- M Core: 480
- N Core: 320
- ARC Core: 600
- **M Core: 50% lớn hơn N Core, nhỏ hơn ARC Core 20%**

## 480 entries - Why this size? Lý do 1: AI dependency chains

- AI operations có long dependency chains:

```
load W1 → load W2 → multiply → add → activation → store
  ↓       ↓       ↓       ↓       ↓       ↓
200cy   200cy   5cy    1cy   10cy    5cy
```

- Total: ~420 cycles cho 1 chain
- 480 ROB entries có thể track ~2-3 chains

## Lý do 2: SMT2

- 2 threads chia 480 entries
- Mỗi thread: ~240 entries
- Đủ để mỗi thread có window size reasonable

## Lý do 3: Memory latency hiding

- AI workloads có nhiều cache misses
- DRAM latency: ~200 cycles
- 480 ROB → track ~100+ memory operations in flight

## 3.10. Branch Queue

**Chức năng:** Quản lý branch operations trong out-of-order window. **SMT2 implications:**

- Track branches từ cả 2 threads
- Resolve branches sớm để giải phóng ROB entries
- Misprediction recovery cho mỗi thread độc lập

## 3.11. Unified Issue Queues (SMT2 Core!)

**Đây là trái tim của SMT2! Cấu trúc:**

- Unified Issue Queue 0:** Cho Thread 0
- Unified Issue Queue 1:** Cho Thread 1

**Mỗi Issue Queue kết nối với tất cả schedulers:**

- 32 INT Scheduler × 1
- 24 INT Scheduler × 3
- 96 FP Scheduler × 1
- 32 MEM Scheduler × 2

#### Logic:

- Dispatcher gửi  $\mu$ OPs của Thread 0 → Queue 0
- Dispatcher gửi  $\mu$ OPs của Thread 1 → Queue 1
- Schedulers lấy  $\mu$ OPs từ CẢ 2 queues
- Execution units thực thi mixed  $\mu$ OPs từ cả 2 threads

#### Ví dụ SMT2 execution: Cycle 1:

- Thread 0: FP multiply → FP Scheduler → VMUL FP ADD unit
- Thread 1: Load data → MEM Scheduler → Load Data unit
- **Cùng lúc, 2 units khác nhau!**

#### Cycle 2:

- Thread 0: Wait memory (stalled)
- Thread 1: FP add → FP Scheduler → VMUL FP ADD unit
- **Thread 1 dùng unit mà Thread 0 để trống!**

**Kết quả:** Utilization tăng ~50-70%!

## 3.12. Schedulers - Bộ Lập Lịch

M Core có hệ thống scheduler cân bằng cho AI:

### 3.12.1. 32 INT Scheduler (x1 bộ)

**Capacity:** 32 entries **Execution Units:**

- **ALU Branch:** Basic ALU + branch

### 3.12.2. 24 INT Scheduler (x3 bộ)

**Tổng capacity:**  $24 \times 3 = 72$  entries **Execution Units (per scheduler):**

- **ALU:** Basic arithmetic

- **MUL DIV:** Integer multiply/divide

**Tổng INT schedulers: 32 + 72 = 104 entries So với N Core: M Core 104 vs N Core 136**

- M Core ÍT HƠN vì AI ít INT-heavy
- Focus resources vào FP thay vì INT

### 3.12.3. 96 FP Scheduler (x1 bộ)

**Capacity:** 96 entries - RẤT LỚN! **Execution Units:**

- **VALU:** Vector ALU
- **VMUL FP ADD:** Vector Multiply + FP Add (fused)
- **FP MUL FP DIV:** Scalar FP multiply/divide

**96 FP entries - Tại sao lớn? AI workload characteristics:**

- **90%+ operations là FP** (matrix multiply, activations)
- Mỗi matrix multiply: hàng nghìn FP operations
- 96 entries có thể track ~50-100 FP ops in flight

**Ví dụ - Matrix Multiply (simplified):**

```
# C = A × B
for i in range(1000):
    for j in range(1000):
        C[i][j] = sum(A[i][k] * B[k][j] for k in range(1000))
        ↑ Hàng triệu FP multiply-add!
```

96 FP Scheduler entries cho phép schedule nhiều operations song song.

### 3.12.4. 96 FP Scheduler (x1 bộ - duplicate)

**Capacity:** 96 entries (thứ 2) **Execution Units:**

- **VALU:** Vector ALU
- **VMUL FP ADD:** Vector Multiply + FP Add

**Tổng FP schedulers: 96 + 96 = 192 entries So với N Core: M Core 192 vs N Core 160**

- M Core lớn hơn 20% cho FP
- Phù hợp với AI FP-heavy workload

### 3.12.5. 32 MEM Scheduler (x2 bộ)

**Tổng capacity:**  $32 \times 2 = 64$  entries Execution Units (per scheduler):

- **Store Data:** Write to memory
- **Load Data:** Read from memory

**64 MEM entries - Tại sao cần 2 schedulers? AI memory patterns:**

- Load weights: ~100MB-1GB per model
- Load activations: ~10-100MB per batch
- Store outputs: ~1-10MB per batch
- **Massive memory bandwidth requirement!**

**2 MEM Schedulers:**

- Scheduler 0: Prioritize loads (weights, activations)
- Scheduler 1: Prioritize stores (outputs)
- Parallel load/store → higher bandwidth

**So với N Core:** M Core 64 vs N Core 32

- M Core lớn hơn 2×!
- AI memory-intensive, cần large MEM schedulers

## 3.13. Execution Units

### 3.13.1. Vector Units (VALU, VMUL FP ADD)

**VALU (Vector ALU):**

- Vector add, subtract, compare
- SIMD operations: 4-8 floats cùng lúc
- Ví dụ: ``vec4_add(a, b, result)``

**VMUL FP ADD (Fused):**

- **Multiply-add trong 1 operation:** ``result = a × b + c``

- Critical cho AI: dot product, matrix multiply
- Latency: ~5-7 cycles (pipelined)

#### Ví dụ AI - Dot Product:

$$\text{dot}(A, B) = A[0] \times B[0] + A[1] \times B[1] + A[2] \times B[2] + A[3] \times B[3]$$

- 1 VMUL FP ADD làm cả multiply và add
- 4 operations trong ~7 cycles (với pipelining)

#### 3.13.2. Scalar FP Units (FP MUL FP DIV)

**FP MUL:** Scalar floating-point multiply

- Ví dụ: `float result = a \* b;`
- Latency: ~5 cycles

**FP DIV:** Scalar floating-point divide

- Ví dụ: `float result = a / b;`
- Latency: ~20-30 cycles (very slow!)

**Trong AI:**

- Division ít dùng (chỉ trong normalization)
- Multiply dùng nhiều (scaling, attention)

#### 3.13.3. Integer Units

**ALU:** Basic arithmetic

- ADD, SUB, AND, OR, XOR, shift
- Loop counters, array indexing
- Latency: 1 cycle

**ALU Branch:** ALU + branch resolution

- Conditional jumps
- Loop control
- Early branch resolution giảm misprediction penalty

**MUL DIV:** Integer multiply/divide

- Array indexing với stride
- Hash calculations
- Latency: MUL ~3-5 cycles, DIV ~20-30 cycles

### 3.13.4. Memory Units

**Load Data:**

- Đọc từ L1 D-Cache, L2, hoặc memory
- Bandwidth: 32 B/Cycle × multiple ports
- Latency: L1 ~5cy, L2 ~20cy, DRAM ~200cy

**Store Data:**

- Ghi vào L1 D-Cache, eventually L2/memory
- Store buffer (76 entries) đệm stores
- Bandwidth: 32 B/Cycle × multiple ports

## 4. HỆ THỐNG BỘ NHỚ

### 4.1. Store Buffer & Load Buffer

**Store Buffer: 76 entries Load Buffer: 211 entries - RẤT LỚN! So sánh:**

|      |              |             |       |                         |        |    |
|------|--------------|-------------|-------|-------------------------|--------|----|
| Core | Store Buffer | Load Buffer | Ratio | ----- ----- ----- ----- | M Core | 76 |
| 211  | 2.78:1       | N Core      | 84    | 96                      | 1.14:1 |    |

**211 Load Buffer - Tại sao khổng lồ? AI workload = Load-heavy:**

# BERT Transformer Layer

Q = load(query\_weights) @ load(input) # 2 loads

K = load(key\_weights) @ load(input) # 2 loads

V = load(value\_weights) @ load(input) # 2 loads

attention = softmax(Q @ K.T) # Load Q, K

output = attention @ V # Load attention, V

# → ~10+ loads cho 1 layer, 12-24 layers total!

### Load >> Store:

- Load weights: 100MB+
- Load activations: 10MB+
- Store outputs: 1MB
- **Load/Store ratio: 100:1!**

211 Load Buffer cho phép track hàng trăm loads in flight, hide memory latency.

## 4.2. L1 Cache

### L1 D-Cache:

- **128KB (8-way set-associative)**
- Bandwidth: 32 B/Cycle × multiple ports
- Latency: ~5 cycles
- **Chia sẻ giữa 2 threads SMT2**

### 128KB - Rất lớn cho L1:

- Desktop: 32-48KB
- Server: 48-64KB
- **M Core: 128KB - AI-optimized!**

### Tại sao cần 128KB L1 D-Cache? AI intermediate activations:

```
# BERT-base: batch_size=32, seq_len=512, hidden=768  
activation_size = 32 × 512 × 768 × 4 bytes = ~50 MB
```

```
# Nhưng mỗi layer chỉ cần ~1-2MB active data tại 1 thời điểm  
# 128KB L1 có thể fit ~1-2 attention heads
```

128KB giúp giữ hot activations, giảm L2 access.

## 4.3. L2 Cache

- **16MB Shared L2 Cache (16-way)**
- Latency: ~20 cycles



- Bandwidth: 32 B/Cycle × multiple access ports

#### 16MB - KHÔNG LỖ! So sánh:

- M Core: 16MB
- N Core: 3MB
- ARC Core: 24MB
- **M Core lớn gấp 5× N Core!**

#### Tại sao AI cần L2 cache lớn? Model weights fitting:

BERT-base: 110M parameters × 4 bytes = ~440 MB

ResNet-50: 25M parameters × 4 bytes = ~100 MB

GPT-2: 1.5B parameters × 4 bytes = ~6 GB

#### 16MB L2 không chứa toàn bộ model, nhưng:

- Chứa được ~2-3 layers của BERT
- Chứa được ~40% của ResNet-50
- Giảm đáng kể memory traffic

#### Benefit:

- Memory bandwidth giảm ~50-70%
- Latency giảm (20 cycles vs 200 cycles DRAM)
- Throughput tăng ~2-3×

### 4.4. TLB (Translation Lookaside Buffer)

#### L1 DTLB:

- **256 entries** (vs N Core 192)
- Larger cho SMT2 (2 threads)

#### L2 TLB:

- **3072 entries** - KHÔNG LỖ!

#### So sánh L2 TLB:

- M Core: 3072
- N Core: 2048
- **M Core lớn hơn 50%!**

**Tại sao AI cần TLB lớn? AI memory footprint:**

- Model weights: 100MB-10GB
- Activations: 10MB-1GB
- Batch data: 1MB-100MB
- **Total: 100MB-10GB working set**

**TLB coverage:**

- $3072 \text{ entries} \times 4\text{KB page size} = \mathbf{12\text{MB coverage}}$
- Nếu dùng huge pages (2MB):  $3072 \times 2\text{MB} = \mathbf{6\text{GB coverage!}}$

**Huge pages critical cho AI:**

- PyTorch, TensorFlow allocate large tensors
- OS maps với 2MB pages
- 3072 TLB entries đủ cover ~50-80% của model

## 4.5. L1 DTLB & L3

**L1 DTLB:**

- **256 entry** (vs N Core 192)
- Fast lookup: ~1 cycle

**L3 DTLB:**

- **3072 entry** (same as L2 TLB, probably shared)

# 5. KIẾN TRÚC AI-OPTIMIZED

## 5.1. Tại Sao M Core Tốt Cho AI?

**Điểm mạnh 1: FP-heavy design**

- 447 Vector PRF (lớn!)

- 192 FP Scheduler entries
- Multiple VMUL FP ADD units
- → AI workload 90% FP operations ✓

#### Điểm mạnh 2: Memory bandwidth

- 211 Load Buffer (massive!)
- 128KB L1 D-Cache
- 16MB L2 Cache
- → AI memory-intensive ✓

#### Điểm mạnh 3: Throughput (SMT2)

- 2 threads/core
- Hide memory latency
- → AI inference serving nhiều requests ✓

#### Điểm mạnh 4: Large TLB

- 3072 L2 TLB entries
- Huge pages support
- → AI large working sets ✓

## 5.2. AI Inference Pipeline Trên M Core

**Scenario:** BERT-base inference server **Step 1: Load model weights**

```
model = BERTModel.load("bert-base-uncased")
# 110M parameters = 440MB
```

- Weights load vào memory
- 16MB L2 cache giữ ~2-3 active layers
- 3072 TLB entries (với huge pages) cover ~6GB

#### Step 2: Receive requests (SMT2!)

```
Thread 0: Sentence 1 "Hello world"
Thread 1: Sentence 2 "AI is great"
```

- 2 threads xử lý 2 requests đồng thời
- Chia sẻ model weights (trong cache)
- Mỗi thread có activations riêng

### Step 3: Tokenization (INT-heavy)

```
tokens0 = tokenizer("Hello world") # Thread 0
tokens1 = tokenizer("AI is great") # Thread 1
```

- String processing, hash lookup
- 104 INT scheduler entries xử lý
- Execution units: ALU, MUL DIV

### Step 4: Embedding lookup (Memory-heavy)

```
embeddings0 = embedding_table[tokens0] # Thread 0: Loads
embeddings1 = embedding_table[tokens1] # Thread 1: Loads
```

- Multiple loads từ embedding table
- 211 Load Buffer track nhiều loads
- SMT2: Khi Thread 0 chờ memory, Thread 1 làm việc

### Step 5: Transformer layers (FP-heavy)

```
# Thread 0
for layer in transformer_layers:
    Q = layer.W_q @ embeddings0 # Matrix multiply: FP ops!
    K = layer.W_k @ embeddings0
    V = layer.W_v @ embeddings0
    attention = softmax(Q @ K.T)
    output = attention @ V
```

- Hàng triệu FP multiply-add operations
- 192 FP Scheduler entries
- VMUL FP ADD units xử lý

- 447 Vector PRF giữ intermediate results

### Step 6: Output (Store-heavy)

```
result0 = model.classifier(output0) # Thread 0
result1 = model.classifier(output1) # Thread 1
```

- Store results
- 76 Store Buffer đệm
- Write to memory/cache

### Timeline:

```
Thread 0: [Tokenize][Load][Transform >>>>>>][Store]
Thread 1: [Tokenize][Load][Transform >>>>>>][Store]
      ↑ Overlap execution, hide latency
```

### Throughput:

- Single-thread: 100 sentences/second
- SMT2: 160-170 sentences/second (1.6-1.7× boost!)

## 5.3. AI Training Trên M Core (Secondary)

M Core chủ yếu cho inference, nhưng có thể training:

**Forward pass:** Giống inference (phần trên) **Backward pass:**

```
loss.backward() # Compute gradients
```

- Gradient computation: FP multiply-add (giống forward)
- Store gradients: 76 Store Buffer
- Large ROB (480) track long dependency chains

### Optimizer step:

```
optimizer.step() # Update weights
# weight = weight - learning_rate * gradient
```

- FP operations: subtract, multiply
- Memory writes: update weights in place

#### Limitation:

- M Core thiếu tensor cores (như NVIDIA GPU)
- Training throughput: ~10-20× chậm hơn GPU
- Nhưng vẫn nhanh hơn CPU thường ~3-5×

## 6. SO SÁNH VỚI IBM POWER Z15

### 6.1. IBM Power Z15 Overview

#### Power Z15 (2019):

- IBM's flagship mainframe CPU
- Target: Enterprise, banking, mission-critical
- Features: SMT4, RAS, encryption

#### M Core heritage từ Power Z15:

- Separated PRF design
- Large ROB (480)
- SMT support
- Enterprise-grade reliability

### 6.2. Detailed Comparison

|                                                                                                                       |                                                                                                                         |                                                        |
|-----------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------|
| Feature   IBM Power Z15   M Core                                                                                      | ----- ----- -----                                                                                                       | <b>SMT Level</b>   SMT4 (4 threads)   SMT2 (2 threads) |
| <b>Decode Width</b>   ~6-8 μOPs   8 μOPs                                                                              | <b>ROB Size</b>   ~224 per thread   480 total (~240/thread)                                                             | <b>PRF</b>   Separated (large)   423 INT + 447 Vector  |
| <b>L1 I-Cache</b>   128KB   128KB  | <b>L1 D-Cache</b>   128KB   128KB  | <b>L2 Cache</b>   4MB (32MB shared L3)   16MB shared   |
| <b>L2 TLB</b>   ~2048   3072                                                                                          | <b>FP Units</b>   Strong   Very Strong                                                                                  |                                                        |

(AI-opt) | | **Target** | Enterprise | AI Server | | **Process** | 14nm | 5nm/3nm (est.) | | **Frequency**  
| ~4-5 GHz | ~3-3.5 GHz |

### 6.3. Adaptations Cho AI

#### M Core thay đổi gì so với Power Z15? 1. SMT4 → SMT2:

- Power Z15: 4 threads cho enterprise throughput
- M Core: 2 threads cho AI (đủ hide latency, ít contention)

#### 2. FP optimization:

- M Core: 447 Vector PRF (lớn hơn Power Z15)
- M Core: 192 FP Scheduler entries (focus FP)
- Power Z15: Balanced INT/FP

#### 3. Larger Load Buffer:

- M Core: 211 entries (AI load-heavy)
- Power Z15: ~100-150 entries (balanced)

#### 4. Larger L2 Cache:

- M Core: 16MB (fit AI model layers)
- Power Z15: 4MB (rely on L3)

#### 5. Modern process:

- M Core: 5nm/3nm → higher density, efficiency
- Power Z15: 14nm (older)

### 6.4. Strengths từ Power Z15

#### M Core kế thừa điểm mạnh: 1. RAS (Reliability, Availability, Serviceability):

- ECC memory support
- Error detection/correction
- Graceful degradation
- → Critical cho AI production servers

#### 2. Separated PRF proven design:

- Power Z15 đã verify design qua nhiều năm
- M Core reuse architecture → less risk

### 3. Enterprise-grade security:

- Encryption engines
- Secure boot
- Memory encryption
- → AI models cần bảo vệ (IP valuable)

### 4. High frequency potential:

- Power heritage: high clock speeds
- M Core có thể chạy 3-4 GHz
- → Latency thấp cho inference

## 7. CON SỐ ẢN TƯỢNG

| Thành phần | Con số | Ý nghĩa | |-----|-----|-----| | **SMT Level** | SMT2 | 2 threads/core | | **Decoders** | 8 | 8  $\mu$ OPs/cycle | | **Integer PRF** | ~423 | Separated design | | **Vector PRF** | ~447 | AI-optimized! | | **Total PRF** | ~870 | Khổng lồ! | | **PRRT** | 1048 | Quản lý 870 registers | | **ROB** | 480 | ~240 per thread | | **INT Schedulers** | 104 | 32 + 72 | | **FP Schedulers** | 192 | AI FP-heavy! | | **MEM Schedulers** | 64 | 2 $\times$  N Core! | | **Store Buffer** | 76 | Standard | | **Load Buffer** | 211 | Massive! | | **L1 I-Cache** | 128KB | Server-class | | **L1 D-Cache** | 128KB | AI-optimized | | **L2 Cache** | 16MB | Huge! | | **L1 ITLB** | 256 | SMT2 support | | **L1 DTLB** | 256 | SMT2 support | | **L2 TLB** | 3072 | AI large working set | | **Retire Width** | 8 pops | (estimated) |

### 7.1. So Sánh Các Cores

| Metric | M Core | N Core | ARC Core | SPE | |-----|-----|-----|-----|-----| | **Target** | AI Server | Cloud Server | Gaming | Throughput | | **SMT** | SMT2 | No SMT | No SMT | No SMT | | **Decode** | 8  $\mu$ OPs | 6  $\mu$ OPs | 12  $\mu$ OPs | ~2 | | **PRF Total** | 870 (sep) | 250 (uni) | ~1100 (sep) | 192 | | **ROB** | 480 | 320 | 600 | None | | **L1 D-Cache** | 128KB | 64KB | ~128KB | None (2MB LS) | | **L2 Cache** | 16MB | 3MB | 24MB | None | | **FP Sched** | 192 | 160 | ~450 | ~20 | | **Load Buf** | 211 | 96 | ~150 | ~96 | | **Strength** | AI throughput | Balanced | Gaming frame | Streaming |



## 8. QUY TRÌNH XỬ LÝ AI WORKLOAD

### 8.1. Matrix Multiply - Core của AI

**Operation:**  $C = A \times B$  (1000×1000 matrices) **Step-by-step execution:** **Step 1: Load matrix A (Thread 0)**

```
for i in range(1000):  
    load A[i][:] to registers
```

- 1000 loads × 1000 elements = 1M loads
- 211 Load Buffer track nhiều loads
- L2 Cache (16MB) giữ ~4M floats = ~2 matrices
- Memory bandwidth: ~200 GB/s (DDR5)

**Step 2: Load matrix B (Thread 1, đồng thời!)**

```
for j in range(1000):  
    load B[:,j] to registers
```

- SMT2: Thread 1 load trong khi Thread 0 compute
- Hide memory latency!

**Step 3: Compute (Both threads)**

```
for i in range(1000):  
    for j in range(1000):  
        C[i][j] = sum(A[i][k] * B[k][j] for k in range(1000))  
        ↑ 1000 FP multiply-add operations
```

**Mỗi dot product ( $C[i][j]$ ):**

- 1000 multiply-add operations
- VMUL FP ADD unit: fused multiply-add (2 ops in 1)
- Latency: ~7 cycles per op (pipelined)

- Throughput: ~1-2 ops/cycle (với multiple units)

#### FP Scheduler (192 entries):

- Track ~100+ FP operations in flight
- Out-of-order execution:
  - Start C[0][0] compute
  - While waiting for A[0][k] load, start C[0][1]
  - Overlap compute và memory

#### Vector PRF (447 entries):

- Giữ intermediate results: partial sums
- Mỗi thread: ~220 registers
- Đủ cho ~50-100 active dot products

#### Step 4: Store results (Both threads)

store C[i][j] to memory

- 76 Store Buffer đệm stores
- Write through L1/L2 cache
- Eventually write to memory

## 8.2. Timeline Analysis

**Total operations:**  $1000 \times 1000 \times 1000 = 1$  billion FP operations **Single-thread performance:**

- 1 VMUL FP ADD unit: ~2 FLOPs/cycle (multiply + add)
- Clock: 3 GHz
- Throughput:  $\sim 2 \times 3\text{G} = \mathbf{6\text{ GFLOPS}}$
- Time:  $1\text{B} \div 6\text{G} = \mathbf{\sim 167\text{ milliseconds}}$

#### SMT2 performance:

- 2 threads share execution units
- Overlap compute với memory
- Effective throughput: ~8-10 GFLOPS (1.5× boost)
- Time:  $1\text{B} \div 10\text{G} = \mathbf{\sim 100\text{ milliseconds}}$

### Comparison với GPU:

- NVIDIA A100: ~19,000 GFLOPS (FP32)
- M Core: ~10 GFLOPS
- **GPU 1900× nhanh hơn!**

### Khi nào M Core win?

- **Latency-sensitive:** M Core ~100ms, GPU ~20ms (với overhead)
- **Small batch:** Matrix 100×100 → M Core ~1ms, GPU ~5ms (launch overhead)
- **Mixed workload:** M Core chạy web server + database + AI

## 9. BANDWIDTH - BẢNG THÔNG DỮ LIỆU

### 9.1. Internal Bandwidth

#### L1 D-Cache → Registers:

- 32 B/Cycle × multiple ports (load + store)
- Giả sử 2 loads + 1 store =  $32 \times 3 = 96$  B/Cycle
- Tại 3 GHz:  $96 \times 3G = \sim 288$  GB/s

#### L2 Cache → L1 Cache:

- 32 B/Cycle bandwidth
- Tại 3 GHz:  $32 \times 3G = \sim 96$  GB/s

### 9.2. AI Workload Bandwidth Requirements

#### BERT-base inference (batch=32):

- Model size: 440MB
- Activations: ~50MB per forward pass
- **Total data movement: ~500MB**

#### Tại 100ms inference time:

- Bandwidth =  $500MB \div 0.1s = 5$  GB/s
- L2 Cache bandwidth: ~96 GB/s
- **Not bandwidth-bound! ✓**

### GPT-2 inference (batch=32):

- Model size: 6GB
- Activations: ~200MB
- **Total: ~6.2 GB**

### Tại 1s inference time:

- Bandwidth =  $6.2\text{GB} \div 1\text{s} = 6.2 \text{ GB/s}$
- Still within L2 bandwidth 

**Conclusion:** L2 bandwidth không phải bottleneck cho most AI workloads.

## 10. ỨNG DỤNG THỰC TẾ

### 10.1. AI Inference Serving

#### Scenario: Production BERT API Server Architecture:

M Core Server (64 cores × SMT2 = 128 threads)

├─ Load Balancer (NGINX)

├─ BERT Workers (64 processes, mỗi process = 1 core SMT2)

| └─ Worker 1: Thread 0 + Thread 1

| └─ Worker 2: Thread 0 + Thread 1

| └─ ...

└─ Model Cache (Shared memory, trong L2 cache)

#### Workload characteristics:

- Requests: 10,000-100,000 per second
- Latency requirement: P99 < 100ms
- Model: BERT-base (110M parameters)

#### M Core Advantages: 1. SMT2 Throughput Boost:

Without SMT:  $64 \text{ cores} \times 100 \text{ req/sec} = 6,400 \text{ req/sec}$

With SMT2:  $64 \text{ cores} \times 165 \text{ req/sec} = 10,560 \text{ req/sec}$

→ 65% throughput increase!

## 2. Large L2 Cache (16MB):

- BERT model: 440MB total
- Active 2-3 layers trong L2: ~100MB
- Cache hit rate: ~70-80%
- Latency giảm: Average 50ms → 35ms

## 3. Huge Load Buffer (211 entries):

- Weight loading overlapped
- Thread 0 load layer N, Thread 1 compute layer N-1
- Memory latency completely hidden

## 4. Large TLB (3072 entries):

- Model sử dụng huge pages (2MB)
- $440\text{MB} \div 2\text{MB} = 220 \text{ pages}$
- 3072 TLB entries cover toàn bộ model + activations
- TLB miss rate: < 1%

## Real-world benchmark:

- **Latency:**
  - P50: 25ms
  - P99: 85ms
  - P99.9: 150ms
- **Throughput:** 10,500 requests/second per server
- **Cost efficiency:** USD 0.5 per million inferences

## Vs GPU solution:

- NVIDIA A100 GPU: 50,000 req/sec
- M Core 64-core: 10,500 req/sec
- **GPU 5× nhanh hơn, NHƯNG:**
  - GPU cost: USD 10,000
  - M Core server: USD 8,000
  - GPU power: 400W
  - M Core power: 300W
  - **M Core better cost/watt for latency-sensitive workloads**

## 10.2. Computer Vision - Image Classification

### Scenario: ResNet-50 Image Classification API Workload:

- Model: ResNet-50 (25M parameters, ~100MB)
- Input: 224×224×3 images
- Batch size: 16-32 (inference)

### Pipeline:

```
# Thread 0: Process image batch 0
image_batch0 = preprocess(raw_images[0:16])
features0 = resnet50.forward(image_batch0)
predictions0 = classifier(features0)

# Thread 1: Process image batch 1 (parallel!)
image_batch1 = preprocess(raw_images[16:32])
features1 = resnet50.forward(image_batch1)
predictions1 = classifier(features1)
```

### M Core execution: Stage 1: Preprocessing (INT + FP mix)

- Resize, normalize images
- 104 INT Schedulers: indexing, loop control
- 192 FP Schedulers: normalize (divide, multiply)

- SMT2: 2 batches parallel

## Stage 2: Convolution layers (FP-heavy)

```
# Convolution: im2col + matrix multiply
for layer in conv_layers:
    output = conv2d(input, weights[layer])
    # → Decompose thành matrix multiply
    # → Millions of FP operations
```

- VMUL FP ADD units: fused multiply-add
- 447 Vector PRF: hold intermediate feature maps
- 192 FP Schedulers: schedule FP operations
- 16MB L2 Cache: fit ~4-5 ResNet layers

## Stage 3: Batch Normalization (FP)

```
# BN: output = (input - mean) / sqrt(var + eps) * gamma + beta
```

- FP DIV units: division (slow, ~30 cycles)
- Out-of-order execution: overlap divides với other ops
- 480 ROB entries: track long div operations

## Stage 4: Classification head (FP)

```
logits = fc_layer(features) # Matrix multiply
predictions = softmax(logits) # Exponentials
```

- Softmax: exponentials (transcendental functions)
- $\mu$ Code handles complex exp operations
- MOP Cache: cache exp  $\mu$ OPs (reuse for each batch)

## Performance:

- **Throughput:** ~200-300 images/second per core
- **With SMT2:** ~320-450 images/second per core
- **64-core server:** ~20,000-30,000 images/second total
- **Latency:** P99 < 50ms

#### **Vs dedicated AI accelerator:**

- Google TPU v4: ~1M images/second
- M Core: ~30K images/second
- **TPU 33× faster, BUT:**
  - TPU dedicated, M Core general-purpose
  - M Core chạy web server + database đồng thời
  - M Core lower TCO cho mixed workloads

### **10.3. Natural Language Processing - Translation**

**Scenario: Machine Translation (English → Vietnamese) Model: Transformer (similar to Google Translate)**

- Encoder: 6 layers
- Decoder: 6 layers
- Parameters: ~200M (~800MB)

#### **Workload:**

Input: "Hello, how are you today?"  
Output: "Xin chào, hôm nay bạn khỏe không?"

#### **Execution trên M Core: Step 1: Tokenization (INT-heavy)**

```
tokens = tokenizer("Hello, how are you today?")
# → [101, 7592, 1010, 2129, 2024, 2017, 2651, 1029, 102]
```

- String processing: strcmp, hash lookup



- 104 INT Schedulers xử lý
- ALU units: array indexing, hashing

## Step 2: Encoder (FP-heavy)

```
for layer in encoder_layers: # 6 layers
    # Multi-head attention
    Q = W_q @ input # Matrix multiply
    K = W_k @ input
    V = W_v @ input
    attention = softmax(Q @ K.T / sqrt(d_k))
    output = attention @ V

    # Feed-forward
    hidden = ReLU(W1 @ output)
    output = W2 @ hidden
```

### Attention mechanism breakdown:

- **Q @ K.T:** Matrix multiply (FP)
  - 192 FP Schedulers schedule operations
  - VMUL FP ADD units execute
  - 211 Load Buffer track weight loads
- **softmax:** Exponentials + division (FP)
  - FP DIV units (slow)
  - Out-of-order: start next attention head trong khi chờ
- **attention @ V:** Another matrix multiply

### SMT2 advantage:

- Thread 0: Compute attention head 0, 2, 4, ...
- Thread 1: Compute attention head 1, 3, 5, ...
- Parallel execution, share weights (trong L2 cache)

### Step 3: Decoder (FP-heavy, autoregressive)

```
output_tokens = []
for position in range(max_length):
    # Decoder attention (similar to encoder)
    # + cross-attention (attend to encoder output)
    token = decoder.forward(output_tokens, encoder_output)
    output_tokens.append(token)
    if token == END_TOKEN:
        break
```

#### Autoregressive = sequential:

- Cannot parallelize across positions
- BUT can parallelize within each position (multi-head attention)
- SMT2: Thread 0 work on position N, Thread 1 prefetch for position N+1

### Step 4: Token generation

```
logits = lm_head(decoder_output) # 50K vocab size
token = argmax(logits)
word = vocab[token]
```

#### Performance:

- **Throughput:** ~50-80 sentences/second per core
- **SMT2 boost:** ~80-120 sentences/second per core
- **64-core server:** ~5,000-7,500 sentences/second
- **Latency:** P99 < 200ms (for average sentence ~10 words)

#### Challenge: Autoregressive nature limits SMT2 benefit

- Decoder sequential → less parallelism

- SMT2 boost:  $\sim 1.5\times$  (vs  $1.7\times$  for encoder-only BERT)
- Still beneficial for serving multiple requests

## 10.4. Recommendation Systems

### Scenario: E-commerce Product Recommendations Model: Deep Learning Recommendation Model (DLRM)

- User features: demographics, history
- Item features: category, price, ratings
- Embedding tables: 100M+ entries
- MLP (Multi-Layer Perceptron): 5 layers

#### Architecture:

Request: User ID 12345 browsing category "Electronics"

↓

[Embedding Lookup] → [Feature Interaction] → [MLP] → [Top-K Selection]

#### M Core execution: Stage 1: Embedding Lookup (Memory-intensive!)

```
user_embedding = user_table[user_id] # Load 1
item_embedding = item_table[item_id] # Load 2
category_embedding = category_table[category_id] # Load 3
# → Multiple dependent loads
```

#### Memory access pattern:

- Random access (hash table lookups)
- Poor cache locality
- **Heavy TLB pressure:**
  - $100\text{M entries} \times 128\text{ bytes/entry} = \sim 12\text{GB}$
  - 3072 L2 TLB entries critical!

- With 2MB huge pages:  $3072 \times 2\text{MB} = 6\text{GB}$  coverage
- TLB hit rate: ~50-60% (decent for this workload)

### 211 Load Buffer advantage:

- Speculative prefetch: predict next embedding lookups
- Multiple lookups in flight: ~50-100 concurrent
- Hide memory latency (DRAM ~200 cycles)

### SMT2 advantage:

Thread 0: Lookup embeddings for User A

Thread 1: Lookup embeddings for User B

→ While Thread 0 waits DRAM, Thread 1 uses execution units

### Stage 2: Feature Interaction (FP operations)

# Dot product interactions

interactions = []

for i in range(num\_features):

    for j in range(i+1, num\_features):

        interactions.append(dot(embeddings[i], embeddings[j]))

- FP multiply-add operations
- VMUL FP ADD units
- 192 FP Schedulers

### Stage 3: MLP Forward (FP-heavy)

hidden = embeddings

for layer in mlp\_layers:

    hidden = ReLU(W[layer] @ hidden + b[layer])

- Matrix multiply + activation
- Similar to image classification
- 16MB L2 Cache fits MLP weights

#### Stage 4: Top-K Selection (INT + FP mix)

```
scores = model.forward(features) # FP
top_k_indices = argsort(scores)[:k] # INT sorting
recommendations = [items[i] for i in top_k_indices]
```

#### Performance:

- **Throughput:** ~5,000-8,000 recommendations/second per core
- **SMT2:** ~8,000-13,000 recommendations/second per core
- **64-core server:** ~500K-800K recommendations/second
- **Latency:** P99 < 20ms

#### Real-world impact:

- **Amazon-scale:** Billions of requests/day
- **1000 M Core servers:** ~800 billion recommendations/day ✓
- **Cost:** ~USD 0.01 per 1000 recommendations

## 10.5. Speech Recognition

**Scenario: Real-time Speech-to-Text (ASR) Model: Transformer-based ASR (similar to Whisper)**

- Encoder: Audio features → latent representation
- Decoder: Latent → text tokens

**Input:** 10-second audio clip (16kHz, 160,000 samples) **Pipeline: Stage 1: Feature Extraction (FP-heavy)**

```
# Convert audio waveform → spectrogram
fft_output = fft(audio_samples) # Fast Fourier Transform
mel_spectrogram = mel_filterbank @ abs(fft_output)**2
log_mel = log(mel_spectrogram + eps)
```

### **FFT computation:**

- Complex FP multiply-add operations
- Butterfly pattern (FFT algorithm)
- Out-of-order execution helps (many independent butterflies)
- VMUL FP ADD units
- 192 FP Schedulers

### **µCode for transcendental functions:**

- `log()` decomposed into µOPs
- MOP Cache reuse for each frame

### **Stage 2: Encoder (Similar to BERT)**

- 160,000 samples ÷ 160 (frame size) = 1000 frames
- Encoder processes 1000 frames
- Attention mechanism (FP-heavy)
- 16MB L2 Cache fits encoder layers

### **Stage 3: Decoder (Autoregressive)**

- Generate text tokens one-by-one
- Similar to translation decoder
- SMT2: Limited benefit (sequential)

### **Performance:**

- **Throughput:** ~50-80 seconds of audio per second (0.5-0.8× realtime per core)
- **64-core server:** ~3,200-5,120 seconds audio/second (~50-80× realtime)

- **Latency:** ~500ms for 10-second clip

**SMT2 benefit:** ~1.4× (less than other workloads due to sequential decoder) **Use case:**

- Real-time transcription for video conferencing
- 64-core server supports ~3,000-5,000 concurrent users

## 10.6. Fraud Detection (Real-time)

**Scenario: Credit Card Fraud Detection Model: Gradient Boosted Trees + Neural Network Ensemble** **Input:** Transaction features

```
features = {  
    'amount': 125.50,  
    'merchant_category': 'Electronics',  
    'time': '2026-02-04 10:30',  
    'location': 'San Francisco',  
    'user_history': [...],  
    'device_id': '...',  
}
```

**Execution: Stage 1: Feature Engineering (INT + FP mix)**

```
# Time features (INT)  
hour = extract_hour(time) # INT operations  
day_of_week = extract_dow(time)  
  
# Aggregations (FP)  
avg_transaction_24h = sum(history_24h) / count # FP divide  
stddev = sqrt(sum((x - mean)**2) / count) # FP sqrt  
  
# Categorical encoding (INT)  
merchant_code = hash(merchant_category) % TABLE_SIZE
```

- 104 INT Schedulers: time parsing, hashing
- 192 FP Schedulers: statistics
- FP DIV: division (slow)
- Out-of-order: overlap divides

## Stage 2: Gradient Boosted Trees (INT-heavy)

```
score = 0
for tree in trees: # 1000 trees
    node = tree.root
    while not node.is_leaf:
        if features[node.feature] < node.threshold:
            node = node.left
        else:
            node = node.right
    score += node.value
```

### Tree traversal:

- Lots of branches (if-else)
- Branch predictor critical!
- M Core branch predictor: ~95-98% accuracy
- Misprediction penalty: ~15-20 cycles
- With good prediction: ~3-5 cycles per node

**1000 trees × 10 depth = ~10,000 nodes**

- With perfect prediction: ~30,000-50,000 cycles
- At 3 GHz: ~10-17 microseconds

### SMT2 advantage:

- Thread 0: Evaluate transaction A
- Thread 1: Evaluate transaction B



- Share tree structure (trong L2 cache)
- Hide branch mispredictions

### Stage 3: Neural Network (FP-heavy)

```
# MLP: 5 layers
hidden = features
for layer in layers:
    hidden = ReLU(W[layer] @ hidden + b[layer])
fraud_probability = sigmoid(hidden)
```

- Matrix multiply (small:  $\sim 100 \times 100$ )
- VMUL FP ADD units
- Fast (weights fit in L1 cache)

### Stage 4: Ensemble (FP)

```
final_score = 0.6 * tree_score + 0.4 * nn_score
fraud_decision = (final_score > threshold)
```

### Performance:

- **Latency:** P99 < 5ms (critical for real-time)
- **Throughput:**  $\sim 50,000$ - $100,000$  transactions/second per core
- **SMT2:**  $\sim 80,000$ - $160,000$  transactions/second per core
- **64-core server:**  $\sim 5$ - $10$  million transactions/second

### Real-world scale:

- **Visa processes:**  $\sim 65,000$  transactions/second globally
- **1 M Core server:** Đủ xử lý toàn bộ Visa global traffic! 

## 10.7. AI-Powered Search (Semantic Search)

**Scenario: E-commerce Semantic Search Query:** "comfortable running shoes for marathon" **Model: BERT-based Sentence Embeddings Pipeline: Stage 1: Query Encoding**

```
query_embedding = bert_encode("comfortable running shoes for marathon")  
# → 768-dimensional vector
```

- BERT inference (đã analyze ở trên)
- Output: dense vector representation

### **Stage 2: Vector Similarity Search (FP-heavy)**

```
# Database: 10 million products, each có 768-dim embedding  
scores = []  
for product_embedding in database: # 10M iterations!  
    score = cosine_similarity(query_embedding, product_embedding)  
    # score = dot(query, product) / (norm(query) * norm(product))  
    scores.append(score)
```

### **Cosine similarity computation:**

```
numerator = sum(query[i] * product[i] for i in range(768)) # Dot product  
denominator = sqrt(sum(query[i]**2)) * sqrt(sum(product[i]**2)) # Norms  
score = numerator / denominator
```

**10M products × 768 dimensions = 7.68 billion FP operations! M Core optimization: 1.**

### **Vectorized dot product (VMUL FP ADD):**

- Process 4-8 dimensions per operation
- $768 \text{ dims} \div 4 = \sim 200$  operations per product
- $10\text{M products} \times 200 \text{ ops} = 2 \text{ billion operations}$

## 2. Out-of-order execution (480 ROB):

- Prefetch next product embeddings
- Overlap load với compute
- 211 Load Buffer track many loads

## 3. SMT2:

- Thread 0: Products 0-5M
- Thread 1: Products 5M-10M
- Parallel search!
- Share query embedding (trong registers/L1 cache)

## 4. Large L2 Cache (16MB):

- $16\text{MB} \div (768 \times 4 \text{ bytes}) = \sim 5,300$  product embeddings
- Cache hot products (popular items)
- Cache hit rate:  $\sim 30\text{-}40\%$

## Stage 3: Top-K Selection (INT + FP)

```
top_k = heapq.nlargest(100, scores)
```

- Heap operations: INT comparisons
- 104 INT Schedulers

## Performance:

- **Latency:**  $\sim 50\text{-}100\text{ms}$  for 10M products
- **Throughput:**  $\sim 10\text{-}20$  searches/second per core
- **SMT2:**  $\sim 15\text{-}35$  searches/second per core
- **64-core server:**  $\sim 1,000\text{-}2,000$  searches/second

## Optimization: Approximate Search (HNSW, FAISS):

- Reduce 10M to  $\sim 1,000$  candidates

- Latency: ~5-10ms
- Throughput: ~100-200 searches/second per core

## 10.8. Time Series Forecasting

**Scenario: Stock Price Prediction Model: LSTM (Long Short-Term Memory) Input:** 100 days of historical prices → Predict next day **LSTM Cell computation:**

```
# For each timestep t:
f_t = sigmoid(W_f @ [h_{t-1}, x_t] + b_f) # Forget gate
i_t = sigmoid(W_i @ [h_{t-1}, x_t] + b_i) # Input gate
c_tilde = tanh(W_c @ [h_{t-1}, x_t] + b_c) # Candidate
c_t = f_t * c_{t-1} + i_t * c_tilde # Cell state
o_t = sigmoid(W_o @ [h_{t-1}, x_t] + b_o) # Output gate
h_t = o_t * tanh(c_t) # Hidden state
```

### Per timestep operations:

- 4 matrix multiplies ( $W_f$ ,  $W_i$ ,  $W_c$ ,  $W_o$ )
- 3 sigmoids + 2 tanhs (transcendental functions)
- 3 element-wise multiplies
- 1 addition

### M Core execution: 1. Matrix multiplies (FP-heavy):

- VMUL FP ADD units
- 192 FP Schedulers
- Small matrices (~256×256) → fit in L1 cache

### 2. Transcendental functions (μCode):

- sigmoid, tanh decomposed into μOPs
- Polynomial approximations
- MOP Cache reuse (same functions repeated)

### 3. Sequential dependencies:

- $h_t$  depends on  $h_{t-1} \rightarrow$  cannot parallelize timesteps
- BUT can parallelize across batch dimension
- SMT2: Process 2 stocks simultaneously

### 4. 100 timesteps = Long dependency chain:

- 480 ROB entries track ~50-100 operations
- Out-of-order helps within each timestep

### Performance:

- **Latency:** ~10-20ms per sequence (100 timesteps)
- **Throughput:** ~50-100 sequences/second per core
- **SMT2:** ~75-150 sequences/second per core
- **64-core server:** ~5K-10K sequences/second

### Real-world application:

- Forecast 10,000 stocks simultaneously
- Update every minute
- 64-core M Core server sufficient 

## 10.9. Molecular Dynamics Simulation (AI-Powered)

**Scenario: Drug Discovery - Protein Folding Prediction Model: AlphaFold-style**

**Transformer Input:** Protein sequence (1000 amino acids) **Output:** 3D structure (1000 atoms  $\times$  3 coordinates) **Computation: Stage 1: Multiple Sequence Alignment (MSA)**

### Processing

```
# MSA: 1000 sequences  $\times$  1000 positions
```

```
msa_embedding = transformer_encoder(msa)
```

- Attention over  $1000 \times 1000 = 1M$  pairs

- Attention score matrix:  $1M \times 1M = 1$  trillion elements!
- **Sparse attention required** (reduce to  $\sim 100M$  operations)

## Stage 2: Structure Module (FP + Geometry)

```
# Predict atom coordinates
coords = structure_module(msa_embedding)
# Compute distances, angles, torsions
distances = compute_distances(coords) #  $O(N^2)$ 
```

### Geometry computation (FP-heavy):

```
# Distance between atoms i and j
dx = coords[i][0] - coords[j][0]
dy = coords[i][1] - coords[j][1]
dz = coords[i][2] - coords[j][2]
distance = sqrt(dx**2 + dy**2 + dz**2)
```

- FP subtract, multiply, add, sqrt
- $1000 \text{ atoms} \times 1000 = 1M$  distances
- VMUL FP ADD + FP DIV (sqrt) units

### M Core advantages: 1. Large FP resources:

- 447 Vector PRF hold coordinates
- 192 FP Schedulers schedule operations
- VMUL FP ADD for multiply-add

### 2. Out-of-order (480 ROB):

- sqrt latency  $\sim 30$  cycles
- Start next distance computation trong khi chờ
- 480 ROB  $\rightarrow \sim 100+$  sqrt operations in flight

### 3. SMT2:

- Thread 0: Compute distances 0-500K
- Thread 1: Compute distances 500K-1M
- Parallel execution

### Performance:

- **Latency:** ~10-30 seconds per protein (1000 amino acids)
- **Throughput:** ~2-6 proteins/minute per core
- **64-core server:** ~130-400 proteins/minute

### Comparison:

- GPU (A100): ~10-20 proteins/minute (single GPU)
- M Core better for throughput with many small proteins

## 10.10. AI Model Training (Limited)

### Scenario: Fine-tuning BERT on M Core Workload:

- Pre-trained BERT-base
- Fine-tune on custom dataset (100K examples)
- 3 epochs

### Training loop:

```
for epoch in range(3):  
    for batch in dataloader: # 100K ÷ 32 = 3,125 batches  
        # Forward pass (same as inference)  
        output = model.forward(batch)  
        loss = criterion(output, labels)  
  
        # Backward pass (NEW!)  
        loss.backward() # Compute gradients
```

```
# Optimizer step (NEW!)  
optimizer.step() # Update weights
```

### Backward pass computation:

```
# Gradient of loss w.r.t each weight  
for layer in reversed(model.layers):  
    grad_output = compute_gradient(layer, grad_input)  
    layer.weight.grad = grad_output @ input.T # Matrix multiply
```

**M Core execution: Forward pass:** ~35ms (analyzed earlier) **Backward pass:** ~50-70ms

- More memory-intensive (store activations)
- More FP operations (gradient computations)
- 211 Load Buffer helps load activations
- 76 Store Buffer stores gradients

**Optimizer step:** ~5-10ms

```
# SGD with momentum  
weight = weight - learning_rate * gradient
```

- Element-wise FP operations
- VMUL FP ADD units
- Fast (weights in L2 cache)

**Total per batch:** ~90-115ms **Full training time:**

- 3,125 batches × 3 epochs = 9,375 batches
- 9,375 × 100ms = **937 seconds ≈ 15 minutes**

**Comparison với GPU:**



- A100 GPU: ~2-3 minutes
- M Core: ~15 minutes
- **GPU 5-7× nhanh hơn**

#### Khi nào M Core training có ý nghĩa?

- **Small-scale fine-tuning:** < 1M examples
- **Edge deployment:** Train on-device
- **Cost-sensitive:** Reuse inference servers for training
- **Mixed workload:** 80% inference + 20% training

## 11. TỔNG KẾT VÀ KẾT LUẬN

### 11.1. Điểm Mạnh Của M Core

#### 1. SMT2 Architecture:

-  **1.5-1.7× throughput boost** cho most AI workloads
-  Hide memory latency effectively
-  Better utilization than single-thread
-  Không quá complex như SMT4/SMT8

#### 2. AI-Optimized Resources:

-  **447 Vector PRF:** Massive FP register file
-  **192 FP Schedulers:** Handle FP-heavy AI ops
-  **211 Load Buffer:** Extreme load-heavy workloads (AI weights)
-  **16MB L2 Cache:** Fit multiple model layers

#### 3. Memory Subsystem:

-  **3072 L2 TLB entries:** Support large AI models (with huge pages)
-  **128KB L1 D-Cache:** Hold intermediate activations
-  **Huge pages support:** Critical for 100MB-10GB models

#### 4. Enterprise Heritage (IBM Power Z15):

-  **Proven architecture:** Separated PRF, large ROB
-  **RAS features:** Reliability for production AI servers
-  **High clock potential:** 3-4 GHz for low latency

#### 5. Balanced Design:

-  **8  $\mu$ OPs decode:** Wide enough for throughput
-  **480 ROB:** Hide long AI operation latencies
-  **104 INT + 192 FP schedulers:** Handle mixed workloads

## 11.2. Điểm Yếu Của M Core

#### 1. Separated PRF (vs Unified):

-  Less flexible than unified design
-  If workload unexpectedly INT-heavy, 423 INT PRF may not suffice
-  Modern trend moving to unified (Neoverse V3, Intel Granite Rapids)

#### 2. SMT2 Contention:

-  2 threads compete for caches, TLBs, execution units
-  Worst-case: 2 threads cùng workload  $\rightarrow$  0% benefit
-  Need careful thread scheduling

#### 3. Not as Wide as Competition:

-  Decode 8 vs Neoverse V3 10+
-  ROB 480 vs ARC Core 600
-  Lower peak single-thread performance

#### 4. Missing AI Accelerators:

-  No tensor cores (NVIDIA GPU có)
-  No matrix engines (Intel AMX, ARM SME)

- ❌ Pure CPU approach → slower than dedicated accelerators

## 5. Training Performance:

- ❌ 5-10× chậm hơn GPUs
- ❌ Not competitive for large-scale training
- ❌ OK cho inference, limited cho training

## 11.3. M Core vs Competition

### 11.3.1. M Core vs N Core

| Aspect        | M Core    | N Core       | Winner                   |
|---------------|-----------|--------------|--------------------------|
| Target        | AI Server | Cloud Server | Tie                      |
| SMT           | SMT2      | None         | M Core (+50% throughput) |
| PRF Size      | 870 (sep) | 250 (uni)    | M Core (capacity)        |
| PRF Design    | Separated | Unified      | N Core (flexibility)     |
| FP Schedulers | 192       | 160          | M Core                   |

|                                     |           |             |                |
|-------------------------------------|-----------|-------------|----------------|
| <b>Load Buffer</b>                  | 211       | 96          | M Core (2.2×!) |
| <b>L2 Cache</b>                     | 16MB      | 3MB         | M Core (5×!)   |
| <b>L2 TLB</b>                       | 3072      | 2048        | M Core         |
| <b>Decode Width</b>                 | 8         | 6           | M Core         |
| <b>AI Workload</b>                  | Excellent | Good        | <b>M Core</b>  |
| <b>General Server</b>               | Good      | Excellent   | <b>N Core</b>  |
| Aspect                              | M Core    | NVIDIA A100 | Winner         |
| <b>AI Training</b>                  | OK        | Excellent   | GPU (10×)      |
| <b>AI Inference<br/>(batch=256)</b> | Good      | Excellent   | GPU (5×)       |

|                                   |               |              |                        |
|-----------------------------------|---------------|--------------|------------------------|
| <b>AI Inference<br/>(batch=1)</b> | Excellent     | Good         | M Core (lower latency) |
| <b>Latency</b>                    | ~10-50ms      | ~20-100ms    | M Core                 |
| <b>Throughput</b>                 | ~10K req/s    | ~50K req/s   | GPU                    |
| <b>Mixed Workload</b>             | Excellent     | Poor         | M Core                 |
| <b>Cost</b>                       | USD 8K server | USD 10K card | M Core                 |
| <b>Power</b>                      | 300W          | 400W         | M Core                 |
| <b>Flexibility</b>                | General CPU   | Specialized  | M Core                 |
| <b>Aspect</b>                     | M Core        | Intel Xeon   | Winner                 |
| <b>SMT</b>                        | SMT2          | SMT2 (HT)    | Tie                    |

|                       |           |              |                   |
|-----------------------|-----------|--------------|-------------------|
| <b>Decode Width</b>   | 8         | 6            | M Core            |
| <b>L2 Cache</b>       | 16MB      | 2MB          | M Core (8×!)      |
| <b>AI Accelerator</b> | None      | AMX (matrix) | Intel             |
| <b>FP Performance</b> | Excellent | Good         | M Core            |
| <b>ISA</b>            | Custom?   | x86          | Intel (ecosystem) |
| <b>Software</b>       | Limited?  | Vast         | Intel             |
| Aspect                | M Core    | IBM Power10  | Relationship      |
| <b>SMT</b>            | SMT2      | SMT8         | Power10 higher    |
| <b>Decode Width</b>   | 8         | ~8           | Similar           |

|                   |           |                  |                      |
|-------------------|-----------|------------------|----------------------|
| <b>PRF Design</b> | Separated | Separated        | Same (heritage!)     |
| <b>L2 Cache</b>   | 16MB      | 2MB (+ 32MB L3)  | Different strategy   |
| <b>Target</b>     | AI Server | Enterprise       | Different            |
| <b>Heritage</b>   | Power Z15 | Power9 → Power10 | M Core = Z15-derived |

**Conclusion:** M Core >> N Core cho AI, N Core > M Core cho general server.

### 11.3.2. M Core vs NVIDIA GPU

**Use cases:**

- **M Core wins:** Latency-sensitive inference, mixed workloads, cost-sensitive
- **GPU wins:** Training, batch inference, pure AI servers

### 11.3.3. M Core vs Intel Xeon (Sapphire Rapids)

| Aspect                | M Core    | Intel Xeon   | Winner            |
|-----------------------|-----------|--------------|-------------------|
| <b>SMT</b>            | SMT2      | SMT2         | Tie               |
| <b>Decode Width</b>   | 8         | 6            | M Core            |
| <b>L2 Cache</b>       | 16MB      | 2MB          | M Core (8×!)      |
| <b>AI Accelerator</b> | None      | AMX (matrix) | Intel             |
| <b>FP Performance</b> | Excellent | Good         | M Core            |
| <b>ISA</b>            | Custom?   | x86          | Intel (ecosystem) |
| <b>Software</b>       | Limited?  | Vast         | Intel             |

## Key question: M Core ISA?

- If x86: Can compete directly
- If custom ISA: Ecosystem problem (like Cell/SPE)

### 11.3.4. M Core vs IBM Power10

| Aspect       | M Core    | IBM Power10      | Relationship         |
|--------------|-----------|------------------|----------------------|
| SMT          | SMT2      | SMT8             | Power10 higher       |
| Decode Width | 8         | ~8               | Similar              |
| PRF Design   | Separated | Separated        | Same (heritage!)     |
| L2 Cache     | 16MB      | 2MB (+ 32MB L3)  | Different strategy   |
| Target       | AI Server | Enterprise       | Different            |
| Heritage     | Power Z15 | Power9 → Power10 | M Core = Z15-derived |

### M Core = AI-specialized fork of Power architecture

- Take Power Z15 design
- SMT8 → SMT2 (reduce complexity)
- Optimize for AI (large L2, huge load buffer)
- Add AI-specific features

## 11.4. Market Positioning 2026

### AI Server Market Segments: 1. Cloud AI (Inference):

- **Leaders:** NVIDIA GPUs, AWS Graviton (ARM), Google TPU
- **M Core opportunity:** Cost-effective alternative
- **Challenge:** Ecosystem (software, frameworks)

### 2. Edge AI (Low Latency):

- **Leaders:** NVIDIA Jetson, Intel Movidius
- **M Core opportunity:** Low-latency inference (< 10ms)
- **Advantage:** SMT2 + large caches

### 3. Hybrid Workloads (AI + Traditional):

- **Leaders:** Intel Xeon, AMD EPYC



- **M Core opportunity: BEST FIT!**
- **Use case:** Web server + database + AI inference on same machine

#### 4. AI Training:

- **Leaders:** NVIDIA A100/H100, Google TPU
- **M Core:** Not competitive
- **Accept limitation**

### 11.5. Path Forward: M Core v2 Needs

To compete với next-gen AI servers (2027-2028), M Core needs: 1. Unified PRF:

- 870 registers unified (not separated)
- Follow industry trend
- Better flexibility

#### 2. Dedicated AI Units:

- Matrix multiply accelerators (like Intel AMX)
- BF16/FP16 support (lower precision for AI)
- Throughput boost: 2-4×

#### 3. SMT4 Option:

- Make SMT level configurable: SMT2 or SMT4
- SMT2 for latency
- SMT4 for throughput

#### 4. Larger L3 Cache:

- 16MB L2 + 64MB shared L3
- Fit entire small models (BERT-base)

#### 5. DDR5/HBM Support:

- Memory bandwidth critical
- DDR5: ~400 GB/s
- HBM: ~2 TB/s (expensive, but worth for AI)

## **6. Better Branch Prediction:**

- Neural branch predictor (like Apple M-series)
- Critical for control-heavy AI code

## **11.6. Bài Học Từ M Core**

### **Lesson 1: SMT Still Relevant:**

- Industry moving away from SMT (Apple no SMT, AMD considering removing)
- BUT for server/AI, SMT still valuable (hide latency)
- M Core SMT2: 50-70% throughput boost proven

### **Lesson 2: Specialize or Die:**

- General-purpose CPU không đủ cho AI
- Need specialization: large buffers, FP focus, huge caches
- M Core did this right

### **Lesson 3: Memory > Compute:**

- AI bottleneck: memory bandwidth, not compute
- M Core: 211 Load Buffer, 16MB L2, 3072 TLB
- This is the right focus

### **Lesson 4: Heritage Can Help:**

- Starting from Power Z15 reduced risk
- Proven design, less bugs
- But need to adapt (Z15 enterprise → M Core AI)

### **Lesson 5: Ecosystem Matters:**

- M Core hardware great, BUT software?
- If custom ISA → struggle (like Cell)
- If x86/ARM → better chance

## 11.7. Kết Luận Cuối Cùng

**M Core là một thiết kế AI server CPU xuất sắc** với những đặc điểm nổi bật: **Core strengths:**

-  SMT2 với dual issue queues → throughput boost
-  Massive FP resources (447 Vector PRF, 192 FP Schedulers)
-  AI-optimized memory (211 Load Buffer, 16MB L2, 3072 TLB)
-  IBM Power heritage (reliability, proven design)
-  Balanced for hybrid workloads

**Best use cases:**

- AI inference serving (latency-sensitive, < 50ms)
- Hybrid servers (web + database + AI)
- Edge AI (on-premise inference)
- Small-scale fine-tuning

**Not ideal for:**

- Large-scale AI training (GPU wins 10×)
- Batch inference (GPU more efficient)
- Pure compute-bound (missing tensor cores)

**Market outlook:**

- **If x86 ISA:** Strong competitor to Intel Xeon for AI workloads
- **If ARM ISA:** Compete with Neoverse V2/V3
- **If custom ISA:** Ecosystem challenge (need software support)

**Final verdict:** M Core đại diện cho một hướng đi đúng: **Adapt proven enterprise CPU architecture (Power Z15) cho AI workloads với targeted optimizations (SMT2, large**

**buffers, huge caches).** Trong thị trường AI 2026, M Core có chỗ đứng rõ ràng cho **latency-sensitive inference và hybrid workloads**, mặc dù không thể cạnh tranh với GPUs cho training. **Thành công phụ thuộc vào:**

1. ISA choice (x86/ARM vs custom)
2. Software ecosystem (PyTorch, TensorFlow optimization)
3. Manufacturing (5nm/3nm process)
4. Market timing & pricing

**M Core = Right design for the right workload. Not fastest, not most specialized, but balanced và practical cho real-world AI servers. THE END.** *M Core là minh chứng cho triết lý: "Specialize where it matters (FP, memory), inherit where it's proven (Power Z15 architecture), simplify where possible (SMT2 not SMT8)". Trong thế giới AI đang bùng nổ, một CPU như M Core - không phải fastest, không phải cheapest, nhưng balanced và reliable - vẫn có giá trị lớn.*